



0110011100011100100111000111001001110001

Algebraic
Multigrid Methods
for Systems

User's Manual

SAMG, Release 26a1
serial & OpenMP parallel

Document version 1.0
January 2012

Service & Technical Support
samg@scai.fraunhofer.de

Fraunhofer Institute SCAI
Schloss Birlinghoven
53754 Sankt Augustin
Germany



Fraunhofer
SCAI

Purpose of this document

This document describes all features of the **serial SAMG library** including **OpenMP**-based parallel features. However, it does not include features of the MPI-based parallel version of SAMG, called SAMGp. This document also does not cover any help for installing the software. Additional documents are available covering these aspects.

Please check for further documents containing additional and/or most recent information. Additional SAMG-related documents, papers and reports can be downloaded from our Web page

<http://www.scai.fraunhofer.de/samg>.

Regarding technical support, questions, suggestions and the like, do not hesitate to contact us via our service e-mail,

samg@scai.fraunhofer.de¹.

Your SAMG Team

Remarks on highlighting

We use the following symbols for highlighting:

			These symbols are used to draw the reader's attention to particularly important aspects/remarks of this document (order given by increasing importance)
			This symbol marks features which are either not yet installed in the current release or which are in a test phase and not yet supposed to be selected by the average user.

Colored text is used for highlighting as follows:

- Generally, the color brown highlights headlines and important aspects.
- Examples, computer output, program code, program and file names are printed in blue.
- Text printed in green refers to options or features which are either for testing/debugging or under development. In any case, such options or features are not yet released for general use.

¹ **Important:** This e-mail addresses the whole SAMG Team. Therefore, it should not be used to transmit MBytes of data.

1	Introduction and Overview	8
1.1	Review of SAMG	8
1.1.1	Hierarchical solvers	8
1.1.2	Algebraic multigrid (AMG)	8
1.1.3	The SAMG package	8
1.2	Overview of this manual	9
2	Background Information	10
2.1	The structure of SAMG	10
2.2	The performance of SAMG	11
2.3	Features of SAMG, an overview	11
2.3.1	General features	11
2.3.2	More specific features	13
2.4	General remarks on SAMG parameters	14
2.4.1	Type of parameters: primary and hidden	14
2.4.2	Naming convention	15
2.4.3	Multi-purpose use of single parameters	15
2.5	Automatic memory management	16
3	Solution Approaches	17
3.1	Variable-based approach	17
3.2	Unknown-based approach	18
3.3	Point-based approaches	18
3.3.1	The coarsening process	19
3.3.2	The interpolation	20
4	Data Structure	21
4.1	The linear system of equations	21
4.2	Additional data in case of coupled systems	22
4.3	Formal data restrictions	23
5	Calling Sequence: The Primary Parameters	24
5.1	Passing the linear system of equations	24
5.2	First approximation and stopping criterion	25
5.3	Distinguishing scalar and coupled systems	26
5.4	Selecting the solution approach	27
5.4.1	The solution strategy	27
5.4.2	The smoothing strategy	28
5.4.3	Specifying a primary matrix (for point-based approaches)	29
5.4.4	Specifying details of interpolation (all solution approaches)	31
5.4.5	Summary	33
5.5	Selecting SAMG's cycling process	35
5.5.1	One-level solvers	36
5.5.2	Modified cycles (*-cycles)	36
5.5.3	Choice of accelerator	36
5.5.4	Stopping criteria	37
5.5.5	Number of smoothing steps	37
5.5.6	Smoothing by relaxation	37
5.5.7	Smoothing on coarser levels	37

5.5.8	Type of coarsening and interpolation	37
5.5.9	Termination of coarsening	37
5.5.10	Coarsest-level solver	38
5.5.11	Pre-cycling	38
5.6	General control switch	39
5.6.1	Repeated calls to SAMG, reusing previous setup data	39
5.6.2	Memory extension switch	40
5.6.3	Selecting a coarsening strategy via the primary parameters	41
5.6.4	Norms and scratch files	42
5.6.5	Displaying histories of errors	42
5.7	Initial dimensioning	42
5.7.1	Automatic adaptation of the work space	42
5.7.2	Explicitly setting the dimensioning parameters	43
5.7.3	Dynamic adaptation at run time	43
5.8	Checking and printing	43
5.8.1	Input matrix checking	44
5.8.2	Print output during the solution phase	44
5.8.3	Print output during the setup phase	45
5.9	Output parameters	46
5.10	Best practice	47
5.10.1	Logfile and other output	47
5.10.2	Checking the error parameter, <i>ierr</i>	47
5.10.3	Checking the number of cycles performed, <i>ncyc_done</i>	47
5.10.4	Unintentional use of one-level methods	48
5.10.4.1	Solving very small problems	48
5.10.4.2	Changing SAMG's one-level default behavior	48
5.10.4.3	Manually checking the number of levels created	49
5.10.4.4	A practical approach	49
5.10.5	Memory allocation	49
6	Hidden Parameters	50
6.1	Accessing hidden parameters: set- and get-routines	50
6.2	Parameters related to SAMG's setup phase	52
6.2.1	Important threshold values used in coarsening	52
6.2.2	Treatment of positive off-diagonal matrix entries in coarsening	53
6.2.3	Type of coarsening	54
6.2.3.1	Classical or cluster coarsening	54
6.2.3.2	Special aspects regarding classical coarsening	55
6.2.3.3	Special aspects regarding cluster coarsening	56
6.2.4	Termination criteria for the coarsening process	57
6.2.5	Interpolation	58
6.2.6	Special parameters for point-based approaches	60
6.2.7	The Galerkin coarse-level matrices	61
6.2.8	Repeated calls to SAMG, reusing previous setup data	62
6.2.9	Truncation features	63
6.3	Parameters related to SAMG's solution phase	64
6.3.1	The smoothing process	64
6.3.2	Further parameters relevant for smoothing	65
6.3.3	Acceleration and modified multigrid cycles (*-cycles)	66
6.3.4	The coarsest-level (CL) solver	67
6.3.4.1	Selecting the coarsest-level solver	67

6.3.4.2	Important coarsest-level parameters	68
6.3.4.3	Less important coarsest-level parameters	68
6.3.4.4	Some remarks and hints	69
6.4	Further parameters	70
6.4.1	Re-directing and limiting I/O	70
6.4.2	Various default values	71
6.4.3	Some special parameters	72
6.4.4	Description of the default switch <code>n_default</code>	72
7	Special SAMG Features	74
7.1	OpenMP parallelization	74
7.1.1	General remarks	74
7.1.2	Best practice	76
7.1.2.1	“First touch” policy	76
7.1.2.2	Race conditions	77
7.1.2.3	Hardware aspects	77
7.1.2.4	Mixed aspects	78
7.1.2.5	Changing number of threads between SAMG calls	78
7.1.3	Components not yet parallelized	78
7.2	Stopping criteria for the iterative solution process	79
7.2.1	Convergence criteria	79
7.2.2	Safety checks	80
7.2.3	Best practice	82
7.2.3.1	Quasi-residual check	82
7.2.3.2	Residual stagnation check	82
7.2.3.3	Approximation criterion	82
7.3	Providing grid coordinates	83
7.4	Treatment of “special” variables	84
7.4.1	Marking variables as being special	84
7.4.2	Forcing special variables into coarse or fine levels	85
7.4.2.1	Forcing variables into <i>coarse</i> levels	85
7.4.2.2	Forcing variables to remain on the <i>finest</i> level	85
7.4.2.3	Summary	86
7.4.2.4	Parameters to cover some frequent cases	86
7.4.3	Alternating Schwarz method	87
7.4.4	Improved alternating Schwarz method (with overlap)	88
7.4.4.1	The most general case	88
7.4.4.2	Two extreme cases for demonstration	89
7.4.4.3	Automatic setting of an overlap	90
7.4.5	Additional features and concluding remarks	91
7.5	Automatic solver control	92
7.5.1	Typical scenario for the solver control	92
7.5.1.1	Primary solver control (“re-using setup data”)	92
7.5.1.2	Secondary solver control (“solver switching”)	93
7.5.2	Recall: Controlling the re-use of setup data manually	93
7.5.3	The primary solver control: re-using setup data	94
7.5.4	The secondary solver control: solver switching	95
7.5.5	General control parameters	96
7.5.6	Outline of the solver switching	97
7.5.7	Exemplary templates	99

7.6	One-level solvers	101
7.6.1	Intentional use of one-level solvers	102
7.6.1.1	Using any of the smoothers as one-level solver (<i>clsolver_finetest=0</i>)	102
7.6.1.2	Using any of the CL solvers as one-level solver (<i>clsolver_finetest≠0</i>)	103
7.6.1.3	Examples	104
7.6.2	Unintentional use of one-level solvers	105
7.7	More one-level methods	106
7.7.1	Diagonally pre-conditioned CG, GMRes or BiCGstab	106
7.7.2	ICCG for symmetric matrices	106
7.8	Controlling special output (involving files)	107
7.8.1	Specifying file format	107
7.8.2	Displaying error histories	107
7.8.3	Dumping matrices to disk	108
7.8.4	Writing a sequence of approximations (frames) to disk	108
7.8.5	Writing coarsening patterns to disk	109
7.9	Multiple instances of SAMG	109
7.10	Changing parameters at runtime (from file)	109
7.11	Some auxiliary subroutines	111
7.11.1	Optimizing SAMG's memory requirement	111
7.11.2	Checking for disconnected subsets of variables	112
7.12	Smoothed aggregation (under construction)	113
7.12.1	Initializing smoothed aggregation	113
7.12.2	Parameters for smoothed aggregation	114
7.12.3	Typical scenario: linear elasticity	115
7.13	Improvement of classical interpolation (under construction)	116
7.13.1	The main control parameter	116
7.13.2	Details of optimization process	116
7.14	Improving convergence by (re-)scaling	118
8	Special Interfaces	119
8.1	Alternative interfaces to call SAMG	119
8.1.1	Simplified interfaces	119
8.1.2	An AMG1R5-like interface	119
8.2	Defect correction interface	119
8.3	Special user interfaces (in case of source licenses)	121
8.3.1	Interface to the coarsest-level solver	121
8.3.2	Interface to check the input matrix	123
8.3.3	Interface to a user's license checker	124
8.3.4	Interface for getting current cycle number and residual	125
8.3.5	Message routing	125
9	Code Numbers of Errors and Warnings	126
10	What is New?	128
11	References	130
12	Index	133

1 Introduction and Overview

1.1 Review of SAMG

1.1.1 Hierarchical solvers

The efficient numerical solution of large systems of discretized elliptic partial differential equations (PDEs) requires **hierarchical algorithms** which ensure a rapid reduction of both short and long range error components. A breakthrough, and certainly one of the most important advances during the last four decades, was due to the **multigrid principle**. Any corresponding method operates not only on a given discretization grid but rather on a hierarchy of grids, defined a priori by coarsening the given grid in a geometrically natural way ("geometric" multigrid).

Since the early nineties, there is a strong increase of interest in **algebraically oriented** hierarchical methods which construct a reasonable hierarchy *automatically*, based on *algebraic information* (explicitly or implicitly contained in the discretization matrix) rather than *grid information*. One reason for this is certainly the increasing geometrical complexity of applications which, technically, limits the immediate use of geometric multigrid. Another reason is the steadily increasing demand for efficient "plug-in" solvers which can be integrated into existing software packages without the need of a complete code re-structuring. In particular in commercial codes, this demand is driven by increasing problem sizes which clearly exhibit the limits of the classical one-level solvers still used in most packages. Millions of degrees of freedom in the underlying numerical models require hierarchical approaches for an efficient solution.

1.1.2 Algebraic multigrid (AMG)

The first hierarchical, matrix-based approach was **algebraic multigrid (AMG)** which extends the classical ideas of geometric multigrid (based on smoothing and coarse-grid correction) to certain classes of linear systems of equations. Rather than on a hierarchy of *grids*, AMG operates on a hierarchy of smaller and smaller *linear systems of equations* which is constructed fully automatically. In particular, the construction of operators (matrices) used to transfer information between different levels (restriction of residuals and interpolation of corrections) is based on matrix entries, and matrices on coarser levels are computed based on the so-called Galerkin principle ("Galerkin matrices"). This automatism is the major reason for AMG's flexibility in adapting itself to specific requirements of the problem to be solved and is the main reason for its robustness in solving large classes of problems despite using very simple smoothers.

Although the origin of AMG dates back to the early eighties [1-5], it still provides one of the most attractive algebraic approaches. However, substantial research on AMG is still ongoing, mainly aiming at extending its range of applicability. Major research is focusing on applications involving *coupled systems* of PDEs for which a robustness and efficiency has not yet been reached to the same extent as for *scalar* PDEs. Nevertheless, substantial progress has been achieved and further advances are to be expected. For a review of AMG as well as a detailed introduction we refer to [8] and [7], respectively. Regarding techniques and applications for selected coupled systems of PDEs, see, for example, [10 ff.].

1.1.3 The SAMG package

SAMG, written in FORTRAN90, is based on an AMG approach which, by our experience, has turned out to be very flexible, robust and efficient in practice. Essential components of SAMG have been implemented already in the code RAMG [7], which is a successor of the original (public domain) code AMG1R5, described in [3]. Compared to RAMG, however, SAMG is much more general. In particular, SAMG can be applied to both scalar and coupled systems of "elliptic" PDEs.

From the user's point of view, SAMG is a **"plug-in" solver**. Essentially, just the (sparse) linear system of equations to be solved, $\mathbf{A}\mathbf{u}=\mathbf{f}$, has to be passed to SAMG. In general, no information regarding the shape of the domain or the structure of the underlying grid needs to be provided. Thus, besides its robustness and efficiency, the main practical advantage of SAMG is that it can directly be applied to solve certain classes of PDEs on *unstructured* meshes, both *in 2D and 3D*. Moreover, SAMG can even be applied to problems *without any geometric background*, provided that the underlying matrices are of a "similar type" as the ones arising from elliptic PDEs.

1.2 Overview of this manual

The SAMG package is considerably more complex than a standard numerical solver. SAMG is actually not just a fixed solver but rather a complete **multi-level environment**. This implies that its use is not as straightforward as that of a classical solver.

In order to take full advantage of the various features of SAMG, it is very helpful to have some basic understanding of the multi-level principle in general and SAMG's way of operation in particular. Most basic aspects and some conventions are introduced in Section 2. A brief introduction to the solution approaches provided by SAMG and the underlying data structures are given in Section 3 and 4, respectively.

The **main user interface** to SAMG (calling sequence) is described in detail in Section 5. We call the corresponding list parameters "primary parameters". Besides these primary parameters, there are various other ones which are hidden from the user, most of which are explained in Section 6. Under most conditions, a user does not need to access these "hidden parameters". In certain situations, however, a user might wish to modify one or more of the hidden parameters to tune SAMG's performance (→ Section 6.1).

Section 7 describes various **special options and features** of SAMG such as its OpenMP parallelization, a variety of termination criteria for the iterative process, how to provide (optional) coordinate information, possibilities to treat "special" variables, ways to use SAMG as a one-level solver, some automatic solver control for standard applications, reading parameters from files at run time and more. If the source of SAMG is available, some special user interfaces are accessible which are described in Section 8.

In case SAMG terminates with an error or a warning, we refer to Section 9 for a list of error codes. In Section 10 we summarize major new developments and options of SAMG. Section 11 contains some SAMG-related references. Finally, the index in Section 12 is hopefully helpful in scanning for all kind of parameters and further information.

2 Background Information

As mentioned before, SAMG is not just a fixed solver but rather a complete multi-level environment. Consequently, its optimal use requires some basic understanding of the multi-level principle in general, and SAMG's way of operation in particular. In contrast to a classical numerical solver, SAMG takes specific characteristics of a given linear system of equations into account. For instance, SAMG needs to distinguish between problems corresponding to discretized *scalar* or *coupled* systems of PDEs. Corresponding information has to be provided to SAMG.

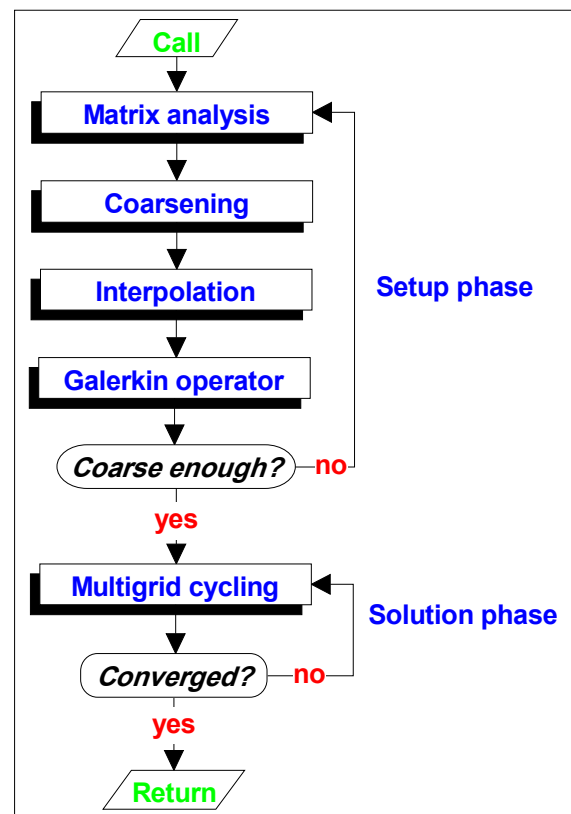
In this section, we summarize some basic facts on SAMG. We refer the interested reader to [7] where details on AMG can be found.

2.1 The structure of SAMG

A standard call to SAMG invokes a two-part process. The first part, a fully automatic **setup phase**, recursively performs four steps:

1. The connectivity of the current matrix is analyzed. The purpose is to separate the non-zero couplings into "strong" and "weak" ones.
2. Based on this, the total set of variables is split into a subset of variables which remain alive (C-variables, defining the next coarser level) and the complementary set (F-variables). Alternatively, coarser levels can be build by *clustering* (or *aggregation*) of variables.
3. Guided by the knowledge of strong couplings, the transfer operators (interpolation and restriction²) are computed.
4. The coarse-level matrix is computed based on the Galerkin principle ("Galerkin matrix").

The individual steps of SAMG's setup phase are controlled by various parameters some of which require a fairly detailed understanding of the whole coarsening process. These parameters are hidden from the normal user of SAMG ("hidden parameters", → Section 2.4.1, → Section 6). All of them are initialized to reasonable default values but can be accessed (read or modified) via specific subroutine calls (→ Section 6.1).



The second part, the **solution phase**, just uses the components resulting from the setup phase to perform normal multigrid iterations ("cycles") until a desired level of tolerance is reached. As with any other multigrid method, SAMG's solution phase consists of three algorithmical components:

- *smoothing* on all but the coarsest level (in the simplest case by plain Gauss-Seidel relaxation),
- data transfer between levels (*interpolation* and *restriction*),
- sufficiently accurate *solution on the coarsest level*.

² Generally, restriction is the transpose of interpolation.

These components (details of which are partly controlled by hidden parameters) are “combined” to **multigrid cycles** which are then employed for the final iterative process. There are essentially three types of cycle, referred to as **V-**, **F-** and **W-cycle**, which differ only in the extent to which they approximate the corresponding **two-level cycle**. That is, these cycles are increasingly more “robust” but also increasingly more expensive. *In practice, the cheap V-cycle is generally the best and most efficient choice!*

2.2 The performance of SAMG

Compared to geometric multigrid, the flexibility of SAMG and its simplicity of use, of course, have a price. The extra overhead caused by the setup phase is one reason for the fact that SAMG is usually somewhat less efficient than geometric multigrid (if applied to problems for which geometric multigrid *can* be applied efficiently). Another reason is that SAMG's hierarchical components can, generally, not be expected to be optimal. In fact, they will always be constructed on the basis of compromises between numerical work and overall efficiency. Nevertheless, as a rough guide line, the computational cost of SAMG's solution phase (ignoring the setup cost) is typically comparable to the solution cost of a *robust* geometric multigrid solver, if applied to standard elliptic test problems.

However, SAMG should not be regarded as a competitor of geometric multigrid. The strengths of SAMG are its robustness, its applicability in complex 2D and 3D geometric situations and its applicability to even solve certain problems which are out of the reach of geometric multigrid, in particular, problems with no geometric or continuous background at all (as long as the given matrix is similar to those arising in PDE applications). That is, SAMG provides an attractive multi-level variant whenever geometric multigrid is either too difficult to apply or cannot be used at all. In such cases, SAMG should be regarded as an efficient alternative to standard numerical solvers such as conjugate gradient accelerated by classical one-level preconditioners.

As mentioned before, substantial research and development on algebraic multigrid is still ongoing and many problems cannot yet be tackled efficiently. However, SAMG's range of applicability is growing steadily, and its robustness and efficiency are continuously being enhanced. In contrast to any one-level method, if applied in mature cases, SAMG has a convergence behavior which does virtually not depend on the size of the given problem. Consequently, SAMG becomes the more attractive, the larger a given problem is!

Remark: Originally, SAMG has been developed as a **stand-alone** iterative solver. Practical experience, however, has clearly shown that SAMG is also a very good **preconditioner**, much better than standard (one-level) preconditioners. Heuristically, the major reason is due to the fact that SAMG, in contrast to any one-level preconditioner, aims at the efficient reduction of *all* error components, short-range as well as long-range. Consequently, highest efficiency is typically obtained if multigrid cycling is accelerated by standard methods such as conjugate gradient (CG), BiCGstab or GMRes all of which are included in SAMG.



2.3 Features of SAMG, an overview

2.3.1 General features

Memory management. Because of the dynamic nature of SAMG, it is virtually impossible to precisely predict the final memory requirement. Consequently, if requested, SAMG performs its own memory management by exploiting dynamic memory allocation at run time ($\rightarrow$ *iextent*, Sections 5.6.2 and 2.5).

Stopping criteria for the iteration. Besides robust convergence, the availability of reasonable and reliable stopping criteria is of major concern for iterative solvers. Corresponding developments are still ongoing, but various practical stopping criteria are available by now (→ Section 7.2).

AMG as pre-conditioner. SAMG can be used in stand-alone mode or as preconditioner³ (→ *ncgrad*, Section 5.5). The following options are available:

- stand-alone iteration,
- preconditioner for standard CG,
- preconditioner for standard BiCGstab,
- preconditioner for *flexible* GMRes,
- preconditioner for *flexible* CG.

Under all normal circumstances, it is recommended to use SAMG as a preconditioner.

Special treatment of exceptional variables. There may be a few exceptional equations which are different in nature from the rest of the linear system. Such equations may often be dealt with by forcing them either into all coarser levels or to remain only on the finest level where they can be treated separately from the AMG process (→ Section 7.4).

Reusing setup data. SAMG may be called repeatedly, reusing previous setup data. In the simplest case, this is used to solve the same system of equations with just different right hand sides. However, successive calls may also re-use parts or even all of previous setup data even if the new matrix is *not* the same as in the previous call. Clearly, for this to make sense, the new matrix should be in some sense “similar” to the previous one (→ Section 5.6.1).

OpenMP parallelism. Most components of SAMG have been OpenMP parallelized (→ Section 7.1). Further optimization is still ongoing.

One-level solvers. Although SAMG is primarily a multi-level solver, it naturally includes also various classical one-level solvers (→ Section 7.6). In particular, various direct solvers (eg, pardiso) as well as CG, GMRes or BiCGstab preconditioned by

- diagonal scaling,
- any of the relaxation methods,
- ILU-based and block ILU-decomposition,
- Uzawa-type iteration.

Multiple instances. Since SAMG is not thread-safe, the possibility of creating “multiple instances” has been provided⁴ (→ Section 7.9).

Use of coordinates. Optionally, grid coordinates can be passed to SAMG, opening new possibilities and/or giving more flexibility in constructing suitable AMG hierarchies (→ Section 7.3).

Automatic solver control. A first automatic solver control has been released which is being continuously developed further. Currently, it is supposed to be used in standard time stepping methods to automatize the re-use of setup data as well as the automatic switching between multi-level and one-level solvers. However, generality, performance and reliability are subject to further improvement (→ Section 7.5).

³ Instead of saying that SAMG is used as a preconditioner of, for example, conjugate gradient, we sometimes say that SAMG is “accelerated by conjugate gradient”.

⁴ This feature requires an SAMG source license.

2.3.2 More specific features

SAMG provides various algorithmic AMG components to adapt to different situations. We here just list major multi-level components without detailed comments.

Coarsening. In extension of older code versions, the current release of SAMG includes coarsening strategies which are not necessarily based on the entries of A . Instead, coarsening may be based on what we call a “primary matrix” (→ Sections 3.3 and 5.4.3). Typical primary matrices are derived from A in one way or the other. Alternatively, a primary matrix may also be defined based on coordinates or distances (if available, → Section 7.3). Regarding the effectiveness (speed) of coarsening, there are essentially three approaches, the standard approach and two more aggressive ones (→ *nred*, Section 6.2.3):

- standard coarsening,
- aggressive coarsening,
- various types of cluster (or aggregative) coarsening.

The purpose of the more aggressive coarsening strategies is to substantially reduce the memory requirements of SAMG, in case memory is a problem. However, generally, this will be at the expense of a slower convergence. Nevertheless, aggressive coarsening often also reduces total computational time. However, whether or not this is true, depends on the application.

Interpolation. Various *types* of interpolation are available (→ *nwt*, Section 6.2.5):

- direct interpolation,
- standard interpolation,
- extended standard interpolation⁵,
- multi-pass interpolation,
- cluster interpolation (piecewise constant),
- Jacobi-interpolation (F-relaxation of interpolation).

For coupled systems of PDEs, different interpolation structures are available (→ *napproach*, Section 5.4.1). The concrete interpolation *weights* can be computed in various ways. For instance, based on the entries in A , based on a primary matrix, geometry-based (if coordinates are available, → Section 7.3), or blockwise (in case of coupled systems of PDEs). For details, → Section 5.4.4.

Truncation. Two truncation options are available (→ *ntr*, Section 6.2.9):

- truncation of interpolation,
- truncation of Galerkin matrices.

While a suitable truncation of interpolation is important and standard, that of the Galerkin matrices should be done with care (→ [7]).

Smoothing. Currently available smoothers (→ *nxtyp*, Section 5.4.2):

- variable-wise Gauss-Seidel relaxation, either lexicographic or colored (eg, C/F),
- block Gauss-Seidel relaxation, either lexicographic or colored (eg, C/F),
- variable-wise Jacobi relaxation,
- block Jacobi relaxation,
- ILU-type smoothers,
- block ILU-smoothers,

⁵ This is being re-designed and should currently not be used.

- Uzawa smoother.

The standard smoother is Gauss-Seidel relaxation.

Cycling. Besides the standard V-cycle, the more expensive F- and W-cycles are available (→ *igam*, Section 5.5). There are variants of these cycles which include a recursive optimization of corrections based on the *minimization of energy norms* and *residual norms* in symmetric and non-symmetric applications, respectively. These variants are denoted by V^{*}-, F^{*}- and W^{*}-cycle, respectively (→ sign of *ncyc*, Section 5.5).

- V- and V^{*}-cycle,
- F- and F^{*}-cycle,
- W- and W^{*}-cycle.

The standard cycle is the V-cycle.

Coarsest-level solvers. For complex applications, the solver used on the coarsest level may become crucial. A variety of solvers is available (→ *nrc*, Section 6.3.4):

- iterative application of the smoother,
- diagonally preconditioned CG or BiCGstab,
- ILU preconditioned CG or BiCGstab,
- dense Gauss elimination (standard),
- sparse Gauss elimination (standard),
- Intel's Pardiso,
- optimized dense Gauss elimination (in SAMGp this solver is MPI parallel)
- least squares (LSQ) solver.

If source is available, an interface to the coarsest-level problem is accessible allowing the incorporation of a user-supplied solver (→ Section 8.3.1).

Remark: The standard and safest choice is (sparse) Gauss elimination, at least if the coarsest level is not too fine. If an iterative solver is selected, in principle, one may use much finer coarsest levels. However, in complex applications, the convergence of iterative solvers may become quite unpredictable and even influence the resulting overall efficiency of SAMG to a large extent.



2.4 General remarks on SAMG parameters

2.4.1 Type of parameters: primary and hidden

Two classes of parameters are used to control the behavior of SAMG:

Primary parameters. These are all parameters which are passed to SAMG via its argument list (→ Section 5). The primary parameters specify the linear system of equations to be solved, and control the most basic features of the solver.

Hidden parameters. These parameters refer to various aspects and can only be accessed (read or modified) via specific subroutine calls. They are described in detail later in this manual (→ Section 6 ff.). All hidden parameters are initialized to reasonable default values.

2.4.2 Naming convention

Regarding the naming of *integer* and *real* parameters, we generally stick to the Fortran convention that names starting with *i*, *j*, *k*, *l*, *m* or *n* refer to integers, all others refer to reals. Furthermore, *real* always means *double precision*. Exceptions to these rules will explicitly be mentioned.

2.4.3 Multi-purpose use of single parameters

Many single SAMG parameters actually define two or more different (sub-) parameters. At first glance, this comprehensive way of using parameters may be confusing. However, after all, it is quite convenient and keeps the total number of parameters low.

Integers

SAMG's integer parameters often correspond to two or more sub-parameters in the sense that different digits serve different purposes, for instance, the parameters *nsolve*, *iswtch* and *ncyc* (→ Section 5). Throughout this manual, we use the following notation to describe sub-parameters of an integer, *intparm*:

<i>intparm</i>[<i>k</i>]	- sub-parameter consisting of the <i>k</i> -th digit (counted from the left)
<i>intparm</i>[<i>n:m</i>]	- sub-parameter consisting of the <i>n</i> -th up to the <i>m</i> -th digit ($0 < n \leq m < 10$)
<i>intparm</i>[<i>k:</i>]	- sub-parameter consisting of the sequence of digits starting at the <i>k</i> -th one

Convention: If individual digits of an integer parameter are used for different purposes and if the total number of used digits is fixed, trailing zeroes can be omitted. Although such parameters are only introduced in Section 5, we want to give some examples for demonstration:

<i>nsolve</i> =20000	and	<i>nsolve</i> =2	are equivalent
<i>iswtch</i> =5110000	and	<i>iswtch</i> =511	are equivalent
<i>ncyc</i> =110200	and	<i>ncyc</i> =1102	are not equivalent ⁶

Reals

Sometimes SAMG uses a *single* real parameter to actually define *two* real sub-parameters, for instance, the parameters *ecg* and *ewt* (→ Section 6.2.1). More specifically, assuming a parameter, *val*, to be of the form ***val* = *ki.j*** with *i* and *j* being integers and *k* being a single digit (or zero), we define

$$val1 = i \cdot 10^{-k} \quad \text{and} \quad val2 = 0.j$$

respectively. Some examples:

<i>val</i> = 21.25d0	→	<i>val1</i> = 0.01d0, <i>val2</i> = 0.25d
<i>val</i> = 0.2d0	→	<i>val1</i> = 0.0d0, <i>val2</i> = 0.2d0
<i>val</i> = 125.0d0	→	<i>val1</i> = 2.5d0, <i>val2</i> = 0.0d0
<i>val</i> = 225.0d0	→	<i>val1</i> = 0.25d0, <i>val2</i> = 0.0d0
<i>val</i> = 101.01d0	→	<i>val1</i> = 0.1d0, <i>val2</i> = 0.01d0
<i>val</i> = 11.01d0	→	<i>val1</i> = 0.1d0, <i>val2</i> = 0.01d0
<i>val</i> = 3.1d0	→	<i>val1</i> = 3.0d0, <i>val2</i> = 0.1d0

Remark: For ease of referencing, sub-parameters will usually be assigned names later in this manual. Note that this does not mean that named sub-parameters can be assigned values directly. In fact, sub-parameters can only be defined via their “root” parameter.



⁶ This is because the number of used digits is **not fixed**: *ncyc*[4:] denotes the maximum number of iterations

2.5 Automatic memory management

If activated, SAMG does its own workspace management (→ *iextent*, Section 5.6.2). Consequently, during execution, SAMG dynamically allocates a substantial amount of memory, for instance, to setup and store the complete AMG hierarchy.

From the user's point of view, this automatic memory management has two implications:

First, there is no use in having SAMG's input arrays being dimensioned larger than necessary. Second, unless SAMG is called as a "one-time solver" (→ *iswit=5*, Section 5.6.1), all relevant parts of its workspace and corresponding data (including the complete AMG hierarchy) are still available after return from SAMG and, if requested, may be reused by the next call to SAMG. If this data is no longer needed, the user may clean the SAMG workspace "manually" by issuing

```
call samg_refresh(ierr).
```

This frees up all memory which has been allocated by SAMG at run time. Should SAMG's *automatic solver control* be activated (→ Section 7.5), the above call will also re-set certain control data as well as the complete history of the past control process and start a new control process from scratch.

Convention: Memory which has been allocated *by the user* (eg via a call to `samg_coo_vec_alloc`, → Section 7.3), is **not** cleared by calling the previous cleanup routine. Instead, special SAMG routines are provided to clear such workspace (`samg_coo_vec_dealloc` in the previous example) and **it is the user's responsibility to explicitly call these routines.**



In addition to cleaning up SAMG's workspace, issuing

```
call samg_leave(ierr)
```

also resets all hidden parameters (→ `samg_reset_hidden()`, Section 6.1). Should the used SAMG library contain a license checker, this call additionally resets the status of the license, that is, the current license will be freed and certain internal counters will be reset. If SAMG is used in OpenMP mode, `samg_leave` has to be called prior to any change in the number of threads between SAMG calls.

3 Solution Approaches

SAMG provides various solution approaches, in particular, if the linear system of equations, $Au=f$, has been derived from a *coupled system* of PDEs. We need to clearly distinguish such applications from those derived from *scalar* PDEs. For a brief overview for SAMG's solution approaches, we need to introduce some basic notation. In particular, we distinguish between what we call *variables*, *unknowns* and *points*:

- A **variable** is any of the solution components u_i of the linear system $Au=f$ to be solved. **Variables are numbered 1,2,...,nnu.**
- An **unknown** is any of the (scalar) physical functions being approximated (eg, pressure, temperature, a velocity or a displacement component). Assuming $nsys$ such functions to be involved in a given linear system of equations, the **unknowns are numbered 1,2,...,nsys.**
- A **point** is a location in space where a single variable or a group of variables (corresponding to different unknowns) is defined. **Points are numbered 1,2,...,npnts.**

Remark: Since we usually have the solution of PDEs in mind, we here may think of **points** as being real physical points in space. However, we want to point out that, from SAMG's point of view, it is not important whether "points" really correspond to physical points. Generally, instead of physical points, one may think of the **nodes of a graph** representing the connectivity structure of A .



In general, each variable is associated with both a point and an unknown. In order to distinguish problems derived from scalar and coupled systems of PDEs, we introduce the following notation. We call a linear system of equations, $Au=f$, a

- **scalar system** if there is *only one* unknown involved, that is, if $nsys=1$ (all variables correspond to the same physical function, for instance, the pressure);
- **coupled system** if there is *more than one* unknown involved, that is, if $nsys>1$.

3.1 Variable-based approach

In the **variable-based approach**, the simplest of SAMG's solution approaches, the coarsening process is on the level of variables without distinguishing between unknowns or points. Consequently, this approach is mainly suitable to solve scalar systems. (For this reason, the variable-based approach is also called **scalar approach**.) If applied to a *coupled system*, the variable-based approach can be efficient only if the coupling between different unknowns is very weak. **Since this is not typically the case, it is generally not recommended to use the variable-based approach to solve coupled systems!**

Using the variable-based approach, SAMG's coarsening process is directly based on the connectivity pattern reflected by the matrix A and, by default, interpolation is constructed based on the matrix entries. Alternatively, SAMG also allows interpolation to be defined based on geometric distances, provided coordinates are available (→ Section 7.3).

Typical scalar systems derived from (elliptic) boundary value problems are characterized by zero row sums (except near boundaries). Moreover, important second order discretizations often lead to matrices which are "close to" being **M-matrices**. For such applications, SAMG is most mature and has been extensively tested in practice. In general, SAMG can be expected to perform efficiently for scalar applications which are symmetric and positive definite (s.p.d.). A two-level theory is available (→ [2,5,7]), proving that, for such applications, SAMG always converges.

However, SAMG does not *require* symmetry of a problem. Although there is not yet a satisfactory non-symmetric theory, there is some heuristic evidence that SAMG performs well also for various non-symmetric applications. Actually, the non-symmetry by itself will typically not cause any particu-

lar difficulties as long as diagonal dominance is not strongly violated and the matrix is still positive definite⁷. What “strongly” precisely means is difficult to quantify and depends to a large extent on the type of problem. For instance, SAMG can efficiently deal with convection-dominated problems if discretized by *first order* upwind. However, *higher order* discretizations require special techniques.

3.2 Unknown-based approach

In the **unknown-based approach**, the simplest of SAMG’s solution approaches for coupled systems ($nsys > 1$), the coarsening process is still on the level of variables, similar to the variable-based approach. However, variables corresponding to different unknowns are now treated separately. For ease of description, let us assume the variables to be ordered by *unknowns*, that is, $Au=f$ has the form

$$\begin{bmatrix} A_{[1,1]} & \cdots & A_{[1,NSYS]} \\ \vdots & & \vdots \\ A_{[NSYS,1]} & \cdots & A_{[NSYS,NSYS]} \end{bmatrix} \begin{bmatrix} u_{[1]} \\ \vdots \\ u_{[NSYS]} \end{bmatrix} = \begin{bmatrix} f_{[1]} \\ \vdots \\ f_{[NSYS]} \end{bmatrix},$$

where $u_{[n]}$ denotes the vector of variables corresponding to the n -th unknown and the matrices $A_{[n,m]}$ reflect the couplings between the n -th and the m -th unknown.

Using this notation, coarsening of variables corresponding to the n -th unknown is based on the connectivity structure reflected by the submatrix $A_{[n,n]}$ and, by default, interpolation is constructed based on this matrix’ entries. Alternatively, SAMG also allows interpolation to be defined based on geometric distances, provided coordinates are available (→ Section 7.3). This is analogous to the variable-based approach, except that each unknown is interpolated separately.

While the interpolation to any variable i involves only coarse-level variables corresponding to the same unknown as i , the Galerkin matrices are - by default - computed w.r.t. the full set of unknowns. Alternatively, if requested, couplings between different unknowns are neglected, leading to *block diagonal approximations* of the complete Galerkin matrices (→ sign of *nsolve*, Section 5.4.1).

The unknown-based approach is the simplest approach for solving coupled systems. The only additional information required by SAMG is information about the correspondence between variables and unknowns. The essential condition for this approach to work is that smoothing the individual equations is sufficient to cause the resulting error to be smooth *separately for each unknown*. In practice, this approach works well and is quite efficient for many applications. One advantage of this approach is that it can easily cope with anisotropies which are different between the different unknowns. Another advantage is that, in principle, the unknowns can be distributed arbitrarily across points allowing, for instance, staggered grids. On the other hand, this approach may be inefficient whenever the coupling between different unknowns is “too strong”. Furthermore, it is not suited for *saddle point problems*.

3.3 Point-based approaches

We talk about **point-based approaches** if coarsening is not on the level of variables (as before) but on the level of points. In order to distinguish this type of coarsening from the previous ones, we call it **point-coarsening**. Although point-coarsening makes sense also for *scalar* systems (→ remark at the end of this section), it is mainly designed for *coupled* systems where one purpose is to ensure that all unknowns are defined on the same hierarchy. (Note that this excludes staggered grid applications.)

⁷ That is, $(Au,u) > 0$ for $u \neq 0$.

Clearly, in addition to information about the correspondence between variables and *unknowns*, SAMG now also needs information about the correspondence between variables and *points*, that is, SAMG needs to know at which points individual variables are located. For ease of description, let us assume the variables to be ordered *pointwise*, that is, $Au=f$ has the form

$$\begin{bmatrix} A_{(1,1)} & \cdots & A_{(1,NPNTS)} \\ \vdots & & \vdots \\ A_{(NPNTS,1)} & \cdots & A_{(NPNTS,NPNTS)} \end{bmatrix} \begin{bmatrix} u_{(1)} \\ \vdots \\ u_{(NPNTS)} \end{bmatrix} = \begin{bmatrix} f_{(1)} \\ \vdots \\ f_{(NPNTS)} \end{bmatrix},$$

where $u_{(i)}$ denotes the “block” of variables located at point i and the matrix $A_{(i,j)}$ represents the “block coupling” between $u_{(i)}$ and $u_{(j)}$.

3.3.1 The coarsening process

The main idea behind point-coarsening is to base the concrete coarsening process on some auxiliary “scalar-type” matrix P , the **primary matrix**, and assign the resulting coarse level to all unknowns. For this process to make sense, the employed primary matrix should reflect the physical connectivity (the general structure as well as the strength of connections) of neighboring variables reasonably well, *simultaneously for all unknowns*. In SAMG, a primary matrix P may be *user-supplied* or it may be defined *internally*. In order to outline a few typical possibilities, we use the notion of a **primary unknown** which may be either one of the given *physical unknowns* or a user-added *dummy unknown*:

- In the simplest case, the user selects one of the unknowns $1 \leq k \leq n_{sys}$ as primary unknown and tells SAMG to use $P=A_{[k,k]}$ as primary matrix. Whether or not this makes sense, depends on the application, in particular, whether the connectivity structure of the k -th unknown is also representative for the other unknowns.
- As mentioned above, the primary unknown may also be a dummy unknown. More precisely, if there is a physically natural choice for P which does not correspond to choosing any of the “real” unknowns, the user can add P to the given system (ie, A is augmented by P). This can be interpreted as adding a dummy unknown to the original system (ie, n_{sys} is increased by 1) which then serves as primary unknown. Note that, clearly, a dummy unknown has *no* couplings to any of the “real” unknowns.
- There are various ways to define a primary matrix *internally* to SAMG. One typical possibility is to compute the entries of P based on *suitable norms* of the block-matrices $A_{(i,j)}$. Another possibility is to use *distances* between points (provided coordinates are available, → Section 7.3). Note that an automatic definition of P requires also a *non-zero sparsity pattern* to be prescribed. In SAMG, this pattern will be copied from a prescribed primary unknown or, if no primary unknown is specified, it will be maximum.

Remark: A point-based approach does not require all unknowns to be defined at all points. However, a reasonable primary matrix necessarily has to “represent” all points. In particular, an unknown can be selected as “primary” only if it is defined everywhere with the possible exception of points where no variable is defined at all. If a primary unknown corresponds to a *dummy* unknown, it has to be assigned *the highest number*. That is, the original physical unknowns are numbered $1, \dots, n_{sys}-1$, and n_{sys} corresponds to the dummy unknown.



Remark: As noted further above, point-based approaches make sense also for **scalar systems**. In fact, the concept of using a primary matrix P for coarsening rather than A itself opens new algorithmic possibilities. The variable-based approach described in Section 3.1 is a special point-based approach, namely the one with $P=A$.



3.3.2 The interpolation

As mentioned before, a major purpose of the primary matrix is to define a coarse level of *points* which can be used for all unknowns simultaneously. Once such a level is constructed, there are now many ways to define a concrete **interpolation**. In particular, a decision has to be made w.r.t. the following aspects:

- the general *structure* of interpolation (eg, separate for each unknown, the same for each unknown or blockwise);
- the interpolation *pattern* (eg, copied from on A or P);
- the concrete way of computing the interpolation *weights* (eg, based on entries of A , based on entries of P or based on coordinates).

SAMG provides a fairly large spectrum of possibilities to define interpolation.

4 Data Structure

In this section, we describe the data structure used to pass the given linear system of equations, $Au=f$, to SAMG. We also describe which additional information has to be provided in case of *coupled* systems.

4.1 The linear system of equations

The entries of the matrix A , denoted by $A(i,j)$, are assumed to be stored in **compressed row format** (compressed row storage, **CSR**) in a vector a , row by row, *each row starting with its diagonal element*. While all diagonal entries have to be stored independent of whether or not they are zero, the off-diagonal entries need to be stored only if they are non-zero⁸. Since the order in which the off-diagonal entries are stored in a is arbitrary, two integer pointer vectors ia and ja are required to identify each element in a . Details are described in the following.

If nnu denotes the number of rows (variables), the non-zero entries of the i -th row ($1 \leq i \leq nnu$) are stored in $a(j)$ where

$$ia(i) \leq j \leq ia(i+1)-1.$$

In particular, according to the above-mentioned convention about the location of the diagonal element, $a(ia(i))$ contains the diagonal entry of row i . Note that $ia(1)=1$ and $ia(nnu+1)=nna+1$ where nna denotes the total number of matrix entries stored.

The pointer vector ja has to be defined so that $ja(j)$ ($1 \leq j \leq nna$) equals the original matrix' column index of $a(j)$, ie, $a(j)$ corresponds to the variable $u(ja(j))$. In particular, since $a(ia(i))$ contains the diagonal entry of row i , we have $ja(ia(i))=i$.

Summarizing, for any $1 \leq i \leq nnu$ and $ia(i) \leq j \leq ia(i+1)-1$, we have $a(j) = A(i,ja(j))$ and the i -th equation of $Au=f$ reads:

$$\sum_{j1 \leq j \leq j2} a(j) u(ja(j)) = f(i)$$

where $u(i)$ and $f(i)$ denote the i -th component of u and f , respectively, and $j1=ia(i)$, $j2=ia(i+1)-1$.

In the following, we summarize some important requirements. **Note that ignoring one or more of these requirements is the reason for most user errors in applying the above data structure!**

- Note that $ia(i)$ has to be defined for all $1 \leq i \leq nnu+1$!
- The order of rows has to be such that the i -th row (equation) corresponds to the i -th variable.
- The order of entries within each row is arbitrary except that the diagonal entry has to be first.
- For symmetric matrices A , *all* (non-zero) entries need to be stored (not just a triangular part).
- Unless explicitly stated otherwise ($\rightarrow$ Section 4.3), we assume diagonal entries to be positive.



⁸ Note that not all off-diagonal matrix entries stored in a necessarily have to be non-zero. In fact, it makes sense to leave some zero entries in a , for instance, entries which are not zero for reasons of discretization (according to the sparsity pattern) but rather happen to be zero only accidentally.

4.2 Additional data in case of coupled systems

If $Au=f$ corresponds to a **coupled system**, depending on the solution approach selected, SAMG needs up to two additional vectors in order to be able to properly distinguish unknowns and points. Moreover, the use of point-based approaches requires a specific ordering of the variables (rows). In the following, we assume that we are dealing with a coupled system involving $nsys>1$ unknowns numbered $1,2,...,nsys$.

Variable-based approach

Besides what has been described in the previous section, no additional information is needed and there are no special requirements on the ordering of variables (rows).

Unknown-based approach

SAMG requires some additional information on the system, namely, the correspondence between variables and unknowns. This information has to be provided via an integer **variable-to-unknown** pointer vector iu : For each variable $1 \leq i \leq nnu$, $iu(i)$ has to contain the number of the unknown this variable represents:

$$iu: [1,...,nnu] \rightarrow [1,...,nsys].$$

Each unknown has to be represented by (at least) one variable. That is, for each unknown $1 \leq k \leq nsys$, we require that $iu(i)=k$ for (at least) one i . Besides what has been described in the previous section, there are no additional requirements on the ordering of variables (rows).

Examples: In the common case that the variables are ordered by *unknowns* ($\rightarrow$ Section 3.2) and assuming $nsys=3$, the vector iu will look like

$$iu [1,2,3,4,5,6,...] \rightarrow [1,1,...,1,2,2,...,2,3,3,...,3].$$

However, any other ordering is fine, for instance something like this:

$$iu [1,2,3,4,5,6,...] \rightarrow [1,3,3,3,1,1,1,2,2,2,1,...].$$

Point-based approaches

In addition to the variable-to-unknown pointer iu , a **variable-to-point** pointer vector ip is required,

$$ip: [1,...,nnu] \rightarrow [1,...,npnts].$$

That is, for each variable $1 \leq i \leq nnu$, $ip(i)$ has to contain the number of the point where it is located. To be more specific, note the following:

Clarification: The *only* purpose of ip is to specify which variables “belong together”. Hence, in principle, numbering of points is completely arbitrary as long as variables sitting at the same point occupy successive ip -ranges and are assigned the same ip -number. All ip -vectors with this property are “**logically equivalent**” from the SAMG point of view. Independent of the user’s input, SAMG will change the numbering to be in ascending order so that the ip -vector used internally will look like $ip=(1,1,...,2,2,...,npnts,npnts,...)$. **Unless the user has specified this order to begin with, ip will be overwritten upon return from SAMG!** In any case, however, note that the returned ip -vector is logically equivalent to the user-provided one!



Naturally, none of the unknowns is allowed to be represented at a point more than once. On the other hand, not all unknowns need to be represented at a point. In general, the number of variables may vary from point to point and there may even be points with no variables attached at all.

Remark: In previous releases of SAMG, there was the additional requirement that the ordering of variables located at the same point had to be **consecutive** relative to the numbering of the unknowns. That is, the variable corresponding to unknown one had to be first, followed by the variable corresponding to unknown two, etc. Although such an ordering at each point seems natural and is recommended, it is no longer required.



Examples: Assuming $nsys=3$ and the same number of variables sitting at each point, typically, the two mappings are as follows:

$$iu [1,2,3,4,5,6....] \rightarrow [1,2,3,1,2,3,1,2,3,....] \text{ and } ip [1,2,3,4,5,6....] \rightarrow [1,1,1,2,2,2,3,3,3,....].$$

Note that, according to the above remark, the following mappings would also be legal:

$$iu [1,2,3,4,5,6....] \rightarrow [1,2,3,3,2,1,1,3,2,....] \text{ and } ip [1,2,3,4,5,6....] \rightarrow [1,1,1,2,2,2,3,3,3,....].$$

In the most general case, the number of variables is allowed to change from point to point, and there may even be points with no variables attached at all. An example for such a situation:

$$iu [1,2,3,4,5,6....] \rightarrow [1,3,3,2,1,1,2,1,1,2,3,2,1,3,....]$$

$$ip [1,2,3,4,5,6....] \rightarrow [1,1,2,2,2,5,5,6,8,8,8,10,10,10,....].$$

For clarity, we have used two colors to distinguish different points and the corresponding unknowns.

4.3 Formal data restrictions

Complex-valued linear systems

SAMG operates on **real-valued** problems only. To use SAMG for **complex-valued** problems, one possibility is to re-write the complex-valued problem, $Au=f$, as a coupled system for the real and imaginary parts. By denoting $A = A_1 + iA_2$, $f = f_1 + if_2$ and $u = u_1 + iu_2$, the following coupled systems are obtained:

$$\begin{pmatrix} A_1 & -A_2 \\ A_2 & A_1 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = \begin{pmatrix} f_1 \\ f_2 \end{pmatrix} \quad \text{or} \quad \begin{pmatrix} A_2 & -A_1 \\ A_1 & A_2 \end{pmatrix} \begin{pmatrix} u_2 \\ u_1 \end{pmatrix} = \begin{pmatrix} -f_1 \\ f_2 \end{pmatrix}.$$

Sign of diagonal entries

Generally, we assume all diagonal entries to be **positive**⁹. This is a pre-requisite for classical AMG components such as variable-wise relaxation and standard interpolation schemes. For the more advanced modules, positive diagonals are formally not necessarily required any more. However, whether or not such problems can be solved efficiently by AMG depends on the application and it is difficult to give some general rules here.

⁹ Of course, this assumes a proper scaling of the given matrices. In particular, definite problems should be scaled so that $(Au, u) > 0$ for $u \neq 0$.

5 Calling Sequence: The Primary Parameters

This section contains a detailed description of SAMG's calling sequence,

```
subroutine samg(nnu,nna,nsys,                                &
               ia,ja,a,f,u,iu,ndiu,ip,ndip,matrix,iscale,    &
               res_in,res_out,ncyc_done,ierr,                &
               nsolve,ifirst,eps,ncyc,iswtch,                &
               a_cmplx,g_cmplx,p_cmplx,w_avrge,              &
               chktol,idump,iout).
```

As already mentioned in Section 2.4.1, we refer to the parameters in SAMG's argument list as **primary parameters**. Most of the primary parameters are *input* parameters¹⁰, described in detail in Sections 5.1-5.8. *Output* parameters are described in Section 5.9. As will be seen in the tables below, many of our primary (integer) parameters actually correspond to *several* different parameters in the sense that different digits serve different purposes. This comprehensive, multi-purpose use of single parameters has already been described in Section 2.4.3.

In contrast to the primary parameters, many other parameters are hidden from a normal user and need to be accessed only if a tuning of SAMG is required which cannot be achieved via its primary parameters. These **hidden parameters** and how they can be accessed will be described in later sections (→ Section 6 ff.).

5.1 Passing the linear system of equations

The following table summarizes how to pass the given system of equations, $Au=f$, to SAMG.

Parameter	Explanation		
Problem size and type			
nnu	Integer. Number of variables, ≥ 1 .		
nna	Integer. Number of matrix entries stored in vector a , $\geq nnu$.		
matrix	Integer. Specifies the type of the matrix A . Regarding <i>isym</i> cf. Remark below!		
	[1:1], isym	1	A is symmetric, full matrix stored (normal case)
		2	A is not symmetric.
		3	A is symmetric, lower triangular part stored (s. Remark below!)
	[2:2], irow0	1	A is a zero row sum matrix ¹¹ . Remark: For such matrices, the solution will be <i>normalized</i> ($\rightarrow$ Section 5.9).
		2	A is not a zero row sum matrix ¹² .
	[3:3], iforce0	0	No action
>0		Various problem modifications for test purposes (not meant to be used by an average user).	
The linear system of equations to be solved: $Au=f$			
ia, ja, a	Integer vectors $ia(1:nnu+1)$, $ja(1:nna)$; double vector $a(1:nna)$. Compressed row storage representation of matrix A ($\rightarrow$ Section 4). Remark: Upon return, ia , ja and a may have changed ($\rightarrow$ warning below).		
f	Double vector $f(1:nnu)$, right hand side.		
u	Double vector $u(1:nnu)$, first guess of solution ($\rightarrow$ <i>ifirst</i> , Section 5.2). Remark: Upon return, u will be overwritten by the solution of $Au=f$.		

¹⁰ Note that **no constants** are allowed as input parameters. Ignoring this, may cause the program to crash!

¹¹ That is, in each row the entries sum up to zero.

¹² That is, for at least one row, the entries do not sum up to zero.

Remark: If *isym*=3, SAMG expects that only the **lower triangular part** of a matrix is passed, that is, $ia(i) \leq j \leq ia(i+1)-1$ with $ja(j) \leq i$ (with diagonals still being located at the beginning of their rows!). However, this is only exploited in case of two very special one-level methods ($\rightarrow igam=5$ or $=6$ in Sections 7.7). In all other cases, providing the lower triangular part only and setting *isym*=3 has the effect that the provided matrix will be expanded to full form and *isym*=3 will be replaced by *isym*=1.



Warning: The input vectors *ia*, *ja* and *a* will be **logically** unchanged upon return. However, note the following: The order of entries *within a row* may have changed (except for the diagonal entry which remains first). Accordingly, the pointer vector *ja* may be different from its input status as well. **Consequently, if the calling program relies on a specific order of the matrix entries in each row, the vector *a* should be a copy of the original matrix!** Otherwise, one may safely ignore this warning.



5.2 First approximation and stopping criterion

The following table describes how to set the first guess (if required) and how to stop the iterative process. Further stopping criteria are accessible through certain hidden parameters ($\rightarrow$ Section 7.2).

Parameter	Explanation		
First approximation and termination parameter			
ifirst	Integer. Selects <i>first approximation</i> for <i>u</i> . If ifirst=0 , the vector <i>u</i> passed to SAMG is taken as first approximation as is. Otherwise:		
	[1:1], itypu	1	First approximation $u \equiv 0$.
		2	First approximation $u \equiv 1$.
		3	First approximation is a <i>random function</i> ($\rightarrow$ next). Remark: <i>u</i> is scaled so that the <i>L2</i> -norm of <i>Au</i> equals 1.
	[2:]	≥ 0	If <i>itypu</i> =3: Any sequence of digits, each sequence resulting in a different random function.
eps	Double. Standard <i>stopping criterion</i> for the AMG iteration ¹³ : Iteration stops if the residual of the current approximation, <i>res</i> , is smaller than a certain threshold, (see also the parameter <i>ncycle</i> in Section 5.5). In detail:		
	eps $\geq$ 0.0d0	Iteration stops if $res \leq eps * res0$ (<i>res0</i> = initial residual).	
	eps $<$ 0.0d0	Iteration stops if $res \leq eps $.	

Remark: By default, residuals are measured in the *L2*-norm. This may be changed to either the *L1*- or the *maximum*-norm by setting the parameter *norm_typ* accordingly ($\rightarrow$ Section 5.6.4).



¹³ If this stopping criterion happens to be already satisfied for the very first approximation, SAMG returns immediately without doing any computation.

5.3 Distinguishing scalar and coupled systems

As outlined in Section 4.2, additional input is required if $Au=f$ corresponds to a *coupled* system. This additional input is described in the following table. Note that the relevance of some of this additional input also depends on which solution approach is finally used (selected by the parameter *nsolve*, → Section 5.4). For scalar systems ($nsys=1$), this input is either irrelevant or its definition is obvious.

Parameter	Explanation
Problem type (scalar or coupled system)	
nsys	Integer. Number of unknowns, $1 \leq nsys \leq 99$. $nsys=1$ and $nsys>1$ distinguish scalar and coupled systems, respectively. Remark: If $Au=f$ corresponds to a <i>coupled</i> system but still $nsys=1$ on input, $Au=f$ will be solved as if it was a <i>scalar</i> system (generally not recommended!).
iscale	Integer vector $iscale(1:nsys)$, indicating which unknowns require scaling. Here, an unknown is said to require scaling if it is determined only up to a constant. Such unknowns will be scaled upon return from SAMG (→ Section 5.9). Only relevant if $nsys>1$ ¹⁴ . Otherwise, <i>iscale</i> is just a dummy vector of length 1.
	$iscale(k)=0$ k -th unknown does not require scaling.
	$iscale(k)=1$ k -th unknown does require scaling.
Variable-to-unknown and variable-to-point pointers	
iu, ndiu	Integer vector $iu(1:ndiu)$, variable-to-unknown pointer (→ Section 4.2). Only relevant if $nsys>1$ in which case $ndiu=nnu$. Otherwise, vector <i>iu</i> is a dummy vector of length $ndiu=1$.
ip, ndip	Integer vector $ip(1:ndip)$, variable-to-point pointer (→ Section 4.2). Only relevant if $nsys>1$ and if any of the point-based approaches is to be used, in which case $ndip=nnu$. Otherwise, vector <i>ip</i> is a dummy vector of length $ndip=1$.

¹⁴ Whether or not the solution of a *scalar* system requires scaling, is controlled by the parameter *irow0*, see above.

5.4 Selecting the solution approach

SAMG's solution approach is specified by the parameter ***nsolve***. In general, this parameter controls many different aspects, a full understanding of which requires the reading of Section 3. However, most of these aspects are only relevant if any of the *point-based* solution approaches is selected. Otherwise, the standard setting is ***nsolve=2*** which selects the

- **variable-based approach** for scalar systems ($nsys=1$),
- **unknown-based approach** for coupled systems ($nsys>1$).

5.4.1 The solution strategy

The general solution strategy is selected by ***napproach***, the first digit of ***nsolve***. The meaning of the remaining digits is described in the subsequent sections.

Parameter	Explanation		
Selection of solution strategy			
nsolve	Integer. Specifies SAMG's solution strategy. Standard choice: <i>nsolve=2</i>. Note that the sign is relevant for the computation of the coarse-level matrices:		
	>0	Compute full Galerkin matrices on coarser levels (standard).	
	<0	Compute block-diagonal approximations to the Galerkin matrices. Only relevant if <i>nsys</i>>1 and if interpolation is not blockwise (see below). Otherwise, the sign will be ignored.	
	[1:1], napproach	Selects approaches for scalar and coupled systems (→ Section 3).	
		1	Variable-based approach. This setting selects the variable-based approach, regardless of the value of <i>nsys</i> . Remember that, generally, the variable-based approach is only meaningful for scalar systems (<i>nsys</i>=1).
		2	Unknown-based approach. This approach requires a coupled system to be given (<i>nsys</i> >1) and the variable-to-unknown vector <i>iu</i> to be provided. For scalar systems (<i>nsys</i> =1), <i>napproach</i> will automatically be reset to 1.
		>2	Point-based approaches. Unless <i>nsys</i> =1, both the variable-to-point vector <i>ip</i> and the variable-to-unknown vector <i>iu</i> have to be provided. The concrete value of <i>napproach</i> selects the general <i>structure</i> of the interpolation to be used:
		3	interpolation is separate for each unknown
		4	interpolation is the same for each unknown
		5	interpolation is point- (block-) wise
Remark: <i>napproach</i> selects only the <i>structure</i> of interpolation. Details on its <i>pattern</i> and <i>weights</i> are selected by the (sub-) parameters <i>nint_pat</i> and <i>nint_weights</i> , respectively (see further below). Special case: For scalar systems (<i>nsys</i> =1), all of the above three settings are equivalent.			

5.4.2 The smoothing strategy

The smoothing *strategy* is selected by ***nxtyp***, the second digit of *nsolve*. The meaning of the remaining digits is described in the subsequent sections. Details of the following smoothing processes are specified by certain hidden parameters all of which are initialized to reasonable default values (→ Section 6.3.1).

Parameter		Explanation	
Selection of smoothing type			
nsolve	[2:2], nxtyp	Selects type of smoothing process. Standard choice: plain relaxation.	
		0	Plain relaxation , variable-wise. If <i>napproach</i> >1, relaxation is <i>unknown by unknown</i> . Depending on the setting of the hidden parameters <i>nrd</i> and <i>nru</i> , relaxation will be either by Gauss-Seidel or by ω -Jacobi, the default being Gauss-Seidel in C/F order (→ Section 6.3.1).
		1	ILU(0) or MILU(0) , depending on the hidden parameter <i>milu</i> , the default being ILU(0) (→ Section 6.3.2). At the expense of a higher computational cost, the amount of memory required can be (slightly) reduced by setting <i>ilu_speed</i> =0 (→ Section 6.3.2).
		2	ILUT . Regarding the setting of the standard threshold values, generally known as <i>lfil</i> and <i>droptol</i> , → Section 6.3.2.
		3	ILUTP or MILUTP , depending on the hidden parameter <i>milu</i> , the default being ILUTP (→Section 6.3.2). Same as before except that column pivoting is used.
		5	Block relaxation . Depending on the setting of the hidden parameters <i>nrd</i> and <i>nru</i> , relaxation will be either by Gauss-Seidel or by ω -Jacobi (→ Section 6.3.1), the default being Gauss-Seidel C/F-relaxation. Only relevant for <i>nsys</i>>1 and point-based approaches. Otherwise, <i>nxtyp</i> will be reset to 0.
		6	Block ILU . Block version of ILU(0). Only relevant for <i>nsys</i>>1 and point-based approaches. Otherwise, <i>nxtyp</i> will be reset to 1.
		9	Uzawa smoothing . Special-purpose smoother for special types of coupled systems (<i>nsys</i> >1). This is not yet released for general use.

ILU-type smoothing

Generally, ILU-type smoothers are very robust. However, they need a substantial amount of additional memory. Very often, compared to straightforward Gauss-Seidel relaxation, the use of ILU for smoothing does not pay.

Under certain conditions it may be efficient to use ILU-type smoothing **only on the finest level** and switch to plain relaxation on the coarser levels. This can be achieved via the hidden parameter *nxtyp_coarse* (→ Section 6.3.2).

Uzawa smoothing

Classical Uzawa has been used as a model for developing a new smoother in SAMG. A typical target area for the new Uzawa smoother is applications based on Biots law of consolidation where we essentially deal with coupled elasticity and pressure problems. Generally, Uzawa smoothing is an option for saddle point problems for which the standard Gauss-Seidel diverges due to the lack of block diagonal dominance.¹⁵

¹⁵ More information will be given soon. Should you be interested in testing, please contact our service team.

5.4.3 Specifying a primary matrix (for point-based approaches)

The remaining digits of the parameter *nsolve* are only relevant if any of the *point-based* approaches has been selected (ie, *napproach*=3, 4 or 5)¹⁶. In that case, according to the description in Section 3, coarsening the given problem is done “at the level of points”, based on a so-called (scalar) **primary matrix**, P . A primary matrix is specified by properly setting the below (sub-) parameters **internal** and **nprim**. Generally, unless set to zero, the parameter *nprim* marks the *nprim*-th unknown to be the so-called **primary unknown**.

Parameter	Explanation		
Definition of primary matrix P (for point-based approaches)			
nsolve	[3:3], internal	Indicates whether the primary matrix , P , is defined internally to SAMG (ie, automatically computed) or whether it is defined externally (ie, user-provided).	
		=0	P is defined externally. In this case, $nprim \neq 0$ is required (see next table) and the system submatrix corresponding to the primary unknown will be used as primary matrix. That is, in the terminology of Section 3.2, $P=A_{[nprim,nprim]}$. Remarks: (1) Note that the primary unknown may be a dummy unknown ($\rightarrow npr_is_dummy$ in the next table). (2) If $nsys=1$, the only legal setting is $nprim=1$. That is, the primary matrix is necessarily A itself: $P=A$.
		>0	P is defined internally. The concrete value of <i>internal</i> selects a specific definition of P . Currently implemented:
		1,2	“Geometric” coarsening: The entries of P are based on point locations. Remarks: These settings are legal for any $nsys \geq 1$ if coordinates are available ($\rightarrow$ Section 7.3). While, in case 1, entries are computed based only on <i>distances</i> , in case 2, also <i>positions</i> of points relative to each other are taken into account.
		3,4	Block coarsening: The entries of P are based on the norms of the coupling blocks $A_{(i,j)}$. Remarks: This setting requires $nsys > 1$. The concrete norm is selected by the parameter <i>prim_norm</i> described in Section 6.2.6, the default being the maximum norm. The difference between the settings 3 and 4 is only in the definition of the diagonal entries of P : In case 3, they are defined as the negative off-diagonal sum. In case 4, they are defined to be the norm of the respective diagonal block, $A_{(i,i)}$.
		5	Variant of previous setting – special test version

¹⁶ An exception is *nint_weights* (=nsolve[7:7]) which is also relevant for *napproach*=1 or 2.

Important: If the primary matrix is defined internally, it is not enough to only define *how* to evaluate the entries of P . In addition, one also needs to prescribe a pattern, indicating at which matrix positions non-zero entries are really to be evaluated. This is done via the parameter $nprim$:

- If a primary unknown is specified, that is if $0 < nprim \leq nsys$, the non-zero pattern of P is copied from that unknown.
- If **no** primary unknown is specified, that is if $nprim=0$, the pattern of P is defined to be “maximum”: First, a row is defined at each point where at least one variable is defined. Second, an off-diagonal entry p_{ij} in the primary matrix is non-zero if *any* of the variables at point i has a non-zero off-diagonal coupling to a variable at point j , ie, if there exists a non-zero entry in $A_{(i,j)}$.

Parameter		Explanation	
Specifying a primary unknown			
nsolve	[4:5], nprim	Integer with $0 \leq nprim \leq nsys$. Unless $nprim=0$, this parameter marks the $nprim$ -th unknown as primary unknown . The purpose of a primary unknown depends on whether or not $internal=0$ (see above). Recall: Only those unknowns can be marked “primary” which are defined everywhere (with the possible exception of points where no unknowns are defined at all).	
	[6:6], npr_is_dummy	Relevant only if $nprim \neq 0$ in which case this parameter indicates whether or not the primary unknown is a dummy unknown ¹⁷ .	
		0	Primary unknown is a physical unknown.
		1	Primary unknown is a dummy unknown. Recall: A dummy unknown has to be assigned the highest number, $nsys$ ($\rightarrow$ Section 4.2).





Remark: If P is defined internally (ie, $internal > 0$), the hidden parameter $prim_print$ can be used to dump the finest-level primary matrix to a file (→ Section 6.2.6).



¹⁷ The only purpose of dummy unknowns is to control the overall AMG strategy. Hence, dummy unknowns should be ignored, for instance, in computing residuals. $npr_is_dummy=1$ tells SAMG to treat the current primary unknown as a dummy unknown. **Note that the setting of this parameter is not yet systematically exploited! In particular, dummy unknowns are unnecessarily taken into account in computing residuals.**

5.4.4 Specifying details of interpolation (all solution approaches)

The following subparameters, *nint_weights* and *nint_pat*, are used to select the **SW-pattern** (ie, the pattern of strong/weak couplings) and the process of how to compute the concrete **weights** of interpolation. The meaning of these parameters is as follows:

Parameter	Explanation		
Selection of interpolation weights and pattern			
nsolve	[7:7], nint_weights	Selects the strategy of how to compute the interpolation weights.	
		0	Selects a default strategy (→ comments below).
		1	A-interpolation. Interpolation weights are computed based on entries in the original matrix <i>A</i> .
		2	C1-interpolation. Interpolation weights are computed based on coordinates, that is, distances between neighboring points. This requires that coordinates are available (→ Section 7.3).
		3	C2-interpolation. Same as previous, except that weights of closely neighbored points (ie, “clusters” of points) are adjusted appropriately in order to avoid an interpolation which is biased in certain directions.
		4	P-interpolation. Interpolation weights are computed based on entries in the primary matrix <i>P</i> . Only relevant for point-based approaches, <i>napproach</i>=3-5 (see above).
		5	For <i>napproach</i> =5 (ie, block interpolation) this setting provides an alternative to <i>nint_weights</i> =1, exploiting highly efficient routines from Intel's MKL. If either <i>napproach</i> ≠5 or the MKL is not available, <i>nint_weights</i> =1 will be used instead. Otherwise, <i>nint_weights</i> =5 will actually become the default.
	[8:8], nint_pat	Selects the pattern of strong/weak couplings ("SW-pattern") on which the interpolation is based. In general, depending on the concrete application, there are various ways to select this pattern. Only relevant for <i>napproach</i>=4, and only if <i>nprim</i>>0.	
		0	Selects a default strategy (→ comments below).
		1	Select SW-pattern according to the “natural” pattern of strong and weak couplings as defined by the entries of the submatrix <i>A</i> _[<i>nprim</i>,<i>nprim</i>] .
		2	Select SW-pattern according to the “natural” pattern of strong and weak couplings as defined by the entries of the matrix <i>P</i> .

Since the above settings need some interpretation and since not all of them make sense in all approaches, we want to comment on the individual cases.

The variable-based approach

In the variable-based approach, by default, interpolation is constructed based on the entries of the matrix A (equivalent to setting `nint_weights=1`). Alternatively, interpolation is allowed to be defined based on geometric distances by selecting `nint_weights=2` or `=3`. The setting `nint_weights=4` is not allowed.

The unknown-based approach

In the unknown-based approach, by default, interpolation to the n -th unknown is constructed based on the matrix entries of the submatrix $A_{[n,n]}$ (equivalent to setting `nint_weights=1`). Alternatively, interpolation is allowed to be defined based on geometric distances by selecting `nint_weights=2` or `=3`. This is analogous to the variable-based approach, except that each unknown n is interpolated separately, based on an interpolation pattern as defined by the submatrix $A_{[n,n]}$. The setting `nint_weights=4` is not allowed.

The point-based approach

For point-based approaches, we have to distinguish **scalar** (`nsys=1`) and **coupled systems** (`nsys>1`):

For **scalar systems**, all point-based approaches (`napproach=3, 4` or `5`) naturally coincide. If `nint_weights=0` (default), the computation of interpolation weights is based on the primary matrix P (equivalent to `nint_weights=4`). All of the above settings of `nint_weights` are legal. The SW-pattern is selected by setting the parameter `nint_pat`. If `nint_pat=0` (default), the SW-pattern of P is selected (equivalent to `nint_pat=2`).

For **coupled systems**, we need to distinguish the different approaches, `napproach=3, 4` and `5`:

- **`napproach=3`, separate interpolation for each unknown.** By default, interpolation of the i -th unknown is based on the diagonal block $A_{[i,i]}$ of A (equivalent to setting `nint_weights=1`). If `nprim=0`, all of the above `nint_weights` settings are legal. If `nprim ≠ 0`, `nint_weights=1-3` are legal, `nint_weights=4`, however, makes no sense. In all legal cases, the interpolation pattern is based on the SW-pattern of $A_{[i,i]}$.
- **`napproach=4`, same interpolation for each unknown.** If `nint_weights=0` (default), computation of the interpolation weights is based on the primary matrix P (equivalent to setting `nint_weights=4`). If `nprim=0`, all settings of `nint_weights` are allowed except for `nint_weights=1`. The interpolation pattern is based on the SW-pattern of P . If `nprim ≠ 0`, all settings of `nint_weights` and `nint_pat` are allowed. If `nint_pat=0` (default), the SW-pattern of P is selected (equivalent to `nint_pat=2`).
- **`napproach=5`, block-based interpolation.** The default interpolation is based on the coupling blocks $A_{[i,j]}$ defined by the matrix A . Depending on whether or not the Intel MKL library is available, this corresponds to the setting `nint_weights=5` or `nint_weights=1`, respectively ($\rightarrow$ `nint_weights` in the table above). In fact, these are the only choices here: all other settings of `nint_weights` are ignored, that is, they are automatically reset to `nint_weights=5` or `nint_weights=1`, respectively.

5.4.5 Summary

The parameter *nsolve* comprises a lot of information characterizing the overall AMG approach. In this section, for convenience, we summarize the legal settings of *nsolve* and all of its subparameters. More precisely, for any of the settings of *napproach*, *internal* and *nprim*, the below tables list the corresponding legal settings of interpolation parameters *nint_weights* and *nint_pat* and what this means in terms of the type of interpolation **weights** as well as the **SW-pattern**.

Variable-based approach:

<i>nsolve</i> [1:1] <i>napproach</i>	<i>nsolve</i> [3:3] <i>internal</i>	<i>nsolve</i> [4:5] <i>nprim</i>	<i>nsolve</i> [7:7] <i>nint_weights</i>	<i>nsolve</i> [8:8] <i>nint_pat</i>	<i>weights</i>	<i>SW-pattern</i>
1	ignored	ignored	0,1	ignored	A	A
			2	ignored	distances	A
			3	ignored	positions	A
			4	error		

Unknown-based approach:

<i>nsolve</i> [1:1] <i>napproach</i>	<i>nsolve</i> [3:3] <i>internal</i>	<i>nsolve</i> [4:5] <i>nprim</i>	<i>nsolve</i> [7:7] <i>nint_weights</i>	<i>nsolve</i> [8:8] <i>nint_pat</i>	<i>weights</i>	<i>SW-pattern</i>
2	0	ignored	0,1	ignored	all $A_{[i,i]}$	all $A_{[i,i]}$
			2	ignored	distances	all $A_{[i,i]}$
			3	ignored	positions	all $A_{[i,i]}$
			4	error		
	>0	reset to internal=0 after warning				

Point-based approach for scalar systems:

<i>nsolve</i> [1:1] <i>napproach</i>	<i>nsolve</i> [3:3] <i>internal</i>	<i>nsolve</i> [4:5] <i>nprim</i>	<i>nsolve</i> [7:7] <i>nint_weights</i>	<i>nsolve</i> [8:8] <i>nint_pat</i>	<i>weights</i>	<i>SW-pattern</i>
3,4,5	0	0	error			
		1	0,1,4	ignored	A=P	A=P
			2	ignored	distances	A=P
			3	ignored	positions	A=P
	1,2,5	0,1	1	1	A	A
				0,2	A	P
			2	1	distances	A
				0,2	distances	P
			3	1	positions	A
				0,2	positions	P
			0,4	1	P	A
				0,2	P	P
	3,4	error				

Point-based approach and coupled systems:

<i>nsolve[1:1]</i>	<i>nsolve[3:3]</i>	<i>nsolve[4:5]</i>	<i>nsolve[7:7]</i>	<i>nsolve[8:8]</i>	<i>weights</i>	<i>SW-pattern</i>	
<i>napproach</i>	<i>internal</i>	<i>nprim</i>	<i>nint_weights</i>	<i>nint_pat</i>			
3	0	0	error				
		>0	0,1	ignored	all A _[i,i]	all A _[i,i]	
			2	ignored	distances	all A _[i,i]	
			3	ignored	positions	all A _[i,i]	
			4	error			
	>0	0	0,1	ignored	all A _[i,i]	all A _[i,i]	
			2	ignored	distances	all A _[i,i]	
			3	ignored	positions	all A _[i,i]	
			4	ignored	P _{max}	all A _[i,i]	
		>0	0,1	ignored	all A _[i,i]	all A _[i,i]	
			2	ignored	distances	all A _[i,i]	
			3	ignored	positions	all A _[i,i]	
			4	error			
4	0	0	error				
		>0	0,1,4	ignored	A _[nprim,nprim] = P	A _[nprim,nprim] = P	
			2	ignored	distances	A _[nprim,nprim] = P	
			3	ignored	positions	A _[nprim,nprim] = P	
	>0	0	1	error			
			2	ignored	distances	P _{max}	
			3	ignored	positions	P _{max}	
			0,4	ignored	P _{max}	P _{max}	
		>0	1	1	A _[nprim,nprim]	A _[nprim,nprim]	
				0,2	A _[nprim,nprim]	P _{nprim}	
			2	1	distances	A _[nprim,nprim]	
				0,2	distances	P _{nprim}	
			3	1	positions	A _[nprim,nprim]	
0,2				positions	P _{nprim}		
0,4			1	P _{nprim}	A _[nprim,nprim]		
			0,2	P _{nprim}	P _{nprim}		

5.5 Selecting SAMG's cycling process

The type of SAMG multigrid cycles, and whether cycling is to be done “stand-alone” or as a preconditioner, is selected by the primary parameter *ncyc*.

Parameter	Explanation		
Cycling and acceleration strategies			
ncyc	Integer. Specifies cycling and acceleration strategies. Standard choice: <i>ncyc</i> =11030.		
	>0	Standard multigrid cycles.	
	<0	Modified multigrid cycles (*-cycles, → Section 5.5.2).	
	[1:1], igam	Selects the “shape” of multigrid cycles.	
		1	V-cycle: standard choice
		2	F-cycle: recursive improvement, more expensive
		3	W-cycle: further recursive improvement, still more expensive
		4	VV-cycle: This corresponds to a V-cycle with the correction from the first coarser level being computed based on two V-cycles.
		The following choices are very special: by setting <i>igam</i> =5-6 multigrid-specific parameters are ignored (such as <i>levelx</i> and <i>clsolver_finest</i>) and immediately the respective classical one-level methods are called.	
		5	Diagonally pre-conditioned Krylov method (→ Section 7.7.1, → <i>ncgrad</i> below).
		6	Classical ICCG (→ Section 7.7.2). This method is only applicable to symmetric matrices for which just the lower tridiagonal part is supplied (→ <i>isym</i> in Section 5.1).
	[2:2], ncgrad	Selects the accelerator (Krylov method) to be used.	
		0	A default accelerator is used, defined by the hidden parameter <i>ncgrad_default</i> (→ Section 6.4.2). Remark: Only if both, <i>ncgrad</i> =0 and <i>ncgrad_default</i> =0, stand-alone cycling is performed!
		1	Conjugate gradient (CG) ¹⁸
		2	BiCGstab
		3	flexible GMRes
		4	flexible CG
	[3:3], nkdim	If <i>ncgrad</i> =3-4, <i>nkdim</i> is used to select the dimension of the Krylov space. If <i>ncgrad</i> =1-2, <i>nkdim</i> is used to select re-start points. Example: <i>nkdim</i> =5 means a Krylov dimension of 6; standard CG and BiCGstab will simply be re-started every 6 iterations.	
		0	Selects a default dimension (→ <i>nkdim_default</i> , Section 6.4.2).
1-8		Dimension = <i>nkdim</i> +1	
9		Dimension = 20	
[4:], ncycle	Maximum number of cycles to be performed, ≥0.		

Various hidden parameters are used to (re-)define important components of an AMG cycle, all of which are initialized to reasonable default values. All available hidden parameters will be introduced later in this manual. In the following we just summarize important aspects and give concrete links to detailed descriptions.

¹⁸ There are actually two implementations of CG available. While the default version is somewhat more robust in numerically critical cases, the alternative one is slightly cheaper (→ *ntyp_accel*, Section 6.4.3).

5.5.1 One-level solvers

By forcing SAMG to operate on only one level, multigrid cycling degenerates to classical one-level iterative methods. Furthermore any of the coarsest-level solvers can be used as one-level solver. Also note the special setting *igam*=5-6 above. For more details, see Sections 7.6 and O.

5.5.2 Modified cycles (*-cycles)

By giving *ncyc* a negative sign, special modifications of a cycle are activated which aim at the optimization of corrections from coarser grids. Such optimizations are based on the *energy norm* or *residual norm* depending on whether the given problem is symmetric or non-symmetric, respectively ($\rightarrow$ *isym*, Section 5.1). In certain cases - in particular, for aggregation-based coarsening strategies - convergence may be improved substantially. However, this is at the expense of additional work and it depends on the situation whether or not the modified cycle finally pay.

There are different types of cycle-modifications, the default one operating only on coarser levels. This can be changed via the hidden parameter *nstar_typ* ($\rightarrow$ Section 6.3.3).

Remark: Since, compared to standard cycles, the modified ones yield *non-linear* iterative processes, *-cycles should be used either stand-alone or in combination with “flexible” accelerators (flexible CG or GMRes, $\rightarrow$ *ncgrad*). See also the figure in the next Section 5.5.3.



5.5.3 Choice of accelerator

For mathematical reasons, not all of the above accelerators can always be used. Essentially symmetry and cycle-type determine which accelerators can be used. In the following table, the shaded positions mark illegal combinations. If an accelerator is used illegally, SAMG will replace it by one which is probably legal. More precisely, replacement will be done as indicated by the blue arrows. Note, however, that SAMG cannot really know, which accelerator is definitely ok. **In the end, a mathematically correct choice is the user's responsibility!**

		standard cycles (<i>ncyc</i> >0)		modified cycles (<i>ncyc</i> <0)	
		symmetric (<i>isym</i> =1)	nonsymm. (<i>isym</i> =2)	symmetric (<i>isym</i> =1)	nonsymm. (<i>isym</i> =2)
CG	(<i>ncgrad</i> =1)				
BiCGstab	(<i>ncgrad</i> =2)		↓	↓ ↓	↓ ↓ ↓
flexible GMRes	(<i>ncgrad</i> =3)		↑	↓ ↓	↓ ↓ ↑
flexible CG	(<i>ncgrad</i> =4)			↓	↓

Some additional comments:

- The switching between accelerators as indicated in the above table can be overruled by means of the hidden parameter *force_accel* ($\rightarrow$ Section 6.3.3).
- By default, a single AMG cycle is done within each accelerator step. The number of “inner cycles” can be increased via the hidden parameter *inner_accel* ($\rightarrow$ Section 6.3.3).
- Conjugate gradient requires not only the matrices to be symmetric but also the complete cycle. This, in turn, requires pre- and post-smoothing to be applied “symmetrically”. If CG is selected and assuming the matrix *A* to be symmetric according to the setting of *isym*, SAMG may modify the selected smoothing process in an attempt to symmetrize it ($\rightarrow$ Section 6.3.1).

5.5.4 Stopping criteria

Cycling will stop when the maximum number of cycles, defined by *ncycle*, is reached, or when the termination criterion, defined by *eps* (→ Section 5.2), is met, whatever happens first. Note that, in addition to these two “standard” stopping criteria, there are several others described in Section 7.2.

Remark: The maximum number of cycles, *ncycle*, is merely a safety limit. You should monitor the true number of cycles used to solve a given problem (→ output parameter *ncyc_done*, Section 5.9). If the true number of cycles equals the prescribed maximum number, be careful: the setting of *ncycle* might have been too small, or there might have been problems with convergence¹⁹. To be on the safe side, the primary parameters *res_in* and *res_out* should be monitored (→ Section 5.9).



5.5.5 Number of smoothing steps

On each level (except for the very coarsest), just one pre- and post-smoothing step is performed by default. This can be changed by modifying the (sub-) parameters *nrdx* and *nrux* of the hidden parameters *nrd* and *nru*, respectively (→ Section 6.3.1). By default, the number of pre- and post-smoothing steps is the same on each level. If requested, the number of smoothing steps can be increased towards coarser levels (→ *ndyn_smo*, Section 6.3.1).

5.5.6 Smoothing by relaxation

Unless the standard smoothing strategy has been changed by explicitly setting the (sub-) parameter *nxtyp* to some non-zero value (→ Section 5.4.2), Gauss-Seidel variable-wise relaxation in C/F order is performed for smoothing. That is, on each level, first all C-variables are relaxed followed by all F-variables (analogous to red/black relaxation in geometric multigrid). Various details of relaxation, in particular the order of relaxation, can be changed by modifying the hidden parameters *nrd* and *nru* (→ Section 6.3.1).

5.5.7 Smoothing on coarser levels

By default, the same smoother is used on all levels. However, should a non-relaxation method have been selected for smoothing (→ *nxtyp*, Section 5.4.2), on coarser levels one can tell SAMG to still use relaxation for smoothing (→ *nxtyp_coarse*, Section 6.3.2).

5.5.8 Type of coarsening and interpolation

By default, some standard coarsening and interpolation process will be done. Full control is obtained via the hidden parameters *ncg* and *nwt* (→ Section 6.2). For convenience, some typical choices can also be done by setting the primary parameter *iswtch* accordingly (→ *n_default*, Sections 5.6.3 and 6.4.4).

5.5.9 Termination of coarsening

Usually, coarsening terminates if the number of variables drops below a given value or the number of levels exceeds an upper limit, whatever is reached first. This can be changed by adjusting the hidden parameters *levelx* and/or *nptmn* (→ Section 6.2.4). Note that there are various other stopping criteria for the coarsening, also described in Section 6.2.4.

¹⁹ It is often the case that the parameter *eps* has been chosen so small that the corresponding termination criterion cannot be satisfied simply because of roundoff (due to the inaccurate representation of numbers).

5.5.10 Coarsest-level solver

By default, the coarsest-level solution is by a **direct solver** (sparse Gauss elimination). This can be changed by adjusting the hidden parameter *nrc*. In this context, also see the hidden parameter *nrc_emergency*. All parameters are described in Section 6.3.4.

5.5.11 Pre-cycling

If SAMG is used as a preconditioner, occasionally it may be reasonable to perform a few stand-alone cycles before an accelerator is activated. By default, no such “pre-cycles” are performed. This can be changed by means of the hidden parameter *iter_pre* (→ Section 6.4.3).

5.6 General control switch

The primary parameter *iswtch* comprises 5 subparameters which control various different aspects:

- **iswit**: repeated calls to SAMG (reusing parts or all of previous setup data),
- **iextent**: dynamic memory management,
- **n_default**: default settings of certain hidden parameters,
- **norm_typ**: type of norm to be used in computing residuals,
- **ioscratch**: I/O unit of scratch files (used for the memory management),
- sign of **iswtch**: display of *error* (in addition to *residual*) histories.

Hints for beginners: If a *single* linear system is to be solved, *iswtch*=51 (equivalent to *iswtch*=5140, → Section 5.6.3) is a typical setting. Situations in which different settings may be better are, eg,

- if the overall memory requirement is to be reduced²⁰. In such cases, *iswtch*=5142 will reduce memory requirement substantially (at the expense of a slower convergence).
- if you have to solve a *series* of linear systems for which the matrices stay the same or change only “slowly” from one system to the next (typical for time-dependent problems). In such cases, setup cost may substantially be reduced by reusing setup data from previous calls to SAMG. This can be done by setting the subparameter *iswit*, accordingly.

Details on *iswtch* are described in the following.

5.6.1 Repeated calls to SAMG, reusing previous setup data

The first digit of *iswtch*, the subparameter **iswit**, tells SAMG whether (and which) setup data from the previous SAMG call is to be reused. If SAMG is called just once (or repeatedly but for “substantially different” applications), *iswit*=5 is the normal setting, meaning that no setup data is to be reused. Settings *iswit*=1-3 do reuse setup data from a previous run. Clearly, these settings require SAMG’s memory to remain allocated between different calls which is achieved by setting *iswit*=4 in the very first call to SAMG. In detail:

Parameter	Explanation		
General control switch: Repeated calls to SAMG			
iswtch	Integer. General control switch. Standard setting for beginners: <i>iswtch</i> =51.		
	[1:1], iswit	Controls the reuse of setup data during repeated SAMG calls. See also the hidden parameter <i>struct_ident</i> (→ Section 6.2.8).	
		5	Complete SAMG run. Upon return, memory is released.
		4	Same as 5 except that memory is not released.
		3	SAMG run with partial setup : Reuse coarser grids and interpolation from previous run but update the Galerkin operators. Upon return, memory is not released.
		2	SAMG run with no setup : Reuse complete setup data including the Galerkin operators from previous run. Upon return, memory is not released.
		1	Same as 2 except that SAMG assumes the matrix <i>A</i> to be exactly the same as in the previous run.

The above settings of *iswit* allow the (manually controlled) reuse of previous setup data. Note that there is a prototype SAMG module which attempts to **automatize** the reuse of previous setup data (→ Section 7.5). Some warnings:

²⁰ Typical case: very sparse matrices such as those corresponding to 5- or 7-point FD discretizations of poisson-like problems.

- When setup data is to be reused, the new matrix A is allowed to be different from the one used in the previous call as long as – at least – the number of variables as well as the vectors iu and ip are the same. However, unless the new A is “similar enough”²¹ to the previous one, reusing setup data may slow down convergence (or even cause divergence). **It is the user’s responsibility to employ some “convergence control”.**
- The option $iswit=3$ is more expensive but safer than the brute-force option $iswit=2$. For example, in case of symmetric positive definite problems, the use of $iswit=3$ may reduce the speed of convergence but will never cause divergence. This is in contrast to $iswit=2$ which, if not monitored carefully, may yield a **divergent** method.
- If previous setup data is to be reused, it is not permitted to modify, at the same time, parameters which would necessarily change SAMG’s setup.



In order to manually **release setup data** (if not yet released by SAMG), just issue

```
call samg_refresh(ierr)
```

(→ Section 2.5). Clearly, if memory has been released this way, the next call to SAMG must not be with $iswit=3$, 2 or 1 but requires the setting $iswit=5$ or $iswit=4$.

5.6.2 Memory extension switch

SAMG’s initial dimensioning is selected by setting the double parameters a_cmplx , g_cmplx , p_cmplx and w_avrge (→ Section 5.7). Since the exact dimensions are not known in advance, SAMG may need to extend the allocated work space at run time. The rules for this are specified by the subparameter **ixtent** (“memory extension switch”):

Parameter		Explanation	
General control switch: Memory extension switch			
iswtch	[2:2], iextent	Defines the behavior of SAMG in case the limits of the initial dimensioning have been reached. Standard choice: iextent=1.	
		0	SAMG returns with an error code.
		1	SAMG attempts to allocate extended memory and continues: The previously allocated data is temporarily copied in-core. If this is not possible, it is copied to disk instead ²² .
		2	SAMG attempts to allocate extended memory and continues: The previously allocated data is temporarily copied in-core. If this is not possible, SAMG terminates with an error code.
		3	SAMG attempts to allocate extended memory and continues: The previously allocated data is temporarily copied to disk ²² .

Remark: If SAMG needs to temporarily copy data to disk, the I/O unit used is defined by the parameter $ioscratch$ (see further below). Should $ioscratch$ be zero or not specified, the unit will be given by the parameter $ioscratch_default$ instead (→ Section 6.4.2). The default scratchfile is [scratch.samg](#). This name can be changed via the hidden parameter $scratchfile$ (→ Section 6.4.2).



²¹ Essentially, this requirement refers to the similarity of eigenfunctions corresponding to the smallest eigenvalues.

²² Data (reals or integers) is written to disk in “blocks”. The maximum size of a block (in terms of number of data) is defined by the hidden parameter $impldo_read_write_len$, the default setting being **2,000,000**. This default setting ensures that reading/writing should work on all hardware. If required, this parameter can be changed like any other hidden parameter.

Nowadays, problem sizes are getting so large that there is an increasing risk of **integer overflow** which is further increased due to SAMG's hierarchy. Unfortunately, integer overflows are not detected by the system. SAMG attempts to detect integer overflows, at least those which occur in memory extension routines.

Parameter	Explanation	
Detecting integer overflows		
ihuge	Integer. Used to detect integer overflows in memory extension routines.	
	=0	SAMG checks if an integer occurs which is larger than the largest representable integer ²³ . If yes, SAMG terminates and returns with an error code.
	> 0	SAMG checks whether an integer occurs which is larger than <i>ihuge</i> . ²⁴

5.6.3 Selecting a coarsening strategy via the primary parameters

Digits 3-4 of *iswch*, the sub-parameter **n_default** ("default switch"), provides a simple means to select typical coarsening strategies which, in general, require the setting of certain hidden parameters (→ *ncg* and *nwt*, Section 6.4.4). Rather than explaining the corresponding settings in detail, we roughly outline what they mean. Regarding further details, → Section 6.4.4.

For each of the below eight ranges of settings (four numbers each), the respective lowest number is the "standard value". Increasing the corresponding second digit causes coarsening to be increasingly aggressive (requiring less memory) at the expense of a slower convergence. Whether or not a more aggressive coarsening pays in terms of CPU time, depends on the application. Generally, aggressive coarsening should only be selected if memory becomes an issue.

Parameter	Explanation	
General control switch: Selecting specific coarsening strategies		
iswch	[3:4], n_default	Selects specific coarsening strategies (→ Section 6.4.4). Typical choices: n_default=40 for standard and n_default=42 for aggressive coarsening. Remark: <i>n_default=0</i> is equivalent to <i>n_default=40</i> . ²⁵
		10-13 To be used if there are no “critical” <i>positive</i> off-diagonal entries. Interpolation is done in the most simple way.
		15-18 Same as previous, except that more effort is invested in the construction of interpolation.
		20-23 20-23 is an alternative to 10-13 in case there are “critical” <i>positive</i> off-diagonal entries.
		25-28 As 20-23, except that more effort is invested in the construction of interpolation. This is currently being re-designed, do not use!
		30-33 35-38 40-43 Completely analogous to 10-13, 15-18, 20-23, 25-28 except that rows with <i>only</i> positive entries are taken into account.
		45-48 This is currently being re-designed, do not use!

²³ According to the Fortran intrinsic function **huge** which returns the largest possible integer for the current integer representation.

²⁴ That is, *ihuge*=0 is equivalent to setting *ihuge*=**huge**(1)-1.

²⁵ Just for completeness: the setting *n_default*=99 is equivalent to the setting *n_default*=20.

5.6.4 Norms and scratch files

The sub-parameter **norm_typ** selects the norm for computing residuals, and **ioscratch** defines the unit number used for scratchfiles.

Parameter		Explanation	
General control switch: Norms and scratch files			
iswtch	[5:5], norm_typ	Selects the norm to be used in computing residuals. Standard choice: norm_typ=0 .	
		0	L2-norm (standard): $\sqrt{\sum_i \left(f_i - \sum_j a_{ij} u_j\right)^2}$
		1	L1-norm: $\sum_i f_i - \sum_j a_{ij} u_j $
		2	Maximum norm: $\max_i f_i - \sum_j a_{ij} u_j $
		3	L2-norm (reserved for test purposes, → Section 7.8.2)
	[6:7], ioscratch	Unit number for scratch files used by SAMG for its memory management (→ Section 5.6.2), see also remarks below	

Should **ioscratch** be zero or not specified, the I/O unit will be set according to the hidden parameter **iocratch_default** instead (→ Section 6.4.2). The default scratchfile is **scratch.samg**. This name can be changed via the hidden parameter **scratchfile** (→ Section 6.4.2).

5.6.5 Displaying histories of errors

If **iswtch** is given a **negative sign**, and if a file containing the solution is available, SAMG will display not only the history of *residuals*, but also that of *errors*. This is mainly for test purposes. For more details, → Section 7.8.2.

5.7 Initial dimensioning

SAMG is a completely dynamic program which makes it impossible to precisely predict its memory requirements: The final memory requirement depends on both the given matrix and the selected solution strategy. The double precision input parameters

a_cmplx, **g_cmplx**, **p_cmplx** and **w_avrge**

define the dimensions used to allocate SAMG's initial memory. These are *relative* quantities so that the corresponding amount of initial memory is not of a fixed size, but rather increases linearly with the problem size. For instance, **a_cmplx** corresponds to the so-called "operator complexity",

$$a_cmplx = \sum_i |A_i| / |A_0|.$$

Reasonable settings for the above parameters are described in the following.

5.7.1 Automatic adaptation of the work space

For convenience, there are two possibilities to simplify the dimensioning process by telling SAMG to automatically adjust itself to the memory requirements:

The standard way. Set all four parameters to **0.0** and turn the memory extension switch ON (by setting **iextent=1**, → Section 5.6.2). In this case, SAMG will start with some default dimensioning and whenever this turns out to be insufficient, memory will automatically be extended. Note, however,

that this will also increase the setup cost to some (limited) extent. (Regarding the default dimensions proposed by SAMG, → Section 6.4.2.)

Minimizing memory. Set all four parameters to **1.0** and turn the memory extension switch ON (by setting *iextent*=1, → Section 5.6.2). In this case, SAMG will start with “no memory” and increase it if necessary. Compared to the previous option, this may increase the setup cost further. However, it will attempt to minimize the amount of wasted memory.

5.7.2 Explicitly setting the dimensioning parameters

To avoid the extra computational cost caused by the automatic adaptation of the previous section (due to allocation and copying processes), one might wish to define the dimensioning explicitly. As long as memory is no problem, one may simply select large enough values for the dimensioning parameters and no memory extension at run time will be necessary. Clearly, since there is no final guarantee, we generally recommend to always turn the memory extension switch ON (by setting *iextent*=1, → Section 5.6.2)!

In the following we give some rough guidelines which may help to explicitly set the dimensioning parameters:

a_cmplx. Should be an upper limit for the *operator complexity* which is defined as the ratio between the total number of matrix entries (summed over all AMG levels) and the number of entries in the given matrix (= *nna*). Depending on the problem and the AMG strategy chosen, this may be a value as low as 1.2, but it may also be higher than 4.0, say. **Typical values are 1.5-3.0.**

g_cmplx. Should be an upper limit for the *grid complexity* which is defined as the ratio between the total number of variables (summed over all AMG levels) and the number of variables in the given problem (= *nnu*). Depending on the problem and the strategy chosen, this may be a value as low as 1.2. **Usually, an upper limit is 2.0.**

p_cmplx. This parameter is relevant only if any of the point-based approaches is selected. *p_cmplx* should then be an upper limit for the *point complexity* which is defined as the ratio between the total number of points (summed over all AMG levels) and the number of points in the given problem. Depending on the problem and the strategy chosen, this may be a value as low as 1.2. **Usually, an upper limit is 2.0.**

w_avrge. Should be an upper limit for the *average row length of interpolation*, that is, the total number of interpolation weights used by SAMG, summed over all levels and divided by the total number of variables (summed over all levels). Depending on the problem and the strategy chosen, this may be a value as low as 1.5, but it may also be as high as 6.0, say. **A typical average value is 3.0.**

5.7.3 Dynamic adaptation at run time

If SAMG is used to solve a whole *sequence* of linear problems of similar size and type, realistic values for *a_cmplx*, *g_cmplx*, *p_cmplx* and *w_avrge* can dynamically be obtained by looking at the “true” memory requirement of a previous run. For details on how to exploit this → Section 7.11.1.

5.8 Checking and printing

This section details options available for checking the input matrix and for controlling the amount of printed output during the setup and the solution phases.

Remark: By default, print output will be send to the **console**. To change this, adjust the hidden parameters *logio* and *logfile*. Furthermore, the amount of (general, warning and error) messages can be reduced by means of the hidden parameter *mode_mess*. For details, → Section 6.4.1.



5.8.1 Input matrix checking

The parameter **chk_{tol}** is used to activate some checking of the input matrix. We **strongly recommend** activating these checks if there are doubts regarding the (logical) correctness of the provided data. Since the full checking process is relatively expensive, for real production runs it should be turned off.

Parameter	Explanation	
Checking of input matrix		
chk _{tol}	Double. Used to control the amount of checking of the input matrix.	
	< 0.0d0	No checking. This is the standard for production runs.
	= 0.0d0	The input matrix is checked for logical correctness.
	> 0.0d0	Enhanced checking for zero entries, symmetry, rowsum zero and the like. Most of these checks require a reasonable tolerance to be prescribed. This tolerance is given by the concrete value of chk _{tol} . Standard choice: chk _{tol} =1.0d-7.

Remark: Setting $chk_{tol} \geq 0.0d0$ will increase the run time to a substantial extent. This option is meant to be used only during a testing phase as long as there remain doubts regarding the correctness of the input matrix.



5.8.2 Print output during the solution phase

The parameter **iout** controls the amount of print output during SAMG's solution phase. The standard setting is *iout*=2 which displays a table of the most important input parameters (primary as well as hidden), the convergence history, as well as a statistics on work, memory and timings.

Parameter	Explanation		
Print output during the solution phase			
iout	Integer. Controls print output related to SAMG’s solution phase. Standard choice: iout=2 .		
	<0	No printout, except for warnings and errors ²⁶ .	
	=0	Minimal output on results and timings.	
	>0	Additional print output specified by the individual digits.	
	[1:1], iout1	1	Table of input data and work statistics.
		2	In addition: history of cycling process.
		3	Extended history: including all levels, full smoothing steps.
		4	Extended history: including all levels, partial smoothing steps.
	[2:2], iout2	0	No action.
		1	Display <i>most relevant</i> hidden parameters.
		2	Display <i>all</i> hidden parameters.
		3	Display <i>all</i> hidden parameters in a single list.

²⁶ In order to suppress really **all** printout, set the hidden parameter *mode_mess* accordingly, see Section 6.4.1.

5.8.3 Print output during the setup phase

The parameter **idump** controls the amount of print output *during the setup phase* including some additional checking. The experienced user may wish to select **idump=-1** which suppresses all output and most of the checking. On the other hand, the setting **idump=0** will cause a coarsening history to be displayed which may be quite instructive, in particular, in connection with new applications.

Parameter	Explanation		
Print output during the setup phase			
idump	Integer. Controls print output related to SAMG's setup. Standard choice: idump=0.		
	<0	No printout, except for warnings and errors ²⁶ .	
	=0	Printout of coarsening history.	
	>0	Additional print output specified by the individual digits:	
	[1:1], idmp	1	Standard print output (coarsening history).
		>1	Reserved for dumping matrices to disk (→ Section 7.8.3).
	[2:2], igdp	Selects print output regarding the coarse levels . igdp>1 is only relevant for coupled systems. Otherwise: ignored.	
		0	No particular output.
		1	Display table on grids (full problem).
		2	Same for all submatrices (only if nsys>1).
	[3:3], iadp	Selects print output regarding the coarse-level matrices . iadp>1 is only relevant for coupled systems.	
		0	No particular output.
		1	Display table on coarse-level matrices (full problem).
		2	In addition: same info for all submatrices.
		3	In addition: connectivity info between unknowns.
	[4:4], iwdp	Selects print output regarding the interpolation matrices . iwdp>1 is only relevant for coupled systems.	
		0	No particular output.
		1	Display table on interpolation matrices (full problem).
		2	In addition: same info for all submatrices.
3		In addition: connectivity info between unknowns.	
[5:5], icdp	If non-zero, a statistics on memory requirement is printed on all levels		

Remark: The setting **idump=0** will increase the setup cost to some (limited) extent. Selecting any of the further options shown above will *substantially* increase the setup cost! Hence, for production runs, select **idump ≤ 0**.



5.9 Output parameters

The following table lists all output parameters. Note that residuals are measured in the L_2 -, L_1 - or the *maximum*-norm. The concrete norm is selected by the parameter *norm_typ* (→ Section 5.6.4), the default being the **L_2 -norm**.

Parameter	Explanation
Output parameters returned by SAMG	
res_in	Double. Residual of first guess. Usually, <i>res_in</i> is an output parameter only. However, by means of the hidden parameter <i>ntake_res_in</i> , you can tell SAMG that it should not recompute the residual of the first guess but rather use the value which is passed via <i>res_in</i> . For details, → Section 6.4.3.
res_out	Double. Residual of final approximation.
ncyc_done	Integer. Total number of cycles (iterations) performed.
u	Double vector $u(1:nnu)$, final approximation. In case a normalization is required, this will be done as follows: For scalar systems: If A is a row sum zero matrix (ie, $irow0=1$), the solution will be normalized so that $(u,1)=0$. For coupled systems: If the k -th unknown requires scaling (ie, if $iscale(k)=1$), the solution will be normalized so that $(u_{[k]},1)=0$.
ierr	Integer. Code number indicating errors or warnings. While <i>positive</i> code numbers indicate errors , <i>negative</i> code numbers indicate warnings . For a list of code numbers, → Section 9.
	=0 No error or warning detected.
	>0 SAMG terminated with a fatal error .
	<0 SAMG completed the solution process but returned with a warning .

Attention: The input vectors *ia*, *ja* and *a* will be **logically** unchanged upon return. However, note the following: The order of entries *within a row* may have changed (except for the diagonal entry which remains first). Accordingly, the pointer vector *ja* may be different from its input status as well. **Should the calling program rely on a specific order of the matrix entries in each row, both *a* and *ja* should be copies of their original versions!** Otherwise, one may safely ignore this warning.



5.10 Best practice

In this section we summarize some basic aspects in using SAMG which may be helpful for a beginner. In particular, we will **strongly recommend** to implement some **safety control** based on the output values of *ierr*, *res_in*, *res_out*, *ncyc_done* (→ Section 5.9) and others.

5.10.1 Logfile and other output

SAMG is usually applied in external simulation programs where it may be called hundreds of times in a single run and it is not practical to create a logfile each time SAMG is called (→ Section 6.4.1). Hence, most users turn off SAMG's output by setting ***idump* = -1** and ***iout* = -1** (→ Section 5.8). Some even turn all other messages off as well, including warnings and errors, by setting the hidden parameter ***mode_mess* = -3** (→ Section 6.4.1).

However, the latter should be done with great care. Recall that SAMG is a very complex solver package which dynamically and automatically attempts to define suitable hierarchies and efficient solution processes. In particular, it is no black-box solver which can be applied to all types of applications. Depending on the application, it may happen that performance is not satisfactory, or SAMG does not even work at all.

Hence, we do not recommend turning off the **warning** or/and **error** messages. Otherwise, important info regarding SAMG's performance will be lost and it may become difficult to track down the reason for some particular failure and/or misbehavior.

5.10.2 Checking the error parameter, *ierr*

Should ***ierr* > 0** upon return, a **fatal error** has happened and it generally makes no sense to continue without understanding what happened. In particular, it should be clarified whether the reason for the error was a user error which can be repaired (eg, error in parameter settings) or whether SAMG did not work correctly. In the latter case, it should be clarified whether it makes sense to just change the SAMG strategy, or whether we have a principle problem with SAMG if applied to the current application (eg, the application is simply not suited because of lack of ellipticity). In the latter case, one might consider switching to a one-level solver instead. Various approaches are available as part of SAMG (→ Sections 7.6 and 0).

Should ***ierr* < 0** upon return, a situation occurred which produced a **warning**. In this case it is good to know the warning messages given by SAMG and understand the reason for the warning. In many cases, one may continue the user program (eg, because some safety check has taken effect → Section 7.2). In other cases SAMG's efficiency may have dropped and it is up to the user to decide whether or not he can live with the situation. Finally, the warning may indicate that SAMG is close to fail in which case one might consider switching SAMG's strategy or switch to a different solver altogether, whatever seems more promising.

5.10.3 Checking the number of cycles performed, *ncyc_done*

To reach the maximum number of cycles, *ncycle*, is a regular termination criterion provided on input (→ Section 5.5). Hence, SAMG does not give any warning if ***ncyc_done* = *ncycle*** upon return. However, to be on the safe side, a user should then always compare *res_in* and *res_out* and check whether his requirements in terms of residual reduction ("eps-requirement" → Section 5.2) have been satisfied.

If not, one should first check whether the *eps*-requirement has been unrealistic to begin with (eg, *eps* was so small that the requirement cannot be reached simply because of the machine's limited accuracy). In this case, the requirement should be adjusted. If this is not the case, one should check

SAMG's speed of convergence. If this is resonable (average convergence factor much smaller than 1.0, say) one might want to just increase the value of *ncycle*. If however, average convergence turns out to be fairly slow (close to a reduction factor of 1.0 or even larger), one should check why this is so. If available, it is usually quite helpful to consider SAMG's convergence history more closely to detect some unexpected behavior. Quite often it will turn out that, for some reason, SAMG unintentionally decided to use a **one-level solver**, for instance because the given problem was simply too small.

Hint: As a matter of experience, the latter is quite often the case when people start playing with SAMG using small problems. One should note, however, that for too small problems ($\rightarrow$ *nptmn* in Section 6.2.4), SAMG may decide to not coarsen at all. In such cases, by default, the smoother will become the solver. Hence, depending on the application, convergence may become unexpectedly slow. Such cases can easily be avoided, see the next Section 5.10.4.



However, SAMG may also have decided to use a one-level method for other reasons, eg coarsening did not work properly. Since, as above, by default the smoother will become the solver, convergence now will *really* become slow. Again, Section 5.10.4 will give some hints of how to improve things.

Finally, SAMG's convergence may also be slow for other reasons, may be the type of problem is not suitable for algebraic multigrid. In this case, one might consider changing SAMG's strategy or switching to another solver altogether.

5.10.4 Unintentional use of one-level methods

If SAMG unintentionally decides to use a **one-level method**, by default a single SAMG cycle simply degenerates to a smoothing step, used as preconditioner for the currently selected accelerator ($\rightarrow$ end of Section 6.2.4). Hence, depending on the application and its size, convergence may become unexpectedly slow. In the following, we give some examples of what to do, more details are found in Section 7.6. The below parameters are hidden parameters ($\rightarrow$ Section 6.1).

5.10.4.1 Solving very small problems

For too small problems SAMG may decide to not coarsen at all. Hence, it may be better to select a suitable one-level solver for small problems to begin with. Various one-level solvers are available for that purpose, see Sections 7.6 and 0. For instance, assuming the number of variables, *nnu*, is smaller than *nptmax* ($\rightarrow$ Section 6.3.4.1), say, one might use a direct solver by setting

levelx=1²⁷, clsolver_finest=111 (or ***clsolver_finest=107***) .

Only if *nnu* will be larger than *nptmax*, go back to normal by resetting *levelx* back to its default value and deactivate *clsolver_finest*:

levelx=25; clsolver_finest=0.

5.10.4.2 Changing SAMG's one-level default behavior

An alternative to the previous section is to change SAMG's default behavior in dealing with one-level situations. More specifically, we recommend proceeding as described in Section 7.6.2. Assuming the hidden parameters *nrc* and *nrc_emergency* still have their default values ($\rightarrow$ Section 6.3.4.1), a reasonable fallback strategy for unintentional one-level cases will be to *permanently* set

clsolver_finest=1.

²⁷ Do not forget to reset *levelx* to its default value, *levelx=25*, say, if you want to go back to true multilevel mode ($\rightarrow$ Section 6.2.4)!

For more details see Section 7.6.2.

5.10.4.3 Manually checking the number of levels created

Occasionally, one might wish to explicitly check whether the exceptional one-level case has occurred. This can be done by calling

```
samg_get_levels_used(ival)
```

after the return from SAMG. The (integer) output value *ival* is as follows:

<i>ival</i>	meaning
-2	No computation done since the first approximation has already satisfied the <i>eps</i> -criterion.
-1	Fatal error has occurred in setting up the hierarchy.
0	SAMG's setup process has not been executed at all.
≥ 1	Setup process successfully passed, <i>ival</i> being the number of levels used.

Hence, if *ival*=1, no true hierarchical SAMG has been performed but just a **one-level method**. The results obtained by SAMG – in particular, its performance – should be judged under this respect.

5.10.4.4 A practical approach

Instead of calling `samg_get_levels_used(ival)` after a complete run, it may be more reasonable to call it right after SAMG's setup and **before** the solution phase. A user would then still have a chance to take a reasonable precaution before starting the solution phase, for instance, like this:

```
iswch=41                ! save hierarchy
ncyc=110                ! just setup, no cycling
call samg(.....)
call samg_get_levels_used(ival)

iswch=11
if (ival.gt.1) then      ! more than 1 level created
  ncyc=11030
  call samg(.....)      ! continue with solution phase
elseif (ival.eq.1) then ! no hierarchy has been created
  call samg_get_clsolver_finetest(iback) ! save status(just in case)
  call samg_set_clsolver_finetest(103)   ! set fallback solver
  call samg(.....)                       ! user-defined one-level method
  call samg_set_clsolver_finetest(iback) ! if required, reset status
else
  .....                                ! treat exceptional situations
endif
```

By properly choosing *clsolver_finetest*, reasonable fallback processes can be programmed. For more details, see Sections 7.6 and 0.

5.10.5 Memory allocation

The automatic memory extension via the setting of the memory extension switch *iextent* (→ Section 5.6.2) is convenient but may cause a noticeable increase of computing time. This is particularly true if OpenMP parallelism is to be exploited. Manually setting the corresponding parameters – *a_cmplx*, *g_cmplx*, *p_cmplx*, *w_avrge* (→ Section 5.7) – is preferable. Most importantly, see Section 7.11.1.

6 Hidden Parameters

Besides the **primary parameters** described before, there are various **hidden parameters**. This section describes those hidden parameters which control SAMG's basic algorithm, or which are closely related to them. Further parameters referring to special modes of operation will be introduced in Section 7.

While the meaning of many hidden parameters is obvious, others require some basic understanding of SAMG's way of operation. In some cases, this may go beyond the information given in this manual. Fortunately, for many applications, such parameters do not need any adaptation and their default settings can safely be used. Regarding more details, we refer the interested reader to [7].

All hidden parameters can be accessed (read or modified) by the **get-** and **set-routines** described in Section 6.1. While Section 6.2 describes all hidden parameters related to SAMG's **setup phase**, Section 6.3 refers to those related to SAMG's **solution phase**. Finally, in Section 6.4, we list further parameters, for instance, to re-direct and control SAMG's **print output**. In Section 6.4.4 we describe which hidden parameters are affected by setting the primary parameter *n_default* which has already been introduced in Section 5.6.3.

6.1 Accessing hidden parameters: set- and get-routines

A user might wish to tune SAMG by adapting particular hidden parameters to his needs. All hidden parameters are initialized to reasonable default values which, if required, can be read or modified via particular subroutine calls as described in the following.

For all hidden parameters, SAMG provides **set-** and **get-routines** which can be used to set new values or read current values, respectively. Generally, these routines are of the form `samg_set_` and `samg_get_`, respectively, followed by the name of the respective parameter. That is, to set a hidden parameter `pname`, say, to a new *value* or to read its current *value*, just issue

```
call samg_set_pname(value) and call samg_get_pname(value),
```

respectively. Here, *value* has to be of the correct type. This is obvious for parameters of type **integer** or **double**. For parameters of type **logical** or **character**, there are the following rules:

Parameters of type logical. If `pname` corresponds to a parameter of type **logical**, the type of *value* must be integer with values 0 or 1 representing *false.* and *true.*, respectively.

Parameters of type character. If `pname` corresponds to a parameter of type **character**, *value* must be a character string. In order to make the respective set- and get-routines easily accessible also from within C and C++, the calling sequence must include also the length of *value*. That is, the corresponding two routines take the following form:

```
call samg_set_pname(value,length) and call samg_get_pname(value,length)
```

respectively, where *length* denotes the length of the character string *value*. Note that, in case of the get-routines, the value of *length* may change upon return. If the value of *length* specified on input does not fit the requested action, *length* will be returned with a negative value.

Remark: The previous character-related routines should work in all Fortran and C environments²⁸. Should, for any reason, this not be true, there is an additional set of routines for parameters of type

²⁸ In case of C programs, also see the C header file, `samg.h`.

character which is completely analogous to the previous one, except that it operates on integer vectors containing ASCII-values (corresponding to the Fortran intrinsic functions *ichar* and *achar*) rather than strings, namely,

```
call samg_iset_purname(value,length) and call samg_iget_purname(value,length)
```

respectively, where *length* now denotes the dimension of the **integer** vector *value*, containing the ASCII values of the individual characters of a string. As before, if the specified length does not fit the requested action, *length* will be returned with a negative value.

Remark: Hidden parameters keep their value until they are explicitly redefined or reset to their initial status by issuing²⁹

```
call samg_reset_hidden().
```

Recall that issuing

```
call samg_leave(ierr)
```

internally also resets all hidden parameters (→ Section 2.5).

Remark: In former SAMG releases there was an alternative set of routines specifically for Fortran users. These routines still exist but will not be supported any more. Hence, we strongly suggest to strictly use only the above routines.



²⁹ The old routine `samg_reset_secondary` does not exist any more.

6.2 Parameters related to SAMG's setup phase

In this section we describe hidden parameters related to SAMG's setup phase, such as parameters controlling the coarsening (its type and degree of aggressiveness), the treatment of positive off-diagonal matrix entries, the basic type of interpolation, the Galerkin matrices, and many others.

6.2.1 Important threshold values used in coarsening

The below table lists four threshold values which are important to specify the coarsening process: **ecg1**, **ecg2**, **ewt1** and **ewt2** (defined by only two real numbers, **ecg** and **ewt**, → Section 2.4.3). Although the precise description of the meaning of these threshold values is beyond the scope of this manual, we at least want to give some rough description in the following.

Parameter ecg2: SAMG's coarsening is based on the concept of *strong couplings* (→ Section 2.1). Here, the definition of coupling strength is based on the entries of a "reference matrix", which – according to the outline given in Section 3 – can be the matrix itself (in case of variable-based approaches), specific block submatrices (in case of unknown-based approaches) or certain primary matrices (in case of point-based approaches). To be more specific, the coupling between any two neighboring variables is considered "strong" if the corresponding reference matrix entry is *negative* and its size exceeds a certain threshold (relative to the size of the maximum *negative* entry of the corresponding reference matrix row). The larger **ecg2**, the more restrictive the condition becomes, that is, the fewer neighbors will be considered strongly coupled.

Remark: ecg2 is the most important threshold contained in the below table. Fortunately, the specific value of **ecg2** is not very sensitive and can be frozen for most applications to its default value which is **ecg2=0.25**. In some critical cases (such as certain *anisotropic FE discretizations*) it may be better to use some larger values such as **ecg2=0.4** or even **ecg2=0.5**.



Parameter ewt2: Usually, only *negative* off-diagonal entries of the reference matrix are taken into account as candidates for strong couplings, while couplings corresponding to *positive* entries are regarded *weak*. However, if positive entries exist and exceed a certain threshold (relative to the size of the maximum *negative* entry of the corresponding reference matrix row), they cannot be ignored in defining strong couplings. The threshold value **ewt2** defines which *positive* couplings are considered "relevant" in this context, a typical standard value being **ewt2=0.2**. The smaller **ewt2**, the less restrictive the condition becomes, that is, the more positive entries are considered relevant.

The other two threshold values are less important, their purpose is to detect two exceptional situations:

Parameter ecg1: On each level of the AMG hierarchy, the purpose of the threshold value **ecg1** is to detect variables which correspond to *strongly diagonally dominant* matrix rows.³⁰ More precisely, if

$$\sum_{j \neq i} |a_{ij}| \leq \text{ecg1} \cdot |a_{ii}|$$

holds for the *i*-th variable, this variable will not be transferred to the next coarser level (ie, such variables will become "forced" F-variables in the coarsening process). Obviously, the smaller the value of **ecg1**, the stronger the requirement regarding strong diagonal dominance: While **ecg1=1.0** corresponds to weak diagonal dominance, **ecg1=0.0** corresponds to the limit case that all off-diagonal entries (corresponding to the same unknown!) vanish. A typical standard value is **ecg1=0.01**.

³⁰ Note that, in case of coupled systems, diagonal dominance is meant here w.r.t. the main diagonal blocks only, that is, cross-unknown couplings are ignored.

Parameter *ewt1*: If, on any level of the AMG hierarchy, there is a limited number of matrix rows which *strongly violate* diagonal dominance, it may be advantageous to put the corresponding variables into the next coarser level (ie, to make them C-variables). The purpose of the threshold value *ewt1* is to detect such rows. The larger *ewt1*, the stronger the condition for violation becomes. Special setting: If *ewt1*=0.0, *ewt1* will internally be replaced by 1.0d70 so that the corresponding violation check will always fail. The standard setting is *ewt1*=0.0.

Parameter	Explanation		Default
Threshold values used in defining strong connectivity			
ecg	Double. Two threshold values ³¹ for defining both strong connectivity and strong (local) diagonal dominance.		21.25d0
	ecg1	0.0 ≤ ecg1 << 1.0. Threshold to check for strong (local) diagonal dominance , the default being ecg1=0.01. ³⁰ Variables for which the check is positive, will not be transferred to coarser levels. The smaller ecg1, the stronger the requirement on diagonal dominance becomes; ecg1=1.0 corresponds to weak diagonal dominance. This check is skipped if ecg1=0.0.	
	ecg2	0.0 << ecg2 < 1.0. Threshold defining strong connectivity , a typical value being ecg2=0.25. The larger ecg2, the more restrictive the condition becomes, that is, the fewer neighbors will be considered strongly coupled (cf. also the remark further above!).	
ewt	Double. Two threshold values ³¹ for defining both strong violation of diagonal dominance, and large positive couplings which should not be ignored in coarsening.		0.2d0
	ewt1	ewt1 >> 1.0 or ewt1=0.0. Threshold to check for strong violation of diagonal dominance . Variables for which the check is positive, will be forced into the next coarser level. This check is skipped if ewt1=0.0 (the default).	
	ewt2	0.0 ≤ ewt2 < 1.0. Threshold defining “ relevant ” positive couplings , a typical value being ewt2=0.2. Such couplings are treated in a special way, provided ncgtyp = 3-5 (→ Section 6.2.2). The smaller ewt2, the more positive couplings are considered relevant. Positive couplings are strictly ignored if ewt2=0.0.	
Special default settings			
ecg_default	Default value to be used for ecg if ecg<0.0 on input.		21.25d0
ewt_default	Default value to be used for ewt if ewt<0.0 on input.		0.2d0

6.2.2 Treatment of positive off-diagonal matrix entries in coarsening

Together with the threshold values of the previous section, the integer parameter *ncg* is used to specify details of SAMG's coarsening strategy. The following table describes only the meaning of *ncg*'s first digit, the remaining ones will be covered in the subsequent subsections.

The main difference between the below options lies in the treatment of "relevant" *positive* off-diagonal matrix entries (as defined via the threshold *ewt2*, see the previous section). While Options 1-2 are meant to be used in connection with matrices *A* which have mostly *negative* off-diagonals, Options 3-5 realize a special treatment of relevant *positive* off-diagonal entries.

³¹ Regarding the definition of the two sub-parameters, see Section 2.4.3.

Parameter	Explanation		Default
Treatment of positive couplings			
ncg ³²	[1:1], ncgtyp	Integer. Specifies the treatment of positive matrix entries in the coarsening process.	
		1	Positive couplings are ignored, that is, they are regarded as “weak” (except for variables which have <i>only</i> positive couplings). This option is supposed to be used if A has mostly negative off-diagonals, positive off-diagonal elements (if any) should be “small”. Variables with only positive couplings will become C-variables.
		2	Same as 1 except that variables which have only positive couplings are treated “by absolute value”, allowing for negative interpolation weights.
		3	Same as 1 except that, for mixed-sign rows , all relevant positive entries (→ ewt2) are “eliminated” before a decision on strong connectivity is made. If, for some variable <i>i</i> , this does not lead to a clear decision, <i>i</i> will become a C-variable.
		4	Same as 1 except that, for mixed-sign rows , all relevant positive entries (→ ewt2) are “eliminated” before a decision on strong connectivity is made. If, for some variable <i>i</i> , this does not lead to a clear decision, this option behaves as Option 1.
		5	Same as 4 except that variables which have only positive couplings are treated “by absolute value”, allowing for negative interpolation weights.

Remark: If, depending on the setting of *ewt2*, there exist *no* relevant positive off-diagonal entries, Options 4 and 5 coincide with 1 and 2, respectively. That is, one might for instance use *ncgtyp*=5 as the standard setting³³. However, if the user *knows* that there are no critical positive off-diagonal entries in his application, it is **computationally cheaper** to select Option 2 instead of 5.



6.2.3 Type of coarsening

6.2.3.1 Classical or cluster coarsening

Digits 2-3 of *ncg* (sub-parameters *nred* and *nredlev*) are used to select the type and aggressiveness of the coarsening strategy:

While *nred*=0-4 select **classical coarsening** variants³⁴, *nred*=5-8 select so-called **cluster** (or aggressive) **coarsening** strategies³⁵. In particular, classical **standard coarsening** is selected by setting *nred*=0. If SAMG's memory requirement will become a problem, one might try classical **aggressive** strategies, for instance, *nred*=1 (for *anisotropic* problems) and *nred*=2 (for *isotropic* problems). Two types of **cluster coarsening** provide additional possibilities for a more aggressive coarsening.

³² The hidden parameters *ncg* and *nwt* can implicitly be set via the primary parameter *n_default* described in Section 5.6.3. Their default values correspond to the setting *n_default*=0. For more details on *n_default*, see Section 6.4.4.

³³ In contrast to *ncgtyp*=4, the setting *ncgtyp*=5 ensures that - formally - coarsening does not get stuck in case there are many rows with *only* positive couplings. Since this happens occasionally, *ncgtyp*=5 is generally the preferred choice.

³⁴ “Classical” coarsening means that coarser levels are defined by constructing subsets of finer-level variables.

³⁵ “Cluster” coarsening means that coarser levels are defined by clustering finer-level variables.

Remark: Generally, reducing SAMG's memory requirements by means of aggressive coarsening is at the expense of a slower convergence. However, what this means in terms of overall computing time, depends on the concrete situation.



Parameter	Explanation		Default	
Type and aggressiveness of coarsening				
ncg ³²	[2:2], nred	Specifies type and aggressiveness of coarsening process		
		0	Standard coarsening	0
		1-4	Aggressive coarsening. 1 is most, 4 is least aggressive; 2-3 are in between. Recommendation: 1 for anisotropic & 2 for isotropic problems.	
		5	Cluster coarsening type 1 & piecewise constant interpol.	
		6	Cluster coarsening type 1 & multi-pass interpolation.	
		7	Cluster coarsening type 2 & piecewise constant interpol.	
		8	Cluster coarsening type 2 & multi-pass interpolation.	
	[3:3], nredlev	If nred>0: Levels where aggressive or cluster coarsening are to be applied. On the remaining levels, standard coarsening will be applied.		
		0-8	From level 1 up to level nredlev+1.	0
		9	On all levels (not recommended!).	

Remark: We generally recommend to apply any of the aggressive coarsening strategies only on the finest level, that is $nredlev=0$. Further reductions of memory requirements by choosing larger values of $nredlev$ usually will not pay if compared with the additional loss in convergence speed.



The cluster strategies ($nred=5-8$) are an alternative to the classical ones ($nred=0-4$). However, they require additional parameters described in Section 6.2.3.3. Moreover, while classical strategies can be combined with relatively accurate interpolation processes as described in Section 6.2.5, cluster strategies are combined with low-accurate piecewise constant or multi-pass interpolation (selected directly through the setting of $nred$). As a consequence, the setup cost of cluster-based SAMG is fairly low at the expense of (usually very) slow convergence. Regarding convergence improvements of cluster coarsening strategies, see also "Smoothed Aggregation" in Section 7.12.

6.2.3.2 Special aspects regarding classical coarsening

The following auxiliary parameters are very special and should not be changed by the average user.

Parameter	Explanation		Default	
Auxiliary parameters for exceptional situations				
ncg	[4:4], nxf_clean	By exceptional F-variables (“XF-variables”) we denote such variables which are left over at the end of a coarsening process. The parameter <i>nxf_clean</i> defines how to treat such variables in case of <i>standard</i> or <i>cluster coarsening</i> .		
		0	Default procedure (currently equivalent to 1)	0
		1	Ensure that all XF-variables have a strong coupling to (at least) one regular F-variable.	
		2	Re-define all XF-variables to become C-variables.	
	[5:5], npcol	Enforce particular conditions by a posteriori adding extra C-variables. <i>npcol</i> >0 should never be used with aggressive or cluster coarsening!		
		0	No action.	0
		1	Enforce weak F-to-F diagonal dominance (factor 1.0).	
2		Enforce strong F-to-F diagonal dominance (factor 0.75).		

6.2.3.3 Special aspects regarding cluster coarsening

Cluster coarsening **type 1** ($nred=5-6$) essentially corresponds to a straightforward modification of the classical coarsening algorithm. As such, there are no extra parameters to influence the shape and/or size of clusters. This is different for the cluster coarsening **type 2** ($nred=7-8$) which allows to control both shape and size of the clusters to some significant extent.

Parameter	Explanation	Default
Special parameters for cluster coarsening type 2		
max_cl_size	Integer. Defines the max. cluster size. All below approaches ($\rightarrow$ <i>cl_order</i> , except 4 & 8) tend to build clusters of size close to <i>max_cl_size</i> .	0
	0 Default value, currently equivalent to 4	
	>0 Maximum cluster size	
min_cl_size	Integer. Defines the min. cluster size. All below approaches ($\rightarrow$ <i>cl_order</i> , except 3, 4 & 8) tend to build clusters of size bigger than <i>min_cl_size</i> . If necessary, neighboring clusters will be merged. Attention: Satisfying the min. cluster size has a higher priority than satisfying the max. size. That is, if necessary, the max. size will be violated.	0
	0 Default value, currently equivalent to 1	
	>0 Minimum cluster size	
cl_cpl	Integer. Selects the definition of “strength” of couplings	3
	1 Standard, symmetrized by logical conjunction (.and.)	
	2 Standard, symmetrized by logical disjunction (.or.)	
	3 Defined by the size of $a_{ij}^2 / a_{ii} a_{jj}$. If <i>isym</i> =2, <i>cl_cpl</i> =3 is automatically replaced by <i>cl_cpl</i> =4!	
	4 Defined by the size of $a_{ij} a_{ji} / a_{ii} a_{jj}$.	
cl_order	Integer. Details on construction of clusters.	2
	0 Clustering tends to start in the <i>interior</i> of a mesh	
	1 Clustering tends to start at the <i>boundary</i> of a mesh	
	2 Clustering tends to spread “continuously”	
	3 Pairwise aggregation aiming at clusters of size power of 2. Number of “pairing steps” is bounded by <i>max_cl_size</i> . ³⁶ <i>min_cl_size</i> is ignored for this clustering.	
	4 Classical clustering (Marian Brezina). ³⁷	
	5 Diameter minimization, automatically bounded diameter	
	6 Clustering based on cellular automaton ³⁸	
	7 Diameter minimization	
	8 Classical clustering, aggressive variant. ³⁷	
max_nb_list_size	Integer. Only used for <i>cl_order</i> =0-2: Max. number of neighbors to be taken into account in building clusters. The setting “0” results in a “dynamic” value, <i>max_nb_list_size</i> =64* <i>max_cl_size</i> .	0

Remarks: (1) Regarding the treatment of **positive couplings** ($\rightarrow$ Section 6.2.2), for technical reasons, only the settings *ncgtyp*=1 and *ncgtyp*=3 are respected for cluster coarsening. Other settings are automatically redefined as follows: *ncgtyp*=2 $\rightarrow$ *ncgtyp*=1 and *ncgtyp*>2 $\rightarrow$ *ncgtyp*=3. Furthermore, if *ncgtyp*=3, *cl_cpl* is automatically re-defined to be *cl_cpl*=4. **(2)** Regarding the **threshold values** ($\rightarrow$ Section 6.2.1), *ecg1* and *ewt1* are ignored in the context of cluster coarsening.



³⁶ More precisely, the average cluster size will be the largest power of 2 which is still smaller than *max_cl_size*.

³⁷ Both, *max_cl_size* and *min_cl_size* are ignored for this clustering.

³⁸ This clustering is not deterministic. It usually causes a particular fast setup but fairly poor convergence rates.

6.2.4 Termination criteria for the coarsening process

There are several criteria used by SAMG to decide when to terminate the coarsening process. The two major ones are given by the parameters *levelx* and *nptmn*. In addition, there are a few “emergency criteria”. For instance, coarsening will be terminated if it simply gets too slow, or if a coarser-level system of equations becomes strongly diagonally dominant (and, hence, allows for a very efficient numerical solution).

For clarity, let us **summarize** the termination criteria resulting from the parameters detailed in the below table: Coarsening will be stopped on some level $m \geq 1$ if one of the following criteria is met,

1. The number of variables on level m is $\leq nptmn$,
2. $m = \min(levelx, max_level)$,
3. On level m , the matrix density exceeds *densx* **and** the number of variables is $\leq nptmax$,
4. According to the parameter *eps_dd*, the matrix on level m is strongly diagonally dominant,
5. According to the setting of *term_coarsening*, coarsening has become unacceptably slow.

Parameter	Explanation	Default
Standard termination criteria		
levelx	Integer. Safety limit: maximum number of levels to be created. Special debug option: Giving <i>levelx</i> a negative sign causes SAMG to display its convergence history also on coarser levels.	25
nptmn	Integer. Minimum number of variables on the coarsest level, ≥ 10 . Coarsening will terminate on the first level for which the number of variables is $\leq nptmn$. Remark: <i>nptmn</i> should not be too small!	100
Emergency criteria		
max_level	Integer. Upper limit of number of levels, independent of <i>levelx</i> .	25
densx	Double. Maximum matrix <i>density</i> on coarsest level (1.0 = 100%).	0.2d0
nptmax	Integer. see Section 6.3.4	1500
eps_dd	Double. Threshold for defining <i>strong diagonal dominance</i> . <i>eps_dd</i> should be < 1.0 , where <i>eps_dd</i> = 1.0 corresponds to <i>weak</i> diagonal dominance.	0.9d0
slow_coarsening	Double. Defines <i>slow coarsening</i> . If the ratio between the number of variables of the current level and its predecessor becomes $\geq slow_coarsening$, a warning will be issued but coarsening continues. The concrete value should be < 1.0 , where = 1.0 would correspond to “no coarsening at all”.	0.75d0
term_coarsening	Double. Defines <i>very slow coarsening</i> causing the coarsening to be terminated. The concrete value should be $> slow_coarsening$ and < 1.0 .	0.9d0

Under all normal circumstances, coarsening will stop on some level $m > 1$ with less than *nptmax* (usually even less than *nptmn*) variables. However, the following **special case** may occur:

Important: If any of the termination criteria is already satisfied for the *finest level*³⁹, no coarse levels will be created by SAMG. Then, by default, the only existing level is regarded as the *finest* (not the *coarsest*!) one. This means that SAMG’s solution process degenerates to the iterative application of the smoother, used as preconditioner for the currently selected accelerator. This default behavior can be changed by means of the hidden parameter *clsolver_finest* ($\rightarrow$ Section 7.6 and, in particular, 7.6.2).



³⁹ For example, if the dimension of $Au=f$ itself is already $\leq nptmn$, or if *levelx*=1.

6.2.5 Interpolation

Generally, the more effort is put into the construction of interpolation, the faster the resulting SAMG convergence can be. However, in most applications, it will not pay to invest too much into the construction of interpolation, since this will also substantially increase the computational work.

The first digit, *nwtint*, of the parameter *nwt* selects the interpolation to be used. The settings ***nwtint=1-3*** belong to the same type of interpolation. The difference lies in the effort which is put into its construction: the larger *nwtint*, the more effort is invested. That is, *nwtint=1* corresponds to the simplest and *nwtint=3* to the most expensive interpolation, the standard choice being *nwtint=2*. The setting ***nwtint=4*** corresponds to the so-called *multi-pass interpolation* which is supposed to be used when there are not enough C-variables to employ the other types of interpolation. This is typically true if aggressive coarsening has been selected. Hence, independent of *nwtint*, SAMG will automatically employ multi-pass interpolation on all levels where aggressive coarsening is applied.

The remaining digits are only relevant if the previous interpolation is to be improved **a posteriori** by performing **Jacobi F-relaxation** [7]. By experience, depending on the application, this can enhance convergence significantly. However, since the computational work typically increases dramatically, we generally recommend that this feature is used only by experienced users.

Parameter	Explanation		Default	
Basic type of interpolation				
nwt ³²	Integer. Selects the basic type of interpolation operator.		2	
	[1:1], nwtint	nwtint is relevant on levels with standard coarsening. On levels with aggressive coarsening, SAMG will automatically select <i>multi-pass interpolation</i> . On levels with cluster coarsening, the interpolation is specified via the parameter <i>nred</i> (→Section 6.2.3).		
		1		Direct interpolation.
		2		Standard interpolation.
		3		Extended interpolation: is being re-designed.
		4		Multi-pass interpolation.
	Jacobi F-relaxation to improve interpolation ⁴⁰			
	[2:2], itint	Number of Jacobi F-relaxation steps to be applied, ≥0. The following digits are relevant only if <i>itint</i> >0:		
		Number of levels where F-relaxation is to be applied.		
	[3:3], intlev	0-8		From level 1 to level <i>intlev</i> +1.
		9		On all levels (not recommended!)
	[4:4], iall	Specifies whether F-relaxation shall be applied in its full form or whether weak couplings shall be ignored.		
0		Ignore weak couplings.		
1		Full Jacobi.		
Further parameters				
eps_diag	Double. Defines the factor by how much the size of a diagonal entry is allowed to drop in assembling interpolation before the point to interpolate to is decided to become a C-point.		1.0d-5	
nint_rowsum1	Integer. This parameter is effective only if it is set to 1 in which case all rowsums of interpolation are re-scaled to equal 1.0.		0	
ipass_max_set	Integer. Max. number of passes in <i>multi-pass interpolation</i> before a new attempt is made. This parameter is effective only if it is assigned a <i>positive</i> value. Otherwise, a value is used as specified internally to SAMG.		0	
numtry_max_set	Integer. Max. number of new attempts in <i>multi-pass interpolation</i> . This parameter is effective only if it is assigned a <i>positive</i> value. Otherwise, a value is used as specified internally to SAMG.		0	

⁴⁰ Not available in connection with block interpolation.

6.2.6 Special parameters for point-based approaches

The following hidden parameters are only relevant for coarsening and interpolation in **point-based** SAMG approaches:

Parameter	Explanation		Default
Special parameters for point-based approaches			
allow_elim	Integer. Relevant only in case of block interpolation (ie, <i>napproach</i> =5). If <i>allow_elim</i> =1, modification of stencils by inserting neighboring stencils is allowed, otherwise not.		1
prim_norm	Integer. Relevant only in case of block coarsening (ie, <i>internal</i> =3-4). Defines the type of block norm to be used.		0
	0	Maximum norm: $\ Q\ = \max_{i,j} \{ q_{ij} \}$	
	1	Row sum norm: $\ Q\ = \max_i \{ \sum_j q_{ij} \}$	
	2	Schur norm: $\ Q\ = \sqrt{\sum_{i,j} q_{ij}^2}$	
prim_print	Integer. Relevant for internally defined primary matrices (ie, <i>internal</i> >0). If <i>prim_print</i> =1, the finest-level primary matrix is written to disk (formatted files <i>prim1.frm</i> and <i>prim1.amg</i>).		0
multipass_allcoup	Integer. Relevant only for multipass interpolation. Defines which couplings to take into account.		0
	0	Include only strong couplings of a variable/point.	
	1	Include all couplings of a variable/point.	
Complexity parameters for internally defined primary matrices (ie, if <i>internal</i> >0)			
b_cmplx	Double. Initial complexity guess for work arrays <i>b</i> and <i>jb</i> (analogous to <i>a</i> and <i>ja</i>). If <i>b_cmplx</i> = 0.0, default values will be selected, see below.		0.0d0
ib_cmplx	Double. Initial complexity guess for work array <i>ib</i> (analogous to <i>ia</i>). If <i>ib_cmplx</i> = 0.0, default values will be selected, see below.		0.0d0
b_cmplx_default	Double. Default for <i>b_cmplx</i> in case of <i>standard</i> coarsening.		2.0d0
b_cmplx_agg_default	Double. Default for <i>b_cmplx</i> in case of <i>aggress.</i> coarsening.		1.2d0
ib_cmplx_default	Double. Default for <i>ib_cmplx</i> in case of <i>standard</i> coarsening.		2.0d0
ib_cmplx_agg_default	Double. Default for <i>ib_cmplx</i> in case of <i>aggress.</i> coarsening.		1.2d0

6.2.7 The Galerkin coarse-level matrices

In computing the Galerkin coarse-level matrices, accidental zero entries may be skipped by default. Under certain conditions, this is not desired, for instance, if *structural symmetry* or *structural identity* of the finest level matrix is requested to be preserved also for all of the coarse-level Galerkin matrices. The following parameter can be used to change the default behavior:

Parameter	Explanation	Default
Treatment of zeroes in computing Galerkin matrices		
glk_mult_zeros	Logical. By default, SAMG may decide to skip (ie, not store) zeros on coarser levels. Set <i>glk_mult_zeros</i> = <i>true</i> . to force SAMG to treat zeros in the Galerkin matrices like any other number.	.false.

Special case: If *structural symmetry* or *structural identity* of the finest level matrix is requested to be preserved also for all of the coarse-level Galerkin matrices, set *glk_mult_zeros*=*true*. Moreover, truncation of the Galerkin matrices must not be applied in such cases (→ Section 6.2.9)!



In case the coarser-level Galerkin matrices contain non-positive diagonal entries, some pre-caution must be taken. Depending on the settings of the below parameters, SAMG attempts to get rid of non-positive diagonal entries by locally modifying interpolation and re-computing the Galerkin operator. If, within the range permitted by the below parameters, these attempts are not successful, SAMG gives a warning (code -153 or -154) and continues anyway, ignoring potential negative entries.

Parameter	Explanation	Default
Parameters regarding negative diagonal Galerkin entries		
neg_diag	Integer. Defines the maximum number of non-positive diagonal entries allowed in computing the Galerkin operators before SAMG gives up its attempts to modify interpolation and continues without further checks. If <i>neg_diag</i> <0, checking of the diagonal is completely de-activated. Warning: In the latter case you should know what you are doing!	20
neg_diag_brute	Integer. Should have a value with $0 \leq \text{neg_diag_brute} \leq \text{neg_diag}$. If interpolation needs to be modified (see previous parameter), this is normally done via a “brute-force method”. However, if the number of non-positive diagonal entries exceeds <i>neg_diag_brute</i> , a “more sophisticated” method is selected.	20
maxop_restart	Integer. Defines the maximum number of re-starts allowed in computing a Galerkin operator before SAMG gives up. Remark: If the maximum number of re-starts has been reached, another more sophisticated attempt for modifying interpolation is performed. If this fails also, SAMG will continue anyway.	2
dump_correctw	Integer. If <i>dump_correctw</i> $\neq$ 0, info regarding non-positive diagonal entries will be written to a file with name <i>filnam</i> and suffix “.crw” .	0

Remark: For positive definite matrices (ie, $(Au, u) > 0$ for all u), whether symmetric or not, diagonal entries in the Galerkin operators cannot become negative, at least not theoretically. Hence, whenever there exists one or more negative diagonal entries on any of the coarser levels, the given problem **cannot be positive definite!**



6.2.8 Repeated calls to SAMG, reusing previous setup data

If *iswit*=2-3, SAMG assumes that matrices change between calls (→ Section 5.6.1), generally causing some restructuring effort on the coarser levels. Often, however, a user will know that matrices are "**structurally identical**" between SAMG calls, that is, their *size* and *structure* remain unchanged. If this is made known to SAMG, some redundant work can be saved:

Parameter	Explanation	Default
Structurally identical matrices		
struct_ident	Logical. Set <i>struct_ident</i> = <i>.true.</i> before calling SAMG with <i>iswit</i> =4 to indicate that this and all further matrices passed with <i>iswit</i> =2 or <i>iswit</i> =3 will be structurally identical. Remark: Do not change <i>struct_ident</i> while calling SAMG with <i>iswit</i> <4!	.false.

Remark: If a given fine-level matrix is structurally identical to another one, this is not necessarily true for the corresponding coarser-level (Galerkin) matrices: Due to accidental zero matrix entries (which may be skipped by SAMG!), some coarser level matrices may not be structurally identical any more. To ensure structural identity also on coarser levels, zeros in computing the Galerkin operators have to be treated like any other number.⁴¹ However, this is ensured automatically and a user does not have to care: *As long as struct_ident is set to .true., glk_mult_zeros is automatically and temporarily also set to .true.* (→ Section 6.2.7).

⁴¹ In addition, clearly, truncation of Galerkin matrices must not be applied (→ *eps_optrunc* in Section 6.2.9!

6.2.9 Truncation features

Truncation is a very important means to keep the complexity of the coarser-level problems (the fill-in) low. Truncation is controlled by the parameters described in the below table.

Attention: In principle both the **interpolation** and the **Galerkin matrices** can be truncated. However, while the truncation of interpolation is important and standard, that of the Galerkin operator has to be done very carefully (→ [7]).



Parameter	Explanation		Default	
Truncation of interpolation and Galerkin				
etr	Double. Two truncation parameters ⁴² for the basis interpolation (ie, the one selected by the setting of <i>nwtint</i>) as well as the one obtained by subsequent Jacobi F-relaxation .		12.2d0	
	etr1	Threshold for truncation of the basis interpolation, $0.0 \leq etr1 < 1.0$. Truncation is de-activated if <i>etr1</i> =0.0 (not recommended!).		
	etr2	Threshold for truncation of the Jacobi interpolation, $0.0 \leq etr2 < 1.0$. Truncation is de-activated if <i>etr2</i> =0.0 (not recommended!).		
ntr	Integer. Type of truncation to be used, ≥ 0 . If <i>ntr</i> =0, no truncation is performed at all (not recommended!).		1	
	[1:1], ntrunc	Type of truncation to be used for the interpolation .		
		1		Truncation by absolute value.
		2		Truncation separately for positive and negative weights.
	[2:3], eps_optrunc	Truncation of Galerkin operators		
		=0		No truncation (recommended).
>0		All Galerkin entries are truncated which are smaller than $10^{*(-ntr[2:3])}$ times the respective diagonal entry.		
Default values				
etr_default	Default value to be used for <i>etr</i> if <i>etr</i> < 0.0 on input.		12.2d0	

⁴² Regarding the definition of the two sub-parameters, see Section 2.4.3.

6.3 Parameters related to SAMG's solution phase

In this section we describe hidden parameters related to SAMG's solution phase, in particular, parameters influencing the cycle, the accelerator, the smoothing process as well as the solver on the coarsest level.

6.3.1 The smoothing process

While the *kind* of smoother (eg, relaxation or ILU-type, variable- or block-wise) is selected by the primary sub-parameter *nxtyp* (→ Section 5.4.2), further details are specified by the parameters of the following table. In particular, the two sub-parameters *nrdx* and *nrux* specify the number of pre- and post-smoothing steps, respectively. Moreover, in case of smoothing by relaxation, *nrd* and *nru* allow to define full smoothing steps as a series of partial steps.

Parameter	Explanation		Default	
Number of smoothing steps and, in case of relaxation, partial smoothing steps				
nrd	Integer. Selects details for pre-smoothing . If <i>nrd</i> =0, no pre-smoothing is performed. Otherwise:		131	
	[1:1], nrdx	Number of pre-smoothing steps to be performed by the smoother as selected by the primary parameter <i>nxtyp</i> .		
	[2:], nrdtyp	Only relevant if smoothing is by relaxation (<i>nxtyp</i> =0/5, → Section 5.4.2): each digit of <i>nrdtyp</i> selects a (partial) relaxation step. The successive performance of all partial steps defines a complete smoothing step. ⁴³ The individual digits can have any of the following values:		
		1		one Gauss-Seidel F-relaxation sweep
		2		one full Gauss-Seidel sweep
		3		one Gauss-Seidel C-relaxation sweep
		4		same as 1 but with reversed order
		5		same as 2 but with reversed order
		6		same as 3 but with reversed order
7	one Jacobi relaxation sweep (→ <i>omega_jacobi</i>)			
nru	Integer. Selects details for post-smoothing .		131	
	nrux	completely analogous to <i>nrdx</i>		
	nrutyp	completely analogous to <i>nrdtyp</i>		
ndyn_smo	Integer. Unless <i>ndyn_smo</i> =0, this parameter increases the number of both pre- & post-smoothing steps (→ <i>nrdx</i> , <i>nrux</i>) on coarser levels.		0	
	0	parameter ignored		
	10	increases # of smoothing steps by 1 starting at level k=2		
	11	increases # of smoothing steps by k-1 starting at level k=2		
	20	increases # of smoothing steps by 1 starting at level k=3		
	21	increases # of smoothing steps by k-2 starting at level k=3		
omega_jacobi	Double. Underrelaxation parameter for Jacobi smoothing. For coupled systems (<i>nsys</i> >1), one may also prescribe different parameters for different unknowns, see further below.		0.5d0	

⁴³ The default setting "131" means **C/F-relaxation**, that is, first the C-variables are relaxed, followed by a sweep over the F-variables. If no C/F-splitting is available (on the very coarsest level, or if only one level exists), C/F-relaxation is replaced by a suitable lexicographic variant.

Reverse order of relaxation: If relaxation has been selected for smoothing ($nxtyp=0/5$, → Section 5.4.2), giving nrd or nru a negative sign, completely reverses the order of relaxation. In the following, we give some examples for nrd (nru is completely analogous):

	nrd	Explanation
Examples for pre-smoothing by relaxation:	131	one C/F relaxation step (analogous to red/black)
	22	two full Gauss-Seidel relaxation steps
	-131	one C/F smoothing step in reversed order (equivalent to 146)
	231	two C/F relaxation steps (equivalent to 13131)
	13172	one C/F relaxation step, followed by one Jacobi step, followed by one full Gauss-Seidel step

Remark: If the matrix A is **symmetric** (ie, if $isym=1$) and SAMG is used as preconditioner for CG, Gauss-Seidel relaxation will automatically be performed in a symmetric way, independent of the settings of nrd and nru . More precisely, if $nrd \neq 0$, nru will automatically be replaced by $nru=-nrd$. Otherwise, nrd will be replaced by $nrd=-nru$.⁴⁴



6.3.2 Further parameters relevant for smoothing

Parameter	Explanation		Default
Smoothing by relaxation			
ibgs_pivot	Integer. Pivoting strategy for block relaxation smoothers:		0
	0	full pivoting is deactivated	
	1	full pivoting is activated	
nxtyp_coarse	Integer. Unless <i>nxtyp_coarse</i> =-1, this parameter selects relaxation for smoothing on coarser levels independent from what has been chosen on the finest level (→ <i>nxtyp</i> , Section 5.4.2).		-1
	-1	parameter ignored	
	0	variable-wise relaxation (details according to <i>nrd</i> and <i>nru</i>)	
	5	block-wise relaxation (details according to <i>nrd</i> and <i>nru</i>)	
Smoothing by ILU-type smoothers			
ilu_speed	Integer. Computational speed of ILU or MILU can be increased at the expense of a (slight) increase of memory:		1
	0	standard version	
	1	faster version, slightly more memory required	
milu	Integer. Parameter used to distinguish between standard and modified ILU versions:		0
	0	standard ILU / ILUTP	
	1	modified MILU / MILUTP	
delta_milu	Double. Undercorrection parameter in case of MILU / MILUTP, $0.0 \leq \textit{delta_milu} \leq 1.0$		1.0d0
	0.0d0	corresponds to <i>no correction</i> (ie, standard ILU / ILUTP)	
	1.0d0	corresponds to <i>full correction</i>	
lfil_smo	Integer. Fill-in parameter <i>lfil</i> of ILUT/ILUTP if used for <i>smoothing</i> .		9
droptol_smo	Double. Tolerance <i>droptol</i> of ILUT/ILUTP if used for <i>smoothing</i> .		0.5d-2

Jacobi smoothing for coupled systems

It is well known that Jacobi smoothing requires *underrelaxation*. The parameter **omega_jacobi** introduced in the above table is used to define a concrete value of the underrelaxation parameter,

⁴⁴ This automatism can be deactivated by setting $force_accel=.true.$ (→ Section 6.3.3).

the default being 0.5. If nothing else is done, the same parameter is used throughout. However, for coupled systems (ie, $nsys > 1$), a user might wish to use different underrelaxation parameters for different unknowns. This can easily be achieved as described in the following.

First, before calling SAMG, the following subroutine needs to be called with $nsys$ being the number of unknowns in the given system,

```
call samg_omega_jacobi_alloc(nsys,ierr).
```

Unless the integer error flag $ierr$ is nonzero upon return from this routine, this will allocate memory (double vector of length $nsys$) needed to store individual underrelaxation parameters separately for each unknown of the given system. All individual underrelaxation parameters are automatically initialized by the current (!) value of $omega_jacobi$. If requested, a new underrelaxation parameter, new_omega , can be assigned to the n -th unknown ($1 \leq n \leq nsys$) by issuing:

```
call samg_omega_jacobi_set(n,new_omega,ierr).
```

Remark: Since the above extra memory is explicitly allocated and defined *outside* SAMG, the corresponding data will not be reset by calling the subroutine `samg_reset_hidden`. Moreover, it is also the user's responsibility to deallocate this memory. That is, it is not sufficient to call one of the subroutines provided to clear SAMG's memory (such as `samg_refresh(ierr)`, → Section 2.5). Instead, to explicitly clear the above memory, the following subroutine needs to be called:

```
call samg_omega_jacobi_dealloc(ierr).
```



6.3.3 Acceleration and modified multigrid cycles (*-cycles)

Two parameters, $nstar_typ$ and $inner_accel$, modify single iteration steps at the expense of additional numerical work. Hence, whether or not these modifications finally pay, depends on the situation. For instance, $inner_accel$ defines the number of AMG cycles ("inner iteration") per accelerator call ("outer iteration": CG, GMRes, ...). Of course, doubling the value of $inner_accel$ essentially doubles the cost per outer iteration step. Hence this pays only, if the convergence becomes faster by more than a factor of two. Cycle-modifications as controlled by the setting of $nstar_typ$ are supposed to be used primarily in connection with aggregation-based coarsening strategies where they attempt to minimize the effect of wrong scalings of the coarse-level matrices

Parameter	Explanation		Default
Modified multigrid cycles (*-cycles)			
nstar_typ	Integer. By giving <i>ncyc</i> a negative sign, modified cycles (*-cycles) are activated ⁴⁵ (→ <i>ncyc</i> , Sections 5.5 and 5.5.2).		0
	0	selects a default, currently equal to setting <i>nstar_typ</i> =2.	
	1	old AMG1R5 *-cycle	
	2	standard *-cycle, only on coarser levels	
	3	standard *-cycle, on all levels	
Parameters influencing the accelerators			
inner_accel	Integer. Number of cycles per accelerator call, 0< <i>inner_accel</i> <10.		1
force_accel	Logical. If set to .true., the accelerator used is <i>forced</i> to be the one selected by <i>ncgrad</i> (ie, it is never changed inside SAMG to anything else, even if it should not be used for mathematical reasons, → Section 5.5.3). Moreover, the user-specified setting of <i>nrd</i> and <i>nru</i> will not be adjusted (→ corresponding Remark in Section 6.3.1). When setting <i>force_accel</i> =.true., the user should know what he is doing!		.false.

⁴⁵ If *-cycles are activated, the sign of $levelx$ will be ignored.

Remark: The effectiveness of *-cycles to some extent depends on whether enough smoothing is performed, it may not be enough to merely perform one pre- and post smoothing step. This is particularly critical in case $nstar_typ=3$ and *non-symmetric* matrices.



6.3.4 The coarsest-level (CL) solver

Various solvers for the coarsest-level equations are included in SAMG. The standard and safest way to solve the coarsest-level equations is by a **direct solver**. However, one has to take care that the computational cost for solving the coarsest-level systems is still reasonable, that is, it should only be a small fraction of the cost of a complete multigrid cycle. Also memory requirement should be acceptable. There is a hidden parameter, $nptmax$, which equals the maximum number of variables still suitable for the currently selected direct solver. Should, for any reason, the number of coarsest-level variables be larger than $nptmax$, the direct solver selected by nrc will switch to an “emergency” one.

Generally, it is not necessary to solve the coarsest-level equations exactly, an accuracy of 1-2 digits is usually sufficient. Hence, one might be tempted to use an **iterative solver** rather than a direct one. This would have the advantage of a lower memory requirement which, in turn, would allow for much finer coarsest levels. However, in complex applications, the convergence behavior of iterative solvers often turns out to be fairly unpredictable, causing iterative solvers to become less favorable.

6.3.4.1 Selecting the coarsest-level solver

Parameter	Explanation	Default
Solver for the coarsest-level equations		
nrc	Integer. Selects coarsest-level solver. If $nrc=0$, a default solver is used, namely, $nrc=11$ or $nrc=7$ depending on whether or not Intel's MKL is has been linked. 1 Iterative application of currently selected smoother 2 ILU(0) preconditioned CG 3 ILUT preconditioned BiCGstab 4 Diagonally preconditioned CG 5 Diagonally preconditioned BiCGstab 6 Dense direct solver (standard) 7 Sparse direct solver (standard) 8 Least squares solver (LSQ; robust but very expensive!) 9 Dummy interface routine ($\rightarrow$ Section 8.3.1) 10 Dense direct solver (highly tuned, $\rightarrow cldense_ctrl$ below) 11 Sparse direct solver (Intel's pardiso) if MKL has been linked 99 Another instance of SAMG	0
nptmax	Integer. Maximum number of variables reasonable for the <i>direct</i> solver selected by nrc (if any). That is, solving $nptmax$ equations by this direct solver should still be reasonable compared to the cost of a complete multigrid cycle. Increasing $nptmax$ might improve SAMG convergence in difficult applications. However, this requires a very efficient solver on the coarsest level such as Intel's pardiso.	1500
nrc_emergency	Integer. <i>Only relevant for direct coarsest-level solvers, that is, if $nrc=6-8, 10$ or 11.</i> In these cases, $nrc_emergency$ is an alternative coarsest-level solver used in the "emergency situation" that the number of variables on the coarsest level turns out to be larger than $nptmax$. Naturally, $nrc_emergency$ should correspond to a robust <i>iterative</i> solver.	3

6.3.4.2 Important coarsest-level parameters

Parameter	Explanation	Default
Termination of iterative solvers (if nrc=1-5)		
conv_stop	Double. Residual reduction required for <i>iterative</i> solvers	0.1d-1
itmax_conv	Integer. Maximum number of iterations for <i>iterative</i> solvers	200
ILUT parameters (if nrc=3)		
lfil_cl	Integer. ILUT's fill-in parameter, known as <i>lfil</i> . In the following, <i>nnu_cl</i> and <i>nnu_cl</i> denote the number of variables and nonzero entries, respectively, on the coarsest level and <i>n</i> denotes any positive integer. Then <i>lfil</i> is defined as follows:	-10
	<i>n</i> <i>lfil</i> = <i>n</i>	
	<i>-n</i> <i>lfil</i> is not fixed but rather depends on the average number of non-zero entries on the coarsest level: $lfil = \max \left\{ 1, \min \left\{ nnu_cl, \text{int} \left(\frac{n}{10.0} \cdot \frac{nnu_cl}{nnu_cl} \right) \right\} \right\}$ That is, <i>lfil</i> is selected to be <i>n</i> /10 times the average number of non-zero entries on the coarsest level.	
	0 <i>lfil</i> = <i>nnu_cl</i>	
droptol_cl	Double. Droptol parameter if ILUT is selected on coarsest level.	0.5d-2
Options for dense direct solver (if nrc=10)		
cldense_ctrl	Integer. Options for dense direct solver	
	Details on usage of direct solver	
	[1:1]	1 LU factorization with partial pivoting
		2 Calculation of full inverse ⁴⁶ . This is attractive, in particular, when many cycles are needed, as the solving part then corresponds to matrix-vector multiplies only, promising best performance.
	Precision of solution process	
	[2:2]	0 Double precision
		1 Single precision. While this option saves memory and run time, it should be used only in "numerically stable" situations (eg, simple scalar applications)

6.3.4.3 Less important coarsest-level parameters

Parameter	Explanation	Default
Further parameters relevant for coarsest-level solver		
full_pivoting	Logical. In case <i>nrc</i> =6: full or partial pivoting	.false.
stability	Double. In case of <i>nrc</i> =7: $0.0 < stability < 1.0$. The smaller <i>stability</i> , the lower the fill-in, but also the stability. The larger <i>stability</i> , the higher the fill-in, but also the stability.	0.25d0
iter_check	Integer. In case <i>nrc</i> =1: Number of iterations to be performed before residuals are checked.	3
rcondx	Double. In case <i>nrc</i> =8: Control parameter for LSQ (least squares) processes needed to correctly detect the rank of a matrix ⁴⁷ .	1.0d-6

⁴⁶ LAPACK function class GETRI⁴⁷ This parameter is relevant for all LSQ solution processes, not just the coarsest-level solution. In particular, it is used in case of the least-squares optimization of interpolation.

6.3.4.4 Some remarks and hints

For an unexperienced user the optimal selection of a coarsest level (CL) solver is not straightforward. However, in nearly all situations, the default selection ($nrc=0$) should be a safe choice. In particular situations, however, fine-tuning may request some other choice. Some more specific remarks:

- **$nrc=2-5$:** If - according to the setting of the parameter *isym* ($\rightarrow$ Section 5.1) - the given problem is not symmetric, the CG solvers ($nrc=2$ and $=4$) will automatically be replaced by the corresponding ones using BiCGstab ($nrc=3$ and $=5$).
- **$nrc=8$:** The **least squares solver** is mainly for test purposes, it is very expensive! On the other hand, it provides the most robust way to solve the coarsest-level equations, in particular, in case of singular or nearly singular matrices.
- **$nrc=10$:** This **dense** direct solver uses BLAS/LAPACK interfaces for factorization and solution of dense systems⁴⁸. Here, our default is to use Intel's MKL library with its highly tuned, OpenMP parallel kernels. However, SAMG has been kept open to also use other (user-defined) BLAS/LAPACK solutions depending on binary or source-based delivery (cf. the "Getting Started" documentation).
- **$nrc=10$:** If - according to the setting of the parameter *isym* ($\rightarrow$ Section 5.1) - the given problem is symmetric, this is exploited by the **dense** direct solver, saving about a factor of 2 in memory and, eventually, in run time.
- **$nrc=11$:** The sparse direct solver **pardiso** is only available if Intel's MKL has been linked, the multithreaded version of which contains an OpenMP parallel version of pardiso.

⁴⁸ GETRF class subroutines for factorization, and GETRS class subroutines for solution

6.4 Further parameters

6.4.1 Re-directing and limiting I/O

By default, all print output is sent to the console. This can be changed by re-defining the hidden parameters *logio* and *logfile* accordingly. In addition to the primary parameters *iout* and *idump* (→ Section 5.8), special output is controlled by the hidden parameter *mode_mess*.

Parameter	Explanation	Default
Re-directing and limiting I/O		
logio	Integer. I/O unit for all print output.	6
logfile	Character. Name of logfile where to print (empty string =console)	" "
mode_mess	Integer. By setting the primary parameters <i>iout</i> and <i>idump</i> to something negative, all output regarding SAMG's performance (setup, cycling history, etc) will be suppressed (→ Section 5.8). However, general messages , warnings and error messages will still be printed unless the hidden parameter <i>mode_mess</i> is adjusted:	0
	1 All kinds of messages are printed.	
	0 Control-messages ⁴⁹ are skipped.	
	-1 In addition: General messages are skipped.	
	-2 In addition: Warning messages are skipped.	
	-3 In addition: Error messages are skipped.	

Remark: All output of SAMG is routed through a single routine, namely, *samg_message*. This routine is located in the file *amguser.f90* and – if source is available - can be adjusted by the user to his needs. For instance, all output of SAMG may be transferred to a GUI (→ Section 8.3.5).



Warning: In order to suppress all SAMG output, including warning and error messages, set *iout* = -1, *idump* = -1 and *mode_mess* = -3. Clearly, in this case, a user should know what he is doing!



⁴⁹ These are messages created by the automatic control process described in Section 7.5.

6.4.2 Various default values

The following table lists default values for some of the primary parameters.

Parameter	Explanation	Default
Default values relevant for the solution approach		
nsolve_default	Integer. Default if <i>nsolve</i> =0 on input	2
ncyc_default	Integer. Default if <i>ncyc</i> =0 on input	11030
ncgrad_default	Integer. Default if <i>ncgrad</i> =0 on input	1
nkdim_default	Integer. Default if <i>nkdim</i> =0 on input	30
Default values in case of standard coarsening		
a_cmplx_default	Double. Default if <i>a_cmplx</i> =0.0 on input.	2.5d0
g_cmplx_default	Double. Default if <i>g_cmplx</i> =0.0 on input.	1.8d0
p_cmplx_default	Double. Default if <i>p_cmplx</i> =0.0 on input.	1.8d0
w_avrge_default	Double. Default if <i>w_avrge</i> =0.0 on input.	2.5d0
Default values in case of aggressive coarsening		
a_cmplx_agg_default	Double. Default if <i>a_cmplx</i> =0.0 on input.	1.2d0
g_cmplx_agg_default	Double. Default if <i>g_cmplx</i> =0.0 on input.	1.2d0
p_cmplx_agg_default	Double. Default if <i>p_cmplx</i> =0.0 on input.	1.2d0
w_avrge_agg_default	Double. Default if <i>w_avrge</i> =0.0 on input.	1.2d0
Scratch files (eg, used for memory management)		
ioscratch_default	Integer. I/O unit to be used for scratch-files if the corresponding primary parameter, <i>ioscratch</i> (→ Section 5.6.4), is not specified or zero on input.	31
scratchfile	Character. Name of Scratchfile.	"scratch.samg"

6.4.3 Some special parameters

Parameter	Explanation		Default		
Parameter influencing the performance					
ntake_res_in	Integer. By default, the primary parameter <i>res_in</i> is an output parameter. However, if the residual of the starting guess is already known before calling SAMG, one may want to tell SAMG to not recompute the residual but rather use the value which is passed via <i>res_in</i> . This can be done by setting <i>ntake_res_in</i> :		0		
	0	SAMG will regard <i>res_in</i> only as an output variable. That is, SAMG will compute the residual of the starting guess and return it via <i>res_in</i> .			
	1	SAMG will assume that, on input, <i>res_in</i> equals the residual of the starting guess. Important: Be sure that this residual has been computed using the correct norm (→ <i>norm_typ</i> , Section 5.6.4).			
ncyc_min	Integer. Minimum number of cycles to be enforced.		0		
iter_pre	Integer. If AMG is used as a preconditioner, very rarely, it may happen that the accelerator runs into technical problems caused by the nature of the selected first approximation. In such cases, one can enforce SAMG to perform a few <i>pre-iteration steps</i> (ie, stand-alone cycles) before the accelerator is actually turned on. <i>iter_pre</i> defines the number of pre-iteration steps (hardly more than two).		0		
Parameter influencing only the output					
ncyc_start	Integer. To compute realistic values for the <i>average</i> residual reduction factor, the very first cycles should be ignored. (This is because residual reduction is generally faster in the very first cycles.) <i>ncyc_start</i> defines the number of cycles to be skipped. Only relevant for the print output.		2		
show_un_res	Integer. In addition to total residuals, display residuals belonging to individual unknowns with number 1,2,..., <i>show_un_res</i> . Only relevant for print output and only if <i>nsys</i> >1.		0		
Parameter for debugging					
mode_debug	Integer. Unless <i>mode_debug</i> =0, certain debug options can be set.		10		
	[1:1]	Options for both SAMG and SAMGp			
		1		If <i>nsys</i> >1 and unknown-based approaches, on all levels, the following is checked: (1) which of the unknowns are still alive; (2) which ones have only 1 variable left although they are not uniquely defined.	
		>1		not yet used	
	[2:2]	reserved for SAMGp			
ntyp_accel	Selects version of conjugate gradient		11		
	11	new conjugate gradient			
	12	old conjugate gradient			

6.4.4 Description of the default switch *n_default*

In Section 5.6.3, we have introduced the primary parameter *n_default*. The below tables display the values which are actually assigned to the hidden parameters, *nwt* and *ncg*, via *n_default*. Recall that extended interpolation, *nwt*=3, is currently being re-designed (→ Section 6.2.5).

Remark: Defining the hidden parameters *nwt* and/or *ncg* explicitly via their set-routines (→ Section 6.1) has a higher priority than selecting them implicitly via *n_default* as described in the previous tables. Generally, explicit settings overrule implicit definitions. Moreover, explicitly defined parameters keep their value until they are either explicitly re-defined or cleared. Consequently, for instance, re-calling SAMG with a new value of *n_default* has no effect on *nwt* and/or *ncg* if they have been explicitly defined via their corresponding set-routine.



<i>n_default</i>	<i>hidden params</i>	<i>comments</i>
Use if matrix A has no “critical” mixed-sign rows		
10	nwt=1, ncg=1	Employs the simplest (<i>direct</i>) interpolation. Coarsening gets the more aggressive, the larger the 2nd digit of <i>n_default</i> .
11	nwt=1, ncg=12	
12	nwt=1, ncg=11	
13	nwt=1, ncg=111	
15	nwt=2, ncg=1	Same as previous, except that <i>standard</i> interpolation is employed. In general, this kind of interpolation is more robust but also more costly.
16	nwt=2, ncg=12	
17	nwt=2, ncg=11	
18	nwt=2, ncg=111	
Use if matrix A does have “critical” mixed-sign rows		
20 or 99	nwt=2, ncg=4	Employs <i>standard</i> interpolation. Coarsening gets the more aggressive, the larger the 2nd digit of <i>n_default</i> .
21	nwt=2, ncg=42	
22	nwt=2, ncg=41	
23	nwt=2, ncg=411	
25	nwt=3, ncg=4	Same as previous, except that <i>extended</i> interpolation, <i>nwt</i> =3, is employed. Recall that this interpolation is currently being re-designed (→ <i>nwt</i> , Section 6.2.5).
26	nwt=3, ncg=42	
27	nwt=3, ncg=41	
28	nwt=3, ncg=411	

The settings of the following table are completely analogous to those in the previous one, except that variables with **only positive couplings** are treated "by absolute value".

<i>n_default</i>	<i>hidden params</i>	<i>comments</i>
no "critical" mixed-sign rows		Settings 30-48 are completely analogous to the settings 10-28, respectively, except that variables with <i>only</i> positive couplings are treated "by absolute value". Hence, the settings in this table ensure that – at least formally - coarsening does not get stuck in case there are many rows with <i>only</i> positive couplings. Since this happens occasionally, the values of the next table are generally preferable to the previous ones. Note, however, that these settings may cause a (slight) increase in the setup time.
30	nwt=1, ncg=2	
31	nwt=1, ncg=22	
32	nwt=1, ncg=21	
33	nwt=1, ncg=211	
35	nwt=2, ncg=2	
36	nwt=2, ncg=22	
37	nwt=2, ncg=21	
38	nwt=2, ncg=211	
"critical" mixed-sign rows		Recommendation: Set <i>n_default</i> =40 or <i>n_default</i> =42 if <i>standard</i> or <i>aggressive</i> coarsening is to be used, respectively. Note that <i>n_default</i> =0 is identical to <i>n_default</i> =40.
40 or 0	nwt=2, ncg=5	
41	nwt=2, ncg=52	
42	nwt=2, ncg=51	
43	nwt=2, ncg=511	
45	nwt=3, ncg=5	
46	nwt=3, ncg=52	
47	nwt=3, ncg=51	
48	nwt=3, ncg=511	

7 Special SAMG Features

This section describes various special SAMG features. Some of them - such as the automated solver control in Section 7.5 - are still under development and optimization. First versions are released for testing and we would appreciate any comments and suggestions for further improvements. Features and/or options printed in green are not yet released for a general use.

All hidden parameters introduced in this section can be accessed as described in Section 6.1.

7.1 OpenMP parallelization

Depending on the type of license purchased, SAMG is OpenMP parallel⁵⁰. However, since AMG's coarsening process is not fully parallel, its parallelization is necessarily a compromise between performance and robustness. Hence, to some extent, the coarsening might depend on the number of threads used. Furthermore, the number of variables per thread should be large enough. Should this cause any problems, the user might wish to adjust certain parameters of the following Section (in particular, *icolor_omp*).

7.1.1 General remarks

If the library has been built with OpenMP support, most multi-processor machines will automatically make use of the OpenMP feature and choose the number of threads to equal the complete number of cores of that machine. If more than one thread is employed, SAMG will report the exact number of threads (assuming the value of *mode_mess* has been set to ≥ 0 , → Section 6.4.1), for instance,

```
>>> MESSG: Current number of threads = 2
```

Via the environment variable `OMP_NUM_THREADS` one can explicitly define the number of threads to be used: Prior to starting the application from the console, just issue

```
setenv OMP_NUM_THREADS <number of threads>
```

for Linux⁵¹ and

```
set OMP_NUM_THREADS=<number of threads>
```

for Windows. Alternatively, the requested number of threads can be defined by calling standard OpenMP routines inside an application.

Important: SAMG is **not thread-safe**, that is, it may be called only by one thread at a time. Instead, multithreading takes place *within SAMG*. As a workaround one may use different instances of SAMG⁵² (up to 10, → Section 7.9 on “Multiple instances”).



Remark: Generally, the number of threads should be *less than or equal to* the number of processors of the machine given. For some machines (in particular, for some SGI's), one should even assure that the number of threads is *strictly less* than the number of processors. Otherwise, performance may substantially degrade. Note that SAMG does not change the number of threads. In this context, see also the remark on **hyperthreading** in the next section.



Regarding an optimal OpenMP performance, there are a few hidden parameters. All of them are initialized to reasonable default values and, if required, can be changed as described in Section 6.1. However, under all normal conditions, there should be no need to change the default values.

⁵⁰ Currently, the most frequently used SAMG's components have been parallelized (→ Section 7.1.3).

⁵¹ Assuming a **(t)csh** shell.

⁵² Currently available only in connection with source licenses.

Parameter	Explanation		Default	
Fine-tuning of OpenMP				
icolor_omp	Integer. Switch to choose between different variants to perform the coarsening process (“coloring”) in parallel. ⁵³			
	[1:1], icolor_omp_mode	Specifies the degree of parallelization		2
		0	No OpenMP parallelization	
		1	The standard coarsening is parallel, the aggressive one not (ie, “reduce”)	
		2	Full OpenMP parallelization	
		3-5	used internally	
	[2:], icolor_omp_level	Specifies the number of levels the OpenMP parallelization is applied to. The default setting “0” corresponds to “3”.		
irestriction_openmp	Integer. Switch to choose between different variants to perform the fine-to-coarse residual restriction in parallel.			
	2	Dynamic choice between 0 and -1: For single-thread runs select “0”, otherwise “-1”.		2
	1	While the actual computation of the residual is done in parallel, its restriction (via the transposed interpolation) is sequential: Parallel efficiency is reasonable, no additional memory required.		
	0	Sequential computation, no OpenMP		
	-1	The transposed interpolation matrix (for restricting the residual) is computed at the beginning of the solution phase and then kept in memory. This results in highest parallel efficiency at the expense of quite some extra memory.		
nmin_matrix	Integer. Min. number of rows per thread in matrix times vector operations required for SAMG to run in parallel.		1	
nmin_vector	Integer. Min. number of rows per thread in vector - vector operations required for SAMG to run in parallel.		1	
nmin_matrix_resc	Integer. Min. number of rows per thread in transposed matrix times vector operations (for computing the restriction) required for SAMG to run in parallel.		1	
ntyp_galerkin	Integer. Switch to choose between different variants to compute the coarse level Galerkin operators in parallel.			
	0	Dynamic choice between 1 and 6: For single-thread runs select “1”, otherwise “6”.		0
	1	Sequential computation, no OpenMP.		
	2-6	As a rule of thumb, the larger the value, the higher the parallel efficiency (at the expense of a higher memory requirement). If memory requirement is of major concern, 2-3 can be recommended, otherwise 6 should be selected.		

Note: As mentioned further above, the construction of coarser levels might, to some extent, depend of the number of threads. If this should cause any problems, the user may wish to change the degree of parallelization by adapting the above parameter *icolor_omp*.



⁵³ While the default setting for SAMG is *icolor_omp*=2, it is *icolor_omp*=0 for SAMGp!

7.1.2 Best practice

The performance experienced with OpenMP-parallel SAMG strongly depends on the underlying hardware, especially CPU and memory. In general, SAMG will benefit from an increased number of threads until either the number of physical cores is reached or the memory bandwidth is saturated. Most frequently, **memory bandwidth** is the performance bottleneck, in particular, in connection with sparse matrix solvers such as SAMG. Moving past these barriers may slow down code execution. This issue gets significantly harder on outdated architectures like Intel's FSB-based ones (see below).

In the following we summarize some basic experiences which should be taken into regard before starting to use SAMG in OpenMP mode. Although well-known to OpenMP practitioners, these experiences may be of quite some help for beginners. Ignoring any of these may cause unexpected behaviors and/or suboptimal performance.

7.1.2.1 "First touch" policy

On machines with "non-uniform memory architecture" (NUMA; common multiprocessor system) each socket has its own memory which it can access faster than the memory of remote sockets. In order to exploit the increased memory bandwidth compared to a single socket, one should ensure that the data is distributed across the nodes in such a way that most of the data used by one specific thread is located at the socket where the respective thread is running. For all relevant data which is "under the control of SAMG" (ie, which is allocated inside SAMG), this is automatically ensured by exploiting the system's 'first-touch' policy⁵⁴. However, since the finest-level matrix and vectors are owned by the calling program, the user himself has to ensure data locality for the finest-level data. This can also be done via the first touch policy by simply initializing the vectors in parallel OpenMP regions as outlined in the following skeletons.

```
allocate(ia(nnu),ja(nna),a(nna),u(nnu),f(nnu))
! initialize vectors
!$OMP PARALLEL DO SCHEDULE(STATIC) PRIVATE(i)
do i=1,nnu
    ia(i)=0; u(i)=0; f(i)=0
enddo

! setup ia,u,f values
! initialize matrix for best data locality
!$OMP PARALLEL DO SCHEDULE(STATIC) PRIVATE(i,j)
do i=1,nnu
    do j=ia(i),ia(i+1)-1
        ja(j)=0; a(j)=0
    enddo
enddo

! fill ja,a values
```

Note that OpenMP parallelism inside SAMG uses **static scheduling** based on the number of variables, *nnu*. This is most efficient if the matrix entries are – at least on the average – balanced across the threads⁵⁵. Also note that, of course, it is not mandatory to first initialize everything by zero. If it is possible to directly fill the correct values in a straight first touch loop, then one should do this (eg, when straightly copying the system data from somewhere else). Thus, the above skeleton always works but can eventually be simplified.

⁵⁴Note that such strategies depend on enforcing and retaining a proper thread placement: the so called *thread affinity* or *pinning* might have to be controlled, see the corresponding system's documentation.

⁵⁵Should the matrix entries be very unbalanced, a manual tiling, which distributes the matrix entries evenly across the threads, may be better. However, SAMG does not yet exploit this.

7.1.2.2 Race conditions

Almost all multithreaded numerical algorithms are subject to so called "race conditions", at least with a high likelihood. That is, small deviations in the floating point arithmetic (normally in the order of round-off influence), depending on the "race" history of a thread parallel run. If run with *different number* of threads - but also if run twice with the *same number* of threads! - a code may produce slightly⁵⁶ different numbers. Even if a code is seemingly watertight in this respect, deviations are possible, for instance, if **OpenMP reductions** are used:

Race conditions introduced by OpenMP. According to the OpenMP standard, **OpenMP reductions** accept round-off deviations, even when called twice with bitwise identical input and identical number of threads! In SAMG, OpenMP reductions are being used and, hence, perfectly identical results with two different runs are not strictly guaranteed.⁵⁷

Recommendation: Race conditions should be accepted. Actually, on the basis of mere OpenMP features, they can hardly be completely avoided. Circumventing race conditions can only be achieved by leaving the easy implicit use of OpenMP.



Algorithmic race conditions: Gauss Seidel. SAMG contains another source of race conditions which is "deliberate": The well-known **Gauss-Seidel relaxation**, used as smoother in SAMG, has an intrinsically serial nature⁵⁸. As such, an exact and efficient parallelization is virtually impossible. Its relative, the **Jacobi relaxation**, is trivially parallel but (usually) has significantly worse smoothing properties. In SAMG, a compromise has been made: By "enforcing" the Gauss-Seidel loop to run in parallel and take into account the indispensable race conditions⁵⁹, one buys the parallelism by accepting that the resulting relaxation is not clean Gauss Seidel anymore, but some (race-dependent) mix of Gauss-Seidel and Jacobi. The experience with this approach: normally, the convergence of this hybrid relaxation is close to clean Gauss-Seidel. However does this approach almost certainly introduce race conditions. If this is not acceptable, we recommend switching to Jacobi relaxation.

7.1.2.3 Hardware aspects

Almost all numerical applications, but in particular those using floating point arithmetic with **sparse matrices** (CG, BiCGstab, GMRes, ILU-CG, SAMG,...), extremely depend on the underlying hardware, as regards performance and multicore parallelism. We do not intend to recommend a certain computer architecture, however should an SAMG user be aware of performance bottlenecks that certain architectures might introduce. For instance, because they are simply not tuned for optimal floating point arithmetic but have been designed for specialized applications (like games) or are targeting at all day commodity aspects (like notebooks). Or because they contain outdated architectural components, see below.

Hyperthreading. Certain architectures offer so called "hardware threads" (a paradigm also known as "hyperthreading" or "SMT"). For instance, with 8 physical cores, 16 "processors" are offered, and 16 would be the default number of OpenMP threads. However, whether the use of all 16 threads would really pay off compared to only 8 threads is quite unpredictable and should be carefully checked.

⁵⁶ Although each single race condition event has an influence only at the level of round-off, be aware that corresponding errors might propagate and accumulate. In extreme, numerically unstable situations the total effect may then become serious.

⁵⁷ In the next release, SAMG will introduce a "deterministic mode" in which, given a fixed number of threads, all SAMG components will be deterministic, that is, two subsequent runs with identical conditions and input will produce identical results. This may be at the expense of overall performance.

⁵⁸ Each step of a relaxation loop depends on the outcome of some or all preceeding steps.

⁵⁹ By default, the so called F/C-order in Gauss-Seidel relaxation is chosen, which diminishes the likelihood for races but cannot guarantee to remove them.

Specialized hardware such as notebooks. The main design focus for general purpose notebooks lies on portability. Regarding performance, there are several drawbacks compared to desktop systems of the same generation. One of them usually is a **reduced memory bandwidth** because memory slots are not equipped properly. Additionally, due to **thermal limitations** the CPUs do not work at their full capacity. Especially multi-core processors only work at reduced frequencies the more cores are active, which in return makes it impossible to observe any scaling in CPU performance. Although these statements may not hold for all notebooks sold on the market, one should keep this in mind whenever running compute- and memory-extensive applications on a mobile computer.

Slow memory, outdated hardware. Example: FSB technology. The front-side bus (FSB) is a communication interface that was used in all of Intel's Atom, Celeron, Pentium, Core 2, and Xeon processor models through about 2008. Its main purpose is to transfer the data between the CPU and the Memory Controller. For fast multi-core CPUs, the FSB causes a serious bandwidth bottleneck. In modern systems based on HyperTransport (AMD) and QPI (Intel), the memory controller (north-bridge) is placed on the CPU itself, providing higher memory bandwidths for memory transfers. Since for SAMG – like for *any other* sparse linear solver – memory bandwidth causes the major performance bottleneck, hardware based on the FSB technology typically shows only very limited performance. Modern technology will generally perform significantly better.

7.1.2.4 Mixed aspects

Measuring computing times. To obtain realistic time comparisons for runs using different numbers of threads (especially if comparing with 1-thread runs), one should always consider **wall-clock times** as obtained by calling the OpenMP runtime routine `OMP_GET_WTIME()`. Otherwise, one may accidentally measure the *accumulation* of CPU times.

Memory allocation. SAMG initially estimates memory requirements and allocates memory accordingly. If the amount allocated will turn out to be insufficient, SAMG will reorganize its memory in a 'memory extension' phase. In this phase, currently, SAMG cannot ensure data locality any more. As a work around one should try to **avoid memory extensions** as much as possible. This can be achieved by properly selecting the complexity values described in Section 5.7. In case many similar linear systems are to be solved successively (as in time dependent applications), the routine `samg_get_complexities` will be helpful in minimizing allocation effort (→ Section 7.11.1).

7.1.2.5 Changing number of threads between SAMG calls

The number of threads should not be changed between SAMG calls. If, for any reason, this cannot be avoided, such a change has to be preceded by a call to `samg_leave` (→ Section 2.5).

7.1.3 Components not yet parallelized

In the current version of SAMG, the most frequently used components are OpenMP parallelized. However, certain others are not yet included, the most important of which are the following:

- routines performing memory extensions⁶⁰,
- ILU-based smoothers,
- interpolation routines of point-based approaches,
- interpolation routines based on coordinates,
- coarsening and interpolation routines for clustering,
- relaxation of interpolation.

⁶⁰ Memory extensions can be avoided by properly selecting the complexity values described in Section 5.7. In case many similar linear systems are to be solved successively (as in time dependent applications), the routine `samg_get_complexities` may be helpful to optimize memory requirements (→ Section 7.11.1).

7.2 Stopping criteria for the iterative solution process

Besides robust convergence, the availability of reasonable and reliable stopping criteria is of major concern for iterative solvers. Two standard criteria have already been introduced earlier, controlled by the primary parameters **ncycle** (maximum number of iterations, → Section 5.5) and **eps** (required residual reduction, → Section 5.2). Although these standard criteria are sufficient for many situations, some further criteria are provided. All currently available criteria (including the above-mentioned standard ones), can be subdivided into the following two classes which will be described separately in the following subsections.

Convergence criteria: Criteria which aim at detecting whether the current approximation satisfies a user-prescribed accuracy.

Safety checks: Checks which aim at detecting whether, for any reason (such as roundoff or divergence), it just makes no sense any more, to continue iterating.

Remark: Except for the standard criteria controlled by *ncycle* and *eps*, all other criteria are effective only in the default situation that AMG is used as a preconditioner for either CG, BiCGstab or GMRes. That is, if accelerators are explicitly de-activated by the user (so that AMG operates as a stand-alone solver), all of the *new* parameters introduced below are ignored.



7.2.1 Convergence criteria

Two criteria are currently available to detect whether the most recent iteration u_i , say, satisfies a user-prescribed accuracy, one based on the *residual* ($res = \|f - Au\|$) and one on the *approximation*.

Residual criterion. Let res_i and res_0 denote the residuals of the i -th and the 0-th iteration step⁶¹, respectively, measured in the norm as selected by the primary parameter *norm_typ* (→ Section 5.6.4). Then, depending on the sign of *eps*, iteration terminates based on either a “relative” or an “absolute” criterion:

$eps \geq 0.0d0$	Relative criterion: iteration stops if $res_i \leq eps \cdot res_0$ is fulfilled
$eps < 0.0d0$	Absolute criterion: iteration stops if $res_i \leq eps $ is fulfilled

Remark: The “absolute” criterion should be used when the “relative” one does not make sense. A typical situation: *time stepping* applications where *eps* is typically of the form $eps = -\eta \|f\|$ with η being a reasonably small – but fixed – number and $\|f\|$ the norm⁶² of the current right hand side.



Approximation criterion. Let u_i and u_{i-1} denote two successive approximations. Then, the iteration terminates if

$$\|u_i - u_{i-1}\|_{\max} < factor_app_var \cdot \max(\|u_i\|_{\max}, \|u_{i-1}\|_{\max})$$

with a given value of the threshold **factor_app_var** (see below table).

Remark: This criterion is only meaningful if the iteration converges sufficiently fast. Otherwise, it may be fulfilled although the current approximation is still far from the wanted solution, for instance, if convergence is extremely slow.



⁶¹ By the 0-th iteration step we mean the starting guess.

⁶² This norm should be the same as the one which is used to measure the residuals. Note that $\|f\|$ then equals the residual w.r.t. the zero vector.

Both of the above criteria can be selected or de-selected via the hidden integer parameter **icrits**. This parameter, as well as the threshold **factor_app_var**, can be accessed and modified in the usual way as described in Section 6.1.

Parameter	Explanation			Default
Parameters related to the convergence criteria				
icrits	Integer. If <i>icrits</i> =0, both convergence criteria are de-selected. Otherwise, the first digit specifies how the selected criteria are to be interpreted.			
	[1:1]	1	Iteration of SAMG stops if <i>all</i> of the <i>selected</i> convergence criteria are fulfilled.	110
		2	Iteration of SAMG stops if <i>just one</i> of the <i>selected</i> convergence criteria is fulfilled.	
	[2:2]	The residual criterion is		
		0	de-selected.	
		1	selected.	
	[3:3]	The approximation criterion is		
		0	de-selected.	
		1	selected.	
factor_app_var	Double. Threshold specifying the approximation criterion.			1.0d-10

7.2.2 Safety checks

The most straightforward safety check is to simply restrict the total number of allowed iteration steps. As already described in Section 5.5, this can be achieved by setting the primary (sub-) parameter **ncycle** to the maximum number of allowed steps. Additional safety checks and related parameters are described in the following.

Quasi- and real residual. At each iteration step, SAMG computes the **quasi residual** (implicitly as part of the currently selected accelerator) and the **real residual** (residual computed "from scratch"). Theoretically, both residuals are identical. In practice, however, during the process of a convergent iteration, roundoff influence will cause the quasi residual to become increasingly different from the real one up to the point, where both residuals may be different in all digits. At that point, it generally does not make much sense to continue iterating: Although the *quasi* residual may still decrease further in subsequent iteration steps, there is no corresponding increase of accuracy any more and all further computation is at the level of roundoff. In any case, the *real* residual typically does *not* decrease any further⁶³. This behavior gives rise to a safety check which aims at detecting whether the level of roundoff has been reached and iteration should be stopped:

Quasi residual check. Let res_q and res denote the *quasi* and *real* residuals, respectively, of a given iteration and denote $ratio = \max(res, res_q) / \min(res, res_q)$. Then, given the threshold **factor_quasi_res** and a (small) number $n > 0$, the *quasi residual check* limits the number of iterations for which $ratio > factor_quasi_res$ is allowed to n . If the number of such iterations exceeds n , SAMG will stop iterating and return with a warning. (Depending on whether the currently active accelerator is CG, BiCGstab or GMRes, the code number is -831, -841, -851, respectively.)

Note: In evaluating the value of **ratio**, residuals are computed in the norm as selected by the primary parameter **norm_typ** ($\rightarrow$ Section 5.6.4). Only exception: If GMRes is used as accelerator, *ratio* is always evaluated w.r.t. the *L2*-norm, independent of the setting of **norm_typ**, the reason being that the quasi residual w.r.t. the *L2*-norm is implicitly computed anyway and, hence, for free.



⁶³ See, however, the remark at the very end of this section.



Note: By default, both types of residuals are computed *in each iteration step*, causing some overhead. The parameter ***nth_res_scratch*** (see below), can be used to skip some steps.

Residual stagnation check. Given the threshold ***factor_res_var*** and a (small) number ***n>0***, this checks the variation of the (real) residuals over the last $n+1$ iterations⁶⁴. If $res_{max}/res_{min} < factor_res_var$, SAMG will stop iterating and return with a warning. (Depending on whether the currently active accelerator is CG, BiCGstab or GMRes, the code number is -832, -842, -852, respectively.) The concrete value of ***factor_res_var*** (see table below), should be (slightly) larger than 1.0. (A value ≤ 1.0 would cause the above inequality to be never fulfilled, meaning that the stagnation check would actually be de-activated.)

Accelerator re-start check. Given a number ***n>0***, this check limits the number of accelerator re-starts⁶⁵ to n . If this number is exceeded, SAMG will stop iterating and return with a warning. (Depending on whether the currently active accelerator is CG, BiCGstab or GMRes, the code number is -833, -843, -853, respectively.)

All safety checks can be selected or de-selected via the parameter ***iauto_stop***. This parameter, as well as all others mentioned above, are summarized in the following table.

Parameter	Explanation		Default	
Parameters related to the safety checks				
iauto_stop	Integer. If <i>iauto_stop</i> =0, all safety checks are de-selected. Otherwise, its digits are used to select/de-select individual safety checks. Regarding the meaning of <i>n</i> in this table, see the above detailed description.			
	[1:1], iauto_stop_level	Currently, the concrete value of the 1st digit has no meaning, just set it to 1.	11	
	[2:2], icheck_quasi_res	0		Quasi residual check is de-selected.
		n		Quasi residual check is selected.
	[3:4], num_old_res	0		Residual stagnation check de-selected.
		n		Residual stagnation check is selected.
	[5:6], num_old_apps	Not yet implemented: reserved for <i>solution</i> stagnation check.		
[7:7], nfailed_accsteps	0	Accelerator re-starts are not limited.		
	n	Maximum number of acceleration re-starts (only CG & BiCGstab)		
factor_quasi_res	Double. Threshold used for the quasi residual check . See also the below parameter <i>nth_res_scratch</i> .		1.5d0	
factor_res_var	Double. Threshold used for the residual stagnation check .		1.01d0	
nth_res_scratch	Integer. By default, <i>real</i> residuals are computed in each iteration. This can be changed by setting <i>nth_res_scratch</i> >1. Naturally, as long as no real residual is available, the quasi residual check is temporarily deactivated. ⁶⁶			
	=0	Real residuals are computed in each iteration	0	
	=n>0	Real residuals are computed only every <i>n</i> -th iteration		

⁶⁴ res_{min} and res_{max} denote the minimum and maximum values of the residuals, taken over the last n iterations.

⁶⁵ For safety reasons, all accelerators are automatically re-started in case of unexpected numerical troubles.

⁶⁶ If no real residual is known, its quasi analog will be displayed instead (if $iout>1$, $\rightarrow$ Section 5.8.2) and also used for checking the standard *eps*-criterion. Hence, the parameter ***nth_res_scratch*** should be used with care: it may happen that the *eps*-criterion w.r.t. the quasi residual is satisfied (and SAMG terminates) while it is **not** yet satisfied w.r.t. the real residual (there will be no warning!). Finally, recall that, in case of GMRes, quasi residuals are always based on the L2-norm. Hence, if a different type of norm has been selected via ***norm_typ***, for reasons of consistency, the *eps*-criterion is checked only every ***nth_res_scratch*** iteration.

Should any of the above-mentioned safety checks detect a problem, SAMG is caused to return and issue a warning. While this seems natural for the *residual stagnation* and the *accelerator re-start check*, the situation is different for the *quasi-residual check*:

Remark on the quasi-residual check. A warning by the *quasi-residual check* will typically occur if a tolerance ($=\text{eps}$) has been prescribed which is so small that it cannot really be reached by the computer (given the limited accuracy of a computer representation). In such cases, SAMG will reach the level of roundoff before the prescribed tolerance is satisfied. It is usually completely normal to stop iterating at that point since additional iterations would just cause a waste of computing time, not giving any further improvement in the solution. However, since SAMG simply did not meet the user-prescribed tolerance, it gives a warning. Under all normal circumstances it is perfectly OK to ignore this warning since SAMG's numerical solution is as good as it can be. Note that the corresponding warning code returned by SAMG is -831, -841 or -851 in case the accelerator used was CG, BiCGstab or GMRes, respectively⁶⁷. Hence it is easy to detect/control such situations after the return from SAMG and, to be on the safe side, one may check the primary SAMG parameters *res_in* and *res_out* manually.

7.2.3 Best practice

7.2.3.1 Quasi-residual check

You can turn the **quasi-residual check** off by changing the parameter *iauto_stop* accordingly (→ above table). However, we would not recommend doing this since this check gives a convenient way to compute a solution only up to roundoff. In connection with CG and BiCGstab, this works quite reliably. In connection with *flexible* CG or GMRes we recommend to reasonably limit the dimension of the Krylov space, for instance, to a value somewhat below the typically expected iteration number (→ *nkdim* in Section 5.5). This is because, making a re-start helps to detect the level of roundoff.

In spite of the fact that the quasi-residual check has been reached, it may happen in certain cases (eg, in case of strange/random first guesses) that the *real* residual can still be reduced further to some limited extent simply by **restarting the accelerator**. To be on the safe side and exclude such situations, one may re-start SAMG once with the previous approximation now being the first guess.

Note: If the previous call to SAMG has been with *iswit*=4, the re-start can be done without a new setup by calling SAMG again with *iswit*=1 (→ Section 5.6.1) and *ifirst*=0 (→ Section 5.2).

7.2.3.2 Residual stagnation check

The **residual stagnation check** gives another possibility to detect whether it is a good idea to terminate the iteration. For instance, one might set *iauto_stop*=1103 instead of *iauto_stop*=11, say. Although this is a very convenient check, we have not selected this to be the default: The residual stagnation check should be activated only if it is known that the typical (fast) AMG performance is to be expected for the applications at hand. Otherwise, there is the risk that the iteration will terminate far from the solution simply because convergence is too slow.

7.2.3.3 Approximation criterion

The **approximation criterion** is also rather convenient. However, just like the *residual stagnation check*, it is only meaningful if it is known that the typical (fast) AMG performance is to be expected for the applications at hand. Otherwise, the approximation criterion may be fulfilled although the current approximation is still far from the wanted solution, simply because convergence is too slow.

⁶⁷ Remember that the safety checks are only available if SAMG is used as a preconditioner for any of the built-in accelerators, CG, GMRes or BiCGstab.

7.3 Providing grid coordinates

If applied in the usual way, SAMG operates purely algebraically, that is, it does not know anything about geometry. However, some knowledge of the geometry in terms of **grid coordinates** opens new possibilities or at least gives more flexibility in constructing suitable AMG hierarchies. For instance, various options which require the knowledge of coordinates in **coarsening** or in the definition of **interpolation** have already been introduced in Section 5.4 ($\rightarrow$ *nsolve*). Moreover, in certain applications (eg, linear elasticity) the availability of coordinates may help to improve SAMG's performance. Although providing coordinate information means that the black-box character of SAMG is lost to some extent, this extra information is usually available anyway, and - as long as no restrictions are put on the *shape* of the domain, or the *kind* of grid to be used - the major advantages of SAMG still hold.

Remark: Some features regarding the exploitation of geometric information in form of coordinates are still under development and not yet released for use in a production environment. We will explicitly mention such cases.



Geometric information is made known to SAMG simply by providing a vector of coordinates. A call to the following subroutine allocates the corresponding memory:

```
subroutine samg_coo_vec_alloc(ndim,nnu,ierr) .
```

Here *ndim* and *nnu* are integer input parameters denoting the spatial dimension and the total number of variables, respectively. Unless the integer error flag *ierr* is nonzero upon return from this routine, this call will allocate memory (*ndim* double precision vectors of length *nnu* each) to store the coordinates. Once this extra memory has been allocated, the coordinates (x,y,z) of any variable *i* with $1 \leq i \leq nnu$ can be set by calling⁶⁸:

```
subroutine samg_coo_vec_set(i,x,y,z,ierr) .
```

If *ierr* is nonzero upon return, the program should be terminated.

Remark: Since the memory provided by calling the above routine is explicitly user-allocated outside SAMG, it is also the user's responsibility to deallocate this memory. That is, it is not sufficient to call one of the subroutines provided to clear SAMG's memory (such as *samg_refresh*, $\rightarrow$ Section 2.5). Instead, to explicitly clear the above memory, the following subroutine needs to be called:

```
subroutine samg_coo_vec_dealloc(ierr) .
```



⁶⁸ If *ndim*=2 the third coordinate is irrelevant and not used.

7.4 Treatment of “special” variables

Depending on the application, certain equations of a linear system may be of a different nature than requested by an efficient AMG process, they are critical from an AMG point of view. For instance, some equations may correspond to constraints which have no direct relation to an elliptic PDE. Or they are related to a subsystem of a given PDE system which is not really elliptic. SAMG provides some possibilities to force critical equations to be treated in a special way, for instance, by not incorporating them in the algebraic multigrid process at all. Of course, the corresponding variables then need to be coupled to the rest of the system in some other way.⁶⁹

To be more specific, SAMG allows marking selected variables as “special” and passing additional information on how to treat them. For example, variables may be forced to exist on all levels, or to strictly remain on the finest level only. In the latter case, such variables may or may not be included in the smoothing on the finest level. Their coupling to the “normal” variables may also be done in the fashion of an **alternating Schwarz method**. Details will be described in Sections 7.4.3 and 7.4.4.

Restriction: If a sequence of linear systems is to be solved, note the following: Whenever setup data is to be re-used (via setting `iswit<4`, →Section 5.6.1), it is not allowed to modify the marking of special variables as described in this section. Should you want to change marked variables you need to make a new setup (`iswit=4` or `=5`).



7.4.1 Marking variables as being special

First, before any variable can be marked as “special”, an integer vector of length `nnu` needs to be allocated to store the corresponding information, with `nnu` denoting the total number of variables in the given system⁷⁰. This is done by calling the following subroutine prior to calling SAMG itself:

```
subroutine samg_cvec_alloc(nproc,nnu,ierr) .
```

For reasons of compatibility with the parallel library, SAMGp, also the number of processes, `nproc`, needs to be passed⁷¹. Here, in the context of the serial SAMG, always set `nproc=1`.

Remark: Since the memory provided by calling the above routine is explicitly user-allocated outside SAMG, it is also the user’s responsibility to deallocate this memory. That is, it is not sufficient to call one of the subroutines provided to clear SAMG’s memory (eg, `samg_refresh` → Section 2.5). Instead, to explicitly clear the above memory, the following subroutine needs to be called:

```
subroutine samg_cvec_dealloc(ierr) .
```



Unless the integer error flag `ierr` is nonzero, a user can now assign certain integer values, so-called “priority values”, to selected variables. Legal values will be described in the next sections. The **initial setting** of priority values is zero, that is, no special condition is imposed and control is completely left to SAMG (the default). Non-zero values mean that a variable is special, and the numerical value tells SAMG what to do with it.

⁶⁹ The options described in this section are not yet available in connection with **Uzawa smoothing**.

⁷⁰ If certain variables are forced into all *coarser* levels (ie, if `ipriority=1` or `2`, see Section 7.4.2), additional integer memory will be required to store information on coarser levels. The user does not need to care of this since the vector allocated via the subroutine `samg_cvec_alloc` will automatically be extended. However, such memory extensions can be avoided by allocating more memory to begin with, for instance, by calling `samg_cvec_alloc` with `2*nnu` rather than `nnu`.

⁷¹ To be more precise, the exact number of processes is not important. It would be sufficient to pass `nproc=1` and `nproc>1` for the *serial* and the *parallel* case, respectively.

To assign a specific priority value, *ipriority*, to the *i*-th variable, $1 \leq i \leq nnu$, just call the subroutine

```
subroutine samg_cvec_set(i,ipriority,ierr).
```

7.4.2 Forcing special variables into coarse or fine levels

7.4.2.1 Forcing variables into *coarse* levels

Any set of variables can be forced to exist **on all coarse levels**. Just call `samg_cvec_set` for any such variable, *i*, with *ipriority*=1 or =2, see the below table. In contrast to variables with negative *ipriority* variables as discussed further below, these variables are still fully integrated in SAMG's hierarchical procedure. It is just the coarsening (C/F-strategy) which is influenced this way.

Warning: If a user decides to force special variables to exist on all coarse levels, he should know what he is doing. Generally, the number of corresponding variables should be relatively small and carefully selected. Otherwise, memory requirement may drastically increase!



Remark: For *point-based approaches* forcing into coarse levels should be “pointwise”. That is, if a variable is forced into all coarser levels, all variables living at the same point should also be forced into all coarse levels by assigning the same priority value to all of them.



7.4.2.2 Forcing variables to remain on the *finest* level

Any set of variables can be forced to strictly remain **only on the finest level**. Such variables will neither occur on any of the coarser levels nor will they obtain any kind of correction from coarser levels. Hence, their numerical treatment is not part of the hierarchical procedure which is applied to the rest of the linear system, and variables of that type need to be included in the overall solution process by other means. Two different approaches are available, corresponding to the priority settings *ipriority*=-1 and *ipriority*=-2, respectively:

- Variables with a priority value of *ipriority*=-1 are still “weakly” included in the SAMG-procedure via the **finest-level smoothing procedure**. Hence, this setting makes sense if the selected **smoother** works sufficiently well for resolving the corresponding set of variables. Assuming a substantial set of variables with *ipriority*=-1, plain relaxation for smoothing will often not be sufficient to couple the special equations with the other ones sufficiently well. As a consequence, overall convergence may become rather slow. In such cases, ILU-based smoothing (→ Section 5.4.2) may be more promising, see also the following remark.

Remark: While ILU-based smoothing may help on the finest level, it is often overkill on coarser levels, the reason being that the critical variables do no longer exist there. In this context, one might consider switching to plain relaxation for smoothing on coarser levels. Remember that this can be achieved by means of the hidden parameter *nxtyp_coarse* (→ Section 6.3.2).



- Variables marked by *ipriority*=-2 are not even included in the fine-level smoothing procedure, that is, these variables are *completely* decoupled from those treated by the AMG approach. Instead, they are coupled in the sense of an “**alternating Schwarz method**”, where alternating is between a direct **block solve** for the special variables, and an AMG-cycle for the other variables. This alternating procedure starts automatically as soon as there exist variables with priority value *ipriority*=-2. For more details and further important options, → Sections 7.4.3 and 7.4.4.

Remark: If variables are marked by *ipriority*=-2, (partial) setups with *iswit*=2-3 must currently not be selected (see Sections 5.6.1 and 7.5.2)!



7.4.2.3 Summary

Parameter	Explanation
Forcing special variables into coarse or fine levels	
ipriority	The “basic” priority values permitted for any variable, i , are summarized in this table. More advanced settings are described later in Section 7.4.4.
	0 Nothing is enforced on the i -th variable, that is, control over i is fully left to SAMG (default).
	1 i is forced to exist on all coarse levels unless SAMG detects that, for some important reason, this may not be a good idea.
	2 i is definitely forced to exist on all coarse levels.
	-1 i is forced to remain only on the finest level, decoupled from the overall AMG process except that such variables are still included in the finest-level smoothing process.
	-2 i is forced to remain only on the finest level, completely decoupled from the overall AMG process. Such variables will be solved for simultaneously by a block solver, embedded in a Schwarz alternating process as outlined above and described in more detail in Sections 7.4.3 and 7.4.4.

Note: Calling `samg_cvec_set` with i and `ipriority=0` is equivalent to not calling it for i at all. The setting `ipriority=0` is needed only if, for some technical reason, a non-zero priority status of some i has to be explicitly re-set to zero.



7.4.2.4 Parameters to cover some frequent cases

To simplify the setting of the above basic priority values in certain frequent situations, some hidden parameters allow to assign priority values 1, 2, -1 or -2 simultaneously for groups of variables:

Parameter	Explanation	Default
Hidden parameters to set priority values for certain groups of variables		
cset_zerodiag	Integer. Assign priority value <code>cset_zerodiag</code> to variables, the corresponding matrix rows of which have a zero diagonal entry.	0
cset_lessvars	Integer. Assign priority value <code>cset_lessvars</code> to points which have less than <code>nsys</code> variables attached.	0
cset_lastpnt	Integer. Assign priority value <code>cset_lastpnt</code> to last finest-level point.	0
cset_longrow	Integer. Assign priority value <code>cset_longrow[1:1]</code> (sign given by the sign of <code>cset_longrow</code>) to variables which correspond to rows with the number of entries being $\geq$ <code>cset_longrow[2:]</code> .	0
cset_unknown	Integer. Assign priority value <code>cset_unknown[1:1]</code> (sign given by the sign of <code>cset_unknown</code>) to all variables which correspond to (at most) three unknowns specified via <code>cset_unknown[2:3]</code> , <code>[4:5]</code> or <code>[6:7]</code> .	0
cset_unknown1 cset_unknown2	Integers. These parameters can be used instead of or in addition to <code>cset_unknown</code> (with identical functionality).	0

Examples:

- **`cset_zerodiag=-2`.** Forces all variables which correspond to zero-diagonal rows to remain on the finest level, completely decoupled from the AMG approach for the other variables. A Schwarz alternating procedure is used to couple the special variables and the other ones. For more details, → Sections 7.4.3 and 7.4.4.

- **cset_lessvars=-1.** Excludes all points which have less than *nsys* variables attached from the coarsening. The corresponding variables will be forced to strictly remain on the finest level. However, all these variables will still be included in the finest-level smoothing steps.
- **cset_lastpnt=1.** Forces all variables which are attached to the last point into the coarse levels, unless SAMG decides that this is not a good idea.
- **cset_longrow=-1100.** Forces all variables to remain on the finest level which correspond to rows with 100 (nonzero) entries or more.
- **cset_unknown=-203.** Forces all variables corresponding to unknown 3 to remain on the finest level, completely decoupled from the AMG approach for the other variables.
- **cset_unknown=-10102, cset_unknown1=-213.** Forces all variables corresponding to unknowns 1, 2 and 13 to remain on the finest level. While unknown 13 is completely decoupled from the AMG approach for the other variables (treated by a Schwarz method), the other ones are considered as part of the finest-level smoothing process.

Should more than one of these parameters be used at the same time, the corresponding setting of priority values is done in the following order:

cset_lessvars → cset_lastpnt → cset_longrow → cset_zerodiag → cset_unknown(1/2).

Remark: If `samg_cvec_alloc` has not been called, setting any of the above parameters will **automatically** allocate the required memory. In this case, its de-allocation will be handled by SAMG in the same way as that for all other internally allocated memory. That is, the subroutine `samg_cvec_dealloc` does **not** need to be called.



7.4.3 Alternating Schwarz method

We have explained in the previous section that variables marked by *ipriority*=-2 are *completely* decoupled from those treated by AMG. In contrast to *ipriority*=-1, the setting *ipriority*=-2 usually characterizes variables for which the internal interdependence is too strong to be resolved just as part of the smoother. Instead, such special variables are coupled to the other ones in the sense of an “**alternating Schwarz method**”, where alternating is between a direct **block solve** for the special variables, and an AMG-cycle for the other variables. This alternating procedure is initialized automatically as soon as there exist variables with priority value *ipriority*=-2 (called “**Schwarz variables**” in the following). If not specified otherwise (→ Section 7.4.4), **no overlap** will be involved. The following parameters define specific aspects.

Parameter	Explanation		Default
Specific aspects of the Schwarz alternating procedure			
nblk_solve	Integer. Specifies <i>when</i> to perform block solves.		
	0	Skip block solve completely (for debugging only!)	1
	1	Block solve after each cycle	
	2	Block solve after each (full) smoothing step	
	3	Block solve after each (partial) smoothing step	
nblk_solver	Integer. Selects the type of block solver used.		
	1	Full Gauss elimination with partial pivoting	3
	2	Full Gauss elimination with total pivoting	
	3	Sparse Gauss elimination (standard version)	
	4	One step of ILUT with <i>lfil_smo</i> , <i>droptol_smo</i>	
	5	Intel's pardiso (if Intel's MKL has been linked in)	
	99	Another instance of SAMG	
nblk_max	Integer. Max. number of variables to be solved for simultaneously in a block solve; see also the following remark.		5000

Attention: If *nblk_max* is exceeded, SAMG will terminate with an error (→ error code 260 in Section 9). Practical values of *nblk_max* depend on the block solver selected. By default, block solution is done by a **direct solver**. Consequently, the number of variables to be solved for simultaneously should not be too large. For instance, for the (standard) sparse Gauss elimination, that number should not exceed 5000, say. If pardiso is available, *nblk_max* may be substantially larger, several 10000's say.

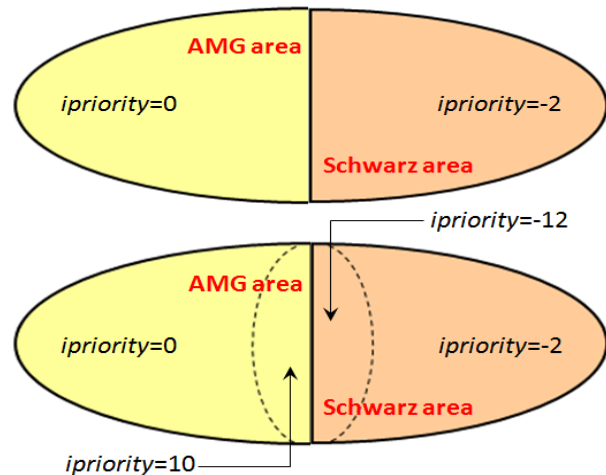


There are a few parameters which may be helpful for a detailed convergence analysis. Usually, however, the corresponding default settings hardly ever need to be changed by an average user.

Parameter	Explanation		Default
Some very special parameters			
nblk_resid	Integer. Used only for debugging		0
	0	Standard residual computation	
	1	Skip Schwarz variables in computing residuals	
nblk_debug	Integer. Initializes some debug information		0
	0	No debug information	
	1	Display residuals of Schwarz block systems	
blk_stab	Double. Control parameter (between 0.0 and 1.0) needed for sparse Gauss elimination: the smaller this parameter, the lower the fill-in, but also the stability (pivoting-strategy!); the larger this parameter, the higher the fill-in, but also the stability.		0.25d0
blk_fillexp	Double. Only needed for sparse Gauss elimination. The smaller this parameter, the lower the memory allocation and the larger this value, the higher the memory allocation. Usually, its value should be around 0.5.		0.5d0

7.4.4 Improved alternating Schwarz method (with overlap)

Should the coupling between the Schwarz variables (variables with *ipriority*=-2) and the "normal" variables be too strong, the above Schwarz alternating method *without overlap* will not converge sufficiently well. Usually convergence will substantially improve, if some "overlap" is incorporated in the alternating process. SAMG provides the possibility to do this "both ways", that is, the block solve can be extended into the AMG area and/or the AMG cycle can be extended into the Schwarz area (via the finest-level smoothing process).



7.4.4.1 The most general case

To indicate that a variable shall belong to the overlap, just add a leading "1" to its current priority value. The following two cases allow defining any kind of overlap:

Alternating Schwarz approach: *ipriority* assignments without and with overlap

1. To indicate that certain Schwarz variables shall also be incorporated in the finest-level smoothing procedure, just replace their priority value -2 by -12. Note that this does not change their status of being Schwarz variables.

2. To indicate that certain variables of the AMG area shall also be incorporated in the block solution process, first remember that variables inside the AMG area have priority values 0, 1, 2 or -1. Changing this to 10, 11, 12 or -11, respectively, indicates that the corresponding variables are to be included in the block solve. Note that any such change does not change their status of belonging to the AMG area.

Parameter	Explanation		
ipriority	Overlap variables inside the “AMG-area”		
	10	Equivalent to <i>ipriority</i> =0	... the only difference being that corresponding variables are additionally taken into account by the block solve.
	11	Equivalent to <i>ipriority</i> =1	
	12	Equivalent to <i>ipriority</i> =2	
	-11	Equivalent to <i>ipriority</i> =-1	
	Overlap variables inside the “Schwarz area”		
	-12	Just like variables with <i>ipriority</i> =-2, those with <i>ipriority</i> =-12 are also Schwarz variables, the only difference being that the latter are additionally taken into account by the finest-level smoothing procedure.	

Clearly, for efficiency reasons, the overlap variables should be carefully selected:

- This is particularly crucial for the overlap which extends from the Schwarz into the AMG area. Since all corresponding overlap variables become part of the block solve, their number should be limited. Remember that, by default, each block solve is done by a direct solver and that the size of the corresponding system is limited by *nblk_max* (→ previous section).
- In principle, the size of the overlap from the AMG into the Schwarz area is less critical, it may even cover the whole area (achieved by replacing **all** priority values -2 by -12). However, in general, the corresponding influence on the convergence is relatively low and selecting such a large overlap usually does not pay.

Remark: As already mentioned earlier: if variables are marked by *ipriority*=-2 (or -12), (partial) setups with *iswit*=2-3 must currently not be selected (see Sections 5.6.1 and 7.5.2)!



7.4.4.2 Two extreme cases for demonstration

1. Assume there exists no variable with *ipriority*=-2 but only some with *ipriority*=-12. This means that all of them are solved for by the block solve (according to the setting of *nblk_solve* and *nblk_solver*) and additionally taken into account in smoothing the variables of the AMG area.
2. Assume there are **no Schwarz variables** at all (ie, neither *ipriority*=-2 nor *ipriority*=-12 occurs). Then the above priority values still make sense. For instance:
 - a. All variables with *ipriority*=10 are treated by SAMG just like variables with *ipriority*=0. In addition, regular block solves are done according to the setting of *nblk_solve* and *nblk_solver*.
 - b. All variables with *ipriority*=-11 are treated by SAMG just like variables with *ipriority*=-1. In addition, regular block solves are done according to the setting of *nblk_solve* and *nblk_solver*.

7.4.4.3 Automatic setting of an overlap

In many standard situations, there is no need to *manually* define an overlap. Instead, a “natural” overlap can be defined most easily by just setting the following hidden parameters, *nblk_overlap* and *ilu4allschwarz*. In particular, *nblk_overlap* is quite convenient to automatically define an overlap from the Schwarz into the AMG area (which is normally the more important one) of width 1 or 2, where the “width” is defined according to the nonzero coupling structure of the given matrix.

Parameter	Explanation		Default
Automatic selection of overlap			
nblk_overlap	Integer. Specifies the width of “overlap” from the Schwarz area (as defined by <i>ipriority</i> =-2) into the AMG area. This can be set <i>instead of</i> or <i>in addition to</i> marking variables manually as described further above. ⁷² See also the remarks following this table.		
	0	No overlap defined	0
	1	Add direct (ie, distance 1) neighbors of Schwarz variables to the overlap. More precisely, priority values -1, 0, 1 and 2 will be treated as -11, 10, 11, 12, respectively.	
	2	Same, applied to neighbors “of distance 2 ”	
ilu4allschwarz	Integer. Applies only to ILU-based smoothing. For other smoothers, this parameter will be ignored.		
	.true.	ILU-based smoothing on the finest level includes all Schwarz variables (equivalent to replacing <i>all</i> priority values -2 by -12).	.false.
	.false.	ILU-based smoothing on the finest level does <i>not</i> include any of the Schwarz variables (ie, there exist no Schwarz variables with priority value -12).	

Warning: Automatically adding overlap variables is very convenient. However, depending on the specific connectivity structure of the variables, a vast number of additional variables might be introduced as outlined in the following:



Assuming the number of Schwarz variables is reasonably small, there are two situations for which the automatic creation of an overlap by means of setting *nblk_overlap* may run into troubles:

1. If particular matrix rows corresponding to Schwarz variables are more or less **dense**, the overlap will become unacceptably large.
2. If, on the other hand, all Schwarz variables correspond to **sparse** rows, there will usually be no particular problem as long as the number of variables with *ipriority* 1, 2 or -1 is small. However, this is not necessarily true. For instance, **complete sets of unknowns** are occasionally assigned the priority value -1 (eg, to prevent hyperbolic components from coarsening) and, depending on the coupling structure of the Schwarz set of variables, the overlap may become too large to be practical. In such cases one may wish to exclude variables with *ipriority*=-1 from the overlap, that is, by setting *nblk_overlap* priority values -1 should **not** be replaced by -11 but rather be left unchanged. This is achieved by giving *nblk_overlap* a negative sign:

nblk_overlap=-1 and **-2**: Identical to *nblk_overlap*=1 and 2, respectively, except that *ipriority* values -1 are left unchanged (that is, they are not treated as -11).

⁷² If *nblk_overlap*=1 or =2, the priority values of all relevant overlap-variables will be modified according to the values shown in the above table. This happens at the very end, that is, *after* all user-settings have been done.

7.4.5 Additional features and concluding remarks

As an alternative to explicitly assigning priority values for selected variables i via calling subroutine `samg_cvec_set`, one may pass a list of priority values to SAMG via a file:

Parameter	Explanation	Default
Specifying priority values via a file		
cset_read	Character string. If not empty, <code>cset_read</code> denotes the name of a file containing a list of <i>ipriority</i> values for all variables $1 \leq i \leq nnu$, variable by variable. The format of <code>cset_read</code> is assumed to be either “formatted”, “unformatted” or “binary”, depending on the setting of <i>ioform</i> ($\rightarrow$ Section 7.8), the default being “formatted”. If “formatted”, one entry will be assumed per row. Otherwise, reading will be via a single <i>implied do-loop</i> .	" "

Remark: If `samg_cvec_alloc` has not been called, providing a file `cset_read` will **automatically** allocate the required memory. In this case, its de-allocation will be handled by SAMG in the same way as that for all other internally allocated memory. That is, subroutine `samg_cvec_dealloc` does **not** need to be called.



Altogether there are various possibilities to assign priority values to variables. In the following, we summarize on the order in which these assignments are done:

1. **Explicit assignments via subroutine calls.** Explicit assignments require `samg_cvec_alloc` to be called before. Since assignments via `samg_cvec_set` are done prior to calling SAMG, they are done first and, hence, have **lowest priority**. All other potential assignments will be done later (as part of the SAMG run), overwriting explicit assignments.
2. **Assignments via a file.** If a filename `cset_read` has been specified, all corresponding assignments are done as part of the SAMG run, before any of the hidden parameters is considered. Since such a file contains priority values for each variable (although far most of them will be zero), all explicit priority assignments will be overwritten.
3. **Assignments via hidden parameters.** Should hidden assignment parameters be set, they will be considered last and in the order as already mentioned at the end of Section 7.4.2.

That is, assignments will be done in the following order,

<explicit> $\rightarrow$ `cset_read` $\rightarrow$ `cset_lessvars` $\rightarrow$ $\rightarrow$ `cset_zerodiag` $\rightarrow$ `cset_unknown(1/2)`.

Remark: We want to conclude with pointing out that the **Schwarz alternating approach** provides a simple means to decouple subsets of variables, namely, sets for which AMG provides an efficient approach and sets for which AMG is not really suitable. Although this approach may work efficiently in many situations, in the context of SAMG its main purpose is to provide a convenient way to test and analyse more complex matrix problems. Whether or not the options provided by SAMG are sufficient for final *production runs* depends on the overall situation. It may be more efficient to “separate” critical sets of variables outside SAMG and by some more sophisticated approach, for instance an **approximate Schur complement approach**, and apply SAMG only to those equations for which AMG provides a natural approach.



7.5 Automatic solver control

Two automatic control processes are implemented which can be used to improve SAMG's performance if not just one but rather a series of many linear systems is to be solved:

1. **Primary solver control.** If activated, SAMG attempts to **reuse AMG setup data** (more precisely, AMG hierarchies) as much as possible. Currently, no further optimization (eg, regarding the choice of SAMG parameters and/or the general AMG strategy) is done. Extensions in this direction are subject to further developments.
2. **Secondary solver control.** Assuming a second, alternative solver is selected, an automatic **solver switching** process can be activated which dynamically switches between the first solver (usually AMG) and the second one (usually a one-level solver), whichever is better. Currently, the second solver has to be selected a priori, no further optimization (eg, regarding the choice of specific parameters) is done. Extensions in this direction are subject to further developments.

These controls can work together or separately. Their use is very simple: SAMG is called in exactly the same way as usually except that one has to specify when to start and when to end the control process. This is done by means of the parameter *iswit*: The control process is activated whenever some *iswit*>5 is used, and it will end as soon as some *iswit*≤5 is used (→ Section 7.5.3). As long as the control process is active, parameter *cntrl_method* is used to decide whether both controls are to be used or just one of them (→ Section 7.5.4). By default, only the primary solver control is active.

Typical scenarios for which the solver control has been developed are outlined in Section 7.5.1. Sections 7.5.2 - 7.5.4 describe how to activate the solver controls. Hidden parameters regarding the solver control in general are described in Section 7.5.5. Section 7.5.6 roughly describes the principle of the solver switching as well as some specific hidden parameters. Finally, in Section 7.5.7 we display some exemplary templates demonstrating the usage of the solver control.

Remark: The automatic solver control described in this section is under development and is being continuously developed further. In particular, its performance and reliability are subject to continuous improvements. We would appreciate any feedback regarding practical experience.



7.5.1 Typical scenario for the solver control

The typical scenario: *Time-dependent* simulation processes where SAMG is not called just once but rather repeatedly to solve a series of many hundreds or even thousands of linear systems. For this scenario, we outline the purpose of the two types of control below.

Remark: The solver control should only be used in situations where all involved solvers are mature and no serious convergence problems are to be expected. Moreover, it should be clear from the physical background of the application that an automatic control really makes sense. For instance, if system matrices tend to change dramatically from one system to the next (eg, in size, structure and/or algebraic properties), any attempt to reuse setup data cannot be efficient and the overall computing time will tend to increase rather than decrease.



7.5.1.1 Primary solver control ("re-using setup data")

Using SAMG by treating each linear system "from scratch" (ie, by doing a full setup followed by a solution phase), the setup times would sum up to quite a substantial fraction of the total solution time: depending on the application and the selected AMG strategy, a full setup typically costs the equivalent of many iteration steps. Obviously, the lower the required accuracy per time step and the faster SAMG's convergence, the more the setup cost will dominate the solution cost. Taking into

account that matrices very often do not change dramatically from one time step to the next, it seems to be a waste of computing time to perform a fresh setup for each linear system to be solved. Instead, one should try to reuse setup data (more precisely, AMG hierarchies) from previous time steps as much as possible. Clearly, compared to “freshly” computed AMG hierarchies, the re-used ones are just “approximations”, usually causing a degradation in convergence. Hence, convergence needs to be monitored and some checking is necessary to decide at each time step whether a fresh setup should be done or whether it would be more efficient to continue with old setup data. This is exactly the purpose of the primary solver control.

7.5.1.2 Secondary solver control (“solver switching”)

The secondary control process starts from the fact that there are situations where AMG is less efficient than standard one-level solvers, say, a typical case being applications which are simply **too small**. Generally, the computational overhead required by AMG compared to a standard one-level solver is only compensated by a faster convergence if the given problems exceed a certain size.

Besides this, there are other cases, for which the use of a truly multilevel AMG is an overkill and standard solvers may be more efficient. In particular, in **time-dependent processes** it depends on the overall discretization whether or not AMG pays. For instance, if time steps are very small and/or if only a very low accuracy is required per time step, the application of a standard one-level method may suffice. It also depends on certain characteristics of the application itself (eg, the extent to which convection dominates a simulation process) whether or not AMG pays. As a matter of fact, the most efficient strategy to solve linear systems in a time-dependent application may even change between time steps. Unfortunately, there is often no way to decide on the best solver strategy beforehand. It is the purpose of the secondary solver control to dynamically decide *at run time* which type of solver to use and switch between them if necessary.

7.5.2 Recall: Controlling the re-use of setup data manually

As a motivation, we first recall how the subparameter **iswit** ($\rightarrow$ *iswtch* in Section 5.6.1) can be used to *enforce* the reuse of setup data: If SAMG is called just once or if it is called repeatedly but with largely different matrices, **iswit=5** is the standard choice. For a *manual* control aiming at reusing previous setup data, **iswit=4,...,1** needs to be employed. In detail:

Parameter	Explanation	
Enforcing SAMG to reuse setup data		
iswit	5	Complete SAMG run “from scratch”, that is, including a full setup phase. Upon return, all memory allocated by SAMG during run time is released and setup data is no longer accessible.
	4	Identical to <i>iswit</i> =5 except that SAMG’s setup data is <i>kept in memory</i> upon return. This setting has to be used before any setup data can be reused in subsequent SAMG calls via any of the settings <i>iswit</i> =1-3.
	3	Indicates that SAMG shall do only a partial setup . That is, SAMG will reuse coarser grids and interpolations but update the Galerkin operators.
	2	Indicates that SAMG shall do no setup . That is, SAMG will reuse the <i>complete</i> hierarchy from the previous call, even the Galerkin operators.
	1	Same as <i>iswit</i> =2 except that SAMG assumes the current matrix to be exactly the same as in the previous call.

Attempting to call SAMG with *iswit=3-1* before it has been called with *iswit=4* will cause SAMG to terminate with a **fatal error**. Note that re-using setup data is only meaningful if the previous and the new matrix are “close enough”. Unfortunately, what this precisely means is difficult to quantify. First, at least the number *nnu* of variables and, in case of coupled systems, *nsys* and the vectors *iu* and *ip*,

have to be the same. Second, clearly, no SAMG parameters should be changed between calls which necessarily *require* a new setup. Finally, in order to limit the degradation of convergence due to a “wrong hierarchy”, the algebraic properties of the new matrix (eg, its smoothest eigenvectors) should be similar to those of the previous one. Whether or not this is true to a sufficient extent needs to be controlled by explicitly monitoring SAMG’s convergence behavior. Based on this, a decision on a new setup can be made “on the fly”.

One-level solvers: Note that *iswit*=2-3 only applies if SAMG is truly multilevel, it does not apply if SAMG is used as an ILU-type one-level solver (→ Sections 7.6.1.1 and 7.7.2). However, some limited cost may also be saved in such cases by re-using previous ILU-decompositions. Independent of *iswit*, this is enforced by setting the hidden parameter *lu_reuse=.true.*, the default being *.false.* **Clearly, re-using ILU-decompositions requires the one-level solver to be used as a preconditioner!** See also “dirty trick” in Section 7.6.1.



7.5.3 The primary solver control: re-using setup data

If the re-use of setup data is enforced via *iswit*=4,...,1, it is the user’s responsibility to implement some kind of convergence monitoring (→ previous section). Such monitoring is done *automatically* if the **primary solver control** is activated. This is done simply by using settings *iswit*=6,...,9 instead of *iswit*=4,...,1: **More precisely, in solving a series of linear systems, the primary control process is activated when some *iswit*>5 is used, and it terminates as soon as some *iswit*≤5 is used.** In contrast to the settings *iswit*=3-1, the settings *iswit*=7-9 do *not* assume a full hierarchy to be available from a previous run. That is, it is ok to call SAMG directly with *iswit*=7, 8 or 9 without calling it with *iswit*=6 before: Should no hierarchy be available, SAMG will automatically perform a fresh setup. The details:

Parameter	Explanation
Options for primary solver control	
iswit	The settings <i>iswit</i> =6,...,9 correspond to <i>iswit</i> =4,...,1 (in this order) with one important difference: While <i>iswit</i> <5 do not include any internal checks (such as monitoring of convergence), the settings <i>iswit</i> >5 do perform some monitoring to find out whether or not the reuse of a previous setup makes sense. If not, SAMG automatically performs a new setup.
	6 Corresponds to <i>iswit</i> =4.
	7 Corresponds to <i>iswit</i> =3. Whenever possible and reasonable, SAMG runs with partial setup . Otherwise, SAMG makes a full setup.
	8 Corresponds to <i>iswit</i> =2. Whenever possible and reasonable, SAMG runs with no setup . Otherwise, SAMG makes a full setup.
	9 Same as <i>iswit</i> =8 except that SAMG assumes the current matrix to be exactly the same as in the previous run.

Manually forcing a new setup: If, for any reason, it is known that the next linear system will be “substantially different” from the previous one, the next call to SAMG should be with *iswit*=6, this way enforcing a new setup. This prevents SAMG from making costly and wasteful attempts to reuse old setup data. Alternatively, one may issue

```
call samg_refresh(ierr) .
```

This will not only enforce a fresh setup next time SAMG is called but will also release all of SAMG’s dynamic memory, re-set certain control data as well as the complete history of the previous control process and start a new control process from scratch. This subroutine should also be called whenever there is a conflict between the requirement to re-use setup data and the current parameter settings of SAMG, for instance, when the problem size (*nnu*, *nsys*) or the general solution strategy (eg, type of coarsening) is to be changed. Finally, note the following:

Remark: If a primary solution process is terminated and, at some later stage, activated again, by default SAMG assumes the overall situation (eg, regarding the type of linear systems) to be similar as before. In particular, SAMG attempts to re-use certain control data as well as the history of the previous control process. To prevent this, calling `samg_refresh(ierr)` tells SAMG to forget the history and start the primary solver control from scratch.



7.5.4 The secondary solver control: solver switching

If the primary solver control has been activated (→ previous section), by default, the secondary solver control is not yet active. This is changed by setting the hidden parameter `cntrl_method=0`. Details:

Parameter	Explanation	Default
Select solver control mode		
cntrl_method	Assuming the primary solver control is active (→ Section 7.5.3), this parameter is used to distinguish control modes	2
	0 Use both primary <i>and</i> secondary solver control	
	1 Employ only second solver	
	2 Use only primary solver control	

The **second solver** itself is obtained by just calling SAMG with a different set of parameters, namely, those shown in the following table. To distinguish them from the standard parameters, the suffix “_2nd” is added. All SAMG parameters not contained in the table cannot be changed, that is, they are the same as the standard parameters. Typically, the second solver will be a **one-level method** the default being **ILU(0) preconditioned CG**. Regarding a description of other one-level solvers which are meaningful here, see Sections 7.6.1 and 0.

Parameter	Explanation	Default
Parameters to define the 2nd solver		
levelx_2nd	Integer. Max. number of levels for 2nd solver (→ Section 6.2.4) ⁷³	1
nsolve_2nd	Integer. Same as <i>nsolve</i> for the 2nd solver (→ Section 5.4)	21
ncyc_2nd	Integer. Same as <i>ncyc</i> for the 2nd solver (→ Section 5.5)	110100
nrd_2nd	Integer (≤9). Same as <i>nrd</i> for the 2nd solver (→ Section 6.3.1)	1
nru_2nd	Integer (≤9). Same as <i>nru</i> for the 2nd solver (→ Section 6.3.1)	0
force_accel_2nd	Logical. Same as <i>force_accel</i> for the 2nd solver (→ Section 6.3.3)	.true.
droptol_2nd	Double. Same as <i>droptol_smo</i> for the 2nd solver (in case ILUT, ILUTP or MILUT are selected → Section 6.3.2)	0.005
lfil_2nd	Integer. Same as <i>lfil_smo</i> for the 2nd solver (in case ILUT, ILUTP or MILUT are selected → Section 6.3.2)	20

examples for second solver	settings which differ from the default ones	remarks
ILU(0) / CG	(default)	Symmetric problems (<i>isym=1!</i>); restart of CG every 30 iterations
ILU(0) / GMRes	<i>nsolve_2nd=21; ncyc_2nd=138100</i>	Krylov dimension = 9
ILUT / BiCGstab	<i>nsolve_2nd=22; ncyc_2nd=129100; lfil_2nd=10; droptol_2nd=0.001</i>	Restart of BiCGstab every 20 iterations
ICCG	<i>ncyc_2nd=600100</i>	Symmetric problems (<i>isym=3!</i>); restart of CG every 30 iterations

⁷³ That is, by default, the second solver is a one-level method. In principle, however, the second solver can also be an alternative AMG method.

Remark: If SAMG decides to switch solvers, a new setup is always enforced! If solver switching is activated, but no attempt shall be made to reuse setup data, just use *iswit=6* throughout (see also *cntrl_release* in Section 7.5.5).



7.5.5 General control parameters

The following table explains some steering parameters.

Parameter	Explanation		Default
Steering parameters			
new_topology	Integer. Used to indicate whether the <i>logical</i> form of the system (ie, the vector <i>ja</i>) has changed. If not, this can be exploited by ILU-based one-level solvers in building the decomposition. ⁷⁴ At the same time, this parameter also passes information on whether or not re-using SAMG's setup data may be critical.		2
	0	<i>ja</i> is unchanged; re-use SAMG setup if possible	
	1	<i>ja</i> has changed; perform new SAMG setup	
	2	<i>ja</i> has changed; re-use SAMG setup if possible	
rho_min	Double. Minimum convergence factor required for the automatic control to be done. If not achieved, a fresh setup will be done.		0.9d0
rho_ok	Double. Convergence factor which shall be accepted without further attempt to optimize.		0.1d0
mode_cntrl	Integer. Except for the first digit, this parameter defines how to deal with warnings : To be on the safe side, by default, a warning causes a fresh setup (→ <i>ignore_warn</i> below). However, this is not always meaningful. For instance, warnings caused by one of the safety checks (→ Section 7.2.2) should normally be ignored.		5
	[1:1], max_cyc	Number of cycles required before speed of convergence is tested, $1 < max_cyc < 10$.	
	[2:2], nstrict_ncyc	If <i>nstrict_ncyc</i> =0, SAMG regards it as a <i>normal</i> termination ⁷⁵ when the maximum number of cycles is reached. Otherwise not.	
	[3:3], nstrict_quasi	If <i>nstrict_quasi</i> =0, SAMG regards it as a <i>normal</i> termination ⁷⁵ when the quasi residual check is met. Otherwise not.	
	[4:4], nstrict_stagr	If <i>nstrict_stagr</i> =0, SAMG regards it as a <i>normal</i> termination ⁷⁵ when the residual stagnation check is met. Otherwise not.	
	[5:5], nstrict_stagu	If <i>nstrict_stagu</i> =0, SAMG regards it as a <i>normal</i> termination ⁷⁵ when the solution stagnation check is met. Otherwise not. (not yet used!)	
	[6:6], ignore_warn	If <i>ignore_warn</i> =0, SAMG will make a fresh setup whenever SAMG reports a warning. Otherwise, warnings are simply ignored.	

⁷⁴ Currently this is only exploited in case the second solver is ICCG.

⁷⁵ That is, corresponding warnings will be suppressed (ie, *ierr=0* upon return). Note that, by default, any warning will enforce a fresh setup to be done ($\rightarrow$ *ignore_warn*).

The following table contains some further control parameters:

Parameter	Explanation	Default
Output		
cntrl_logfile	Logical. Indicates whether or not a logfile shall be created which documents the behavior of the solver control. The name of the logfile is fixed, namely, " <i>cntrl_logfile.txt</i> ". Note that certain information regarding the history of the automatic control process will also be obtained by setting <i>mode_mess</i> =1 (→ Section 6.4.1).	.false.
Further optimization		
cntrl_release	Logical. Indicates whether or not memory shall be released before SAMG returns. This reduces memory between SAMG calls if solver switching is active but no setup data is to be re-used (ie, <i>iswit</i> =6).	.false.
lu_reuse	Logical. Indicates whether or not an attempt shall be made to re-use previous ILU-decompositions. Only relevant for ILU-type one-level solvers . Warning: Re-using ILU-decompositions makes only sense, if ILU is used as a preconditioner for CG, BiCGstab or GMRes!	.false.
Parameters in case of a re-start		
backup	Logical. Indicates whether or not the 1st guess should be saved upon entry to SAMG. This is relevant only if - for any reason – SAMG needs to be restarted: If <i>backup</i> = <i>true.</i> , SAMG can and will re-start with the original 1st guess. Otherwise, SAMG necessarily has to perform a re-start based on some different approximation.	.true.
divergence	Double. Relevant only if <i>backup</i> = <i>false.</i> and if SAMG needs to be restarted because of divergence: Should the average increase of the residual per iteration be greater than <i>divergence</i> , SAMG will be re-started with the zero function. Otherwise, ie, in case of "weak" divergence, SAMG uses the current approximation to re-start with.	1.1d0

7.5.6 Outline of the solver switching

When both solvers are compared, we need a clear definition which of them is regarded "better". To be specific, the following cases can occur:

- If both methods reach the *eps* criterion, the faster method is regarded "better".
- If only one method reaches the *eps* criterion, this one is regarded "better", independent of the time required.
- If both methods converge but neither of them reaches the *eps* criterion within the maximum number of cycles specified (→ *ncycle*, → *ncycle_2nd*), the decision is as follows:
 - If the final residuals of the solvers differ by a small amount only (→ *cntrl_res_diff*), the faster solver will be regarded "better".
 - Otherwise the solver with the smaller residual will be regarded "better".
 In either case a warning will be given⁷⁶ (→ *cntrl_logfile*) but the solution process continues.
- If one method diverges, the other is regarded "better". A warning will be given if the *eps* criterion is not matched⁷⁶ (→ *cntrl_logfile*). In either case, the solution process continues.
- If both methods diverge, SAMG reports a fatal error⁷⁷ and the program should be terminated.

Based on this definition of "better", the solver switching proceeds roughly as follows:

⁷⁶ Error exit code is 290 (→ Section 9)

⁷⁷ Error exit code is 291 (→ Section 9)

1. The very first linear system is solved by either AMG or the secondary solver, depending on the parameter *min_multi_size*. Unless this solution process fails (in which case the alternative solver is tested on the same linear system), the second linear system will be treated by the alternative solver. The subsequent systems will be solved by the “better” solver of the two.
2. This solver is continued to be used until one of the following two things happens:
 - A “checkpoint” is reached ($\rightarrow$ *cntrl_retry_iter*).
 - The current solver failed, that is, did not reach the *eps* criterion within the maximum number of cycles specified, or even diverged.

In either case the alternative method is checked and the “better” method is used for the next linear systems. Go back to the beginning of 2.

Remarks: (1) According to the above, the control process continues even if the *eps* criterion may not strictly be satisfied. By means of the parameter *cntrl_enforce*, a higher priority can be given to the attempt to reach the *eps* criterion, independent of the maximum number of cycles specified.

(2) Assume the current solver is AMG reusing setup data from a previous step. If this solver will fail, it will first be re-started with a fresh setup. Only if this also fails, the secondary solver will be tested.



Attention: Many decisions (in the switching process as well as during the steering of the reuse of setup data) are based on the question, what is the “fastest” approach. In the current version of the control process, such decisions are based on **computing time**. Although this is indeed what counts, it has the drawback that the overall method to some extent depends on the properties and the workload of the used computer. In a next release there will be alternative possibilities to measure “speed”. These will not be optimal since they measure speed only approximately, but they will be independent of the hardware and workload.



The following table summarizes the main parameters referring to the solver switching:

Parameter	Explanation	Default
Parameters referring to the solver switching		
min_multi_size	Integer. Minimum number of variables required to start the solver switching process with AMG. For smaller problems, the second solver ist used to start with.	25000
cntrl_steady	Integer. Setting this to 1 indicates that the current problem is steady-state. In this case, independent of <i>min_multi_size</i> , AMG will be used to start with. (This is actually equivalent to setting <i>min_multi_size</i> =0.)	0
cntrl_retry_iter	Integer. Sets certain “checkpoints”. More precisely, defines the max. number of calls to the same solver before the respective other solver is checked. If the previously used solver continues to be better, <i>cntrl_retry_iter</i> will be linearly increased ⁷⁸ . Otherwise the value of <i>cntrl_retry_iter</i> will be reset to its original value.	10
cntrl_res_diff	Integer. If both solvers have been tested for the same problem, and none of them satisfies the <i>eps</i> criterion, a warning is given ($\rightarrow$ <i>cntrl_logfile</i>), and <i>cntrl_res_diff</i> defines with which method to continue: Should the final residuals of the two solvers differ by at most <i>cntrl_res_diff</i> orders of magnitude, the faster solver will be taken, otherwise that with the smaller residual.	2
cntrl_enforce	Integer. Priority which is given to reach the <i>eps</i> -criterion, ignoring the maximum number of cycles specified. Values are 0, 1 or 2 with the default being 1.	1

⁷⁸ That is, *cntrl_retry_iter* $\rightarrow$ $2*cntrl_retry_iter \rightarrow 3*cntrl_retry_iter \rightarrow 4*cntrl_retry_iter \rightarrow \dots$

7.5.7 Exemplary templates

In the following, we want to outline the use of the solver control in some typical situations. In all examples we assume that many related linear systems of equations (one may think of a time-dependent application) are to be solved one after the other. We will display only skeletons which, however, should be sufficient for demonstration.

Scenario 1. Both primary and secondary solver control are to be activated starting with the second linear system and ending with the n -th system. The primary control is supposed to reuse *full* setups as much as possible. As second solver, the default, that is *ILU(0)-preconditioned CG*, is supposed to be used. Finally, a logfile is requested.

.....	
iswtch=51	
call samg(..<problem 1>..)	normal call to SAMG
.....	
iswtch=81	start primary control re-using <i>full</i> setups
call samg_set_cntrl_method(0)	activate <i>both</i> solver controls
call samg_set_cntrl_logfile(1)	create logfile
call samg(..<problem 2>..)	both solver controls are active
.....	
call samg(..<problem n>..)	solver controls are still active
.....	
iswtch=51	
call samg(..<problem n+1>..)	back to normal
.....	

Scenario 2. Only the primary solver control is supposed to be activated, starting with the second linear system and ending with the n -th system. The primary control is supposed to reuse *full* setups as much as possible. For some reason, the m -th linear system ($2 < m < n$) will be substantially different from the previous ones (for instance, due to a drastic change in the boundary conditions). Except for this, however, no major change in the time-dependent process is expected (from the point of view of matrix properties).

.....	
iswtch=51	
call samg(..<problem 1>..)	normal call to SAMG
.....	
iswtch=81	start primary control re-using <i>full</i> setups
call samg(..<problem 2>..)	only primary solver control is active
.....	
call samg(..<problem m-1>..)	primary solver control is still active
.....	
iswtch=61	indicates a new setup shall be done
call samg(..<problem m>..)	fresh setup is done
.....	
iswtch=81	continue primary solver control
call samg(..<problem m+1>..)	primary solver control continues
.....	
call samg(..<problem n>..)	primary solver control is still active
.....	
iswtch=51	
call samg(..<problem n+1>..)	back to normal
.....	

Scenario 3. Same as before except that the time-dependent process after the m -th step is expected to be significantly different (from the point of view of matrix properties).

..... iswtch=51 call samg(..<problem 1>..)	normal call to SAMG
iswtch=81 call samg(..<problem 2>..)	start primary control re-using <i>full</i> setups only primary solver control is active
call samg(..<problem m-1>..)	primary solver control is still active
call samg_refresh(ierr) call samg(..<problem m>..)	launches new solver control from scratch fresh setup is done, new control starts
call samg(..<problem m+1>..)	primary solver control is still active
call samg(..<problem n>..)	primary solver control is still active
iswtch=51 call samg(..<problem n+1>..)	back to normal

Note that the above special treatment of the m -th system is **not** absolutely necessary. Should the m -th system really behave substantially different, this will be detected by SAMG and a fresh setup will be done automatically. Telling SAMG explicitly to perform a fresh setup is just cheaper.

Scenario 4. The solver control is supposed to switch between AMG and the secondary solver whichever is more efficient. Since we know that the number of variables changes from one time step to the next, no attempt shall be made to reuse setup data. The solver control is to be activated starting with the second linear system and ending with the n -th system. As second solver, *ILUT-preconditioned BiCGstab* is supposed to be used.

..... iswtch=51 call samg(..<problem 1>..)	normal call to SAMG
iswtch=61 call samg_set_cntrl_release(1) ⁷⁹ call samg_set_nsolve_2nd(22) call samg_set_ncyc_2nd(120100) call samg_set_cntrl_method(0) call samg(..<problem 2>..)	enforces always full setups release memory before SAMG returns ILUT selected as secondary solver BiCGstab, max. 100 iterations activate <i>both</i> solver controls solver switching is active
call samg(..<problem n>..)	solver switching is still active
iswtch=51 call samg(..<problem n+1>..)	back to normal

⁷⁹ Should enough memory be available, this call is not absolutely necessary. It just frees memory in between SAMG calls (→ *cntrl_release* in Section 7.5.5).

7.6 One-level solvers

There are two situations when SAMG will “degenerate” to a **one level-solver**:

1. **Intentional case:** The user deliberately forces SAMG to not create any coarser level simply by setting the hidden parameter **levelx=1** (→ Section 6.2.4).
2. **Unintentional case:** For any of the other reasons listed in Section 6.2.4, SAMG decides that a coarsening does not make sense and stays on one level. Typical situations for this to happen:
 - a. the given problem is simply too small (→ *nptmn* in Section 6.2.4),
 - b. the coarsening is extremely inefficient (too slow),
 - c. the given matrix is diagonally dominant (no hierarchy needed).

In either case, by default, SAMG degenerates to the **currently selected smoother** as one-level solver, used as preconditioner for the currently selected accelerator (→ *ncgrad* in Section 5.5). Clearly, there are cases where this is reasonable (eg, for really small or diagonally dominant problems). In other cases, however, the convergence of the smoother may become too slow to be a reasonable “**fallback solver**”. This is true, in particular, if the current smoother is just plain Gauss-Seidel.

The above default behavior can be changed by means of the hidden parameter, **clsolver_finest**. This parameter is used to select an alternative fallback solver (out of the list of coarsest-level (CL) solvers **#1-#11** given in Section 6.3.4.1) which, should SAMG end up with just one level, shall be used instead of the currently selected smoother. The list contains direct and iterative solvers. In case of iterative solvers, there is the option to set a maximum number of iterations and/or to specify whether or not the accelerator (as given by the *ncyc*-parameter) should be used.

Parameter	Explanation		Default	
Alternative “fallback solver” to be used in one-level cases				
clsolver_finest	Integer. Unless <i>clsolver_finest</i> =0, this parameter defines a specific CL solver to be used for taking care of one-level problems. The purpose of <i>clsolver_finest</i> is to change SAMG’s default behavior (s. above).		0	
	[1:1]	Unless <i>clsolver_finest</i> =0, the first digit has to be =1.		
	[2:3]	Selects any of the CL solvers #1-#11 listed in Section 6.3.4.1		
		=0		Use CL solver according to current <i>nrc</i> setting. Should this be a <i>direct</i> solver and if <i>nnu</i> > <i>nptmax</i> , switch to the emergency solver as defined by <i>nrc_emergency</i> .
		>0		Any of the <i>nrc</i> -settings, 01-11. The selected solver will be used independently of the size of <i>nnu</i> . ⁸⁰
	[4:4]	Defines whether or not the accelerator as defined via <i>ncyc</i> ⁸¹ shall be used.		
		0		For solver #1 accelerator is to be used, otherwise not. ⁸²
		1		Accelerator is not to be used.
		2		Accelerator is to be used.
	[5:]	Maximum number of iterations		
=0		For solvers #6-#11 : one iteration. For solvers #1-#5 : number of iterations according to the setting of <i>ncyc</i> . ⁸³		
>0		Maximum number of iterations to be used.		

⁸⁰ That is, it is the responsibility of the user to make sure that the selected solver is reasonable. In particular, if a *direct* solver has been selected, it has to be feasible in terms of memory and computing time.

⁸¹ That is, according to the sub-parameters *ncgrad* and *nkdim*.

⁸² Recall that #2-#5 have their own accelerators, #6-#11 are direct solvers (→ Section 6.3.4.1).

⁸³ That is, according to the sub-parameter *ncycle*.

Remarks

- Note that the iterative solvers **#1-#5** have their **own stopping criteria**, namely, *conv_stop* and *itmax_conv* (→ Section 6.3.4.2). While these define an “inner” iteration, *clsolver_finetest*[5:] specifies the maximum number of “outer” iterations. This “nested” iteration terminates if either the maximum number of (outer) iterations is reached, or if the final residual drops below *eps* (→ Section 5.2).
- Note that there are more parameters which might be adjusted in defining the CL solver to be used (→ Section 6.3.4). For instance, *lfi_cl* and *droptol_cl* for the ILUT-based solvers.
- Note that the iterative solvers **#2-#5** have their **own accelerator**. Hence they should normally not be combined with the additional “outer” accelerator as specified as part of *ncyc*. This is different for solver **#1** which has no accelerator attached.
- Solver **#6-#11** (ignoring the dummy **#9**) are **direct solvers** and normally should be applied only once and with any accelerator turned off.

In the following we give a selection of examples regarding typical choices of *clsolver_finetest*. We distinguish the **intentional** and **unintentional** use of one-level solvers.

7.6.1 Intentional use of one-level solvers

We call the use of a one-level solver “intentional” if a user has deliberately set the hidden parameter *levelx*=1. Depending on whether or not *clsolver_finetest*=0, the user then has access to all available **smoothers** and **CL-solvers**, respectively. Below, we look at these two cases separately.

Remark: It has already been stated earlier (→ end of Section 7.5.2) that *iswit*=2-3 does only apply in case of true multilevel methods: for one-level methods there exists no hierarchy to be re-used. However, *iswit*=1 can be used to indicate that the matrix hasn’t changed and that an (existing) **ILU** or **LU decomposition** is to be re-used. Typical case: A direct method is to be used to solve a series of problems with just different right hand sides. This can be done by solving the first linear system with *iswit*=4 (cleaning up and saving the next decomposition) and the following ones with *iswit*=1. Formally, one can even employ different solvers and re-use several decompositions. Examples and some “dirty trick” are given in Section 7.6.1.3.

**7.6.1.1 Using any of the smoothers as one-level solver (*clsolver_finetest*=0)**

To select any of the smoothers as a one-level method, simply set the hidden parameters *levelx*=1 and *clsolver_finetest*=0. This makes each AMG “cycle” degenerate to just the smoothing on the finest level, used as preconditioner if *ncgrad*=1-4. More precisely, each such cycle consists of a complete smoothing process as defined by *nxtyp* (→ *nsolve* in Section 5.4.2) as well as both *nrd* and *nru* (→ Section 6.3.1). Some examples:

examples for one-level solvers	general settings: <i>levelx</i> =1 and <i>clsolver_finetest</i> =0	remarks
ILU(0) / CG	<i>nsolve</i> =11; <i>ncyc</i> =110500;	Only for sym. problems; restart of CG every 30 iterations (→ <i>nkdim</i> in Section 5.5)
Block ILU(0) / GMRes	<i>nsolve</i> =36; <i>ncyc</i> =139500	<i>nsys</i> >1 and <i>ip</i> vector has to be provided; Krylov dimension = 20
ILUT / BiCGstab	<i>nsolve</i> =12; <i>ncyc</i> =128500;	Restart of BiCGstab every 9 iterations

Note that a complete smoothing step – here used as preconditioner – is actually the combination of pre- and post-smoothing as defined by the current setting of *nrd* and *nru*, respectively. That is, if **single** smoothing steps are required per cycle, the setting of *nrd* and *nru* will have to be adjusted, for instance, by setting *nrd*=1 and *nru*=0. In this context, recall the following:

Attention: Recall that – for symmetric problems and if conjugate gradient is used as an accelerator – neither *nrd* nor *nru* is allowed to be zero in the context of a true multilevel cycle. Otherwise, SAMG will automatically re-define *nrd* and/or *nru* to ensure symmetry of the cycle. In the context of one-level methods any automatic re-setting of *nrd* and/or *nru* should be **de-activated** by explicitly setting ***force_accel=.true.*** (→ Section 6.3.3). **Note, however, that a user then has to take care of symmetry requirements himself (cf. the below examples)!**



The following examples give a further illustration of settings which take *nrd* and *nru* into account:

examples for one-level solvers	general settings: <i>levelx=1</i> , <i>clsolver_finetest=0</i> , and <i>force_accel=.true.</i>	remarks
ILU(0) / CG	<i>nsolve=11; ncyc=110500;</i> <i>nrd=1; nru=0</i>	Only for sym. problems (<i>isym=1!</i>); restart of CG every 30 iterations (→ <i>nkdim</i> in Section 5.5)
6x ILU(0) / CG ⁸⁴	<i>nsolve=11; ncyc=110500;</i> <i>nrd=2; nru=4</i>	same as above
6x ILU(0) / CG	<i>nsolve=11; ncyc=110500;</i> <i>nrd=1; nru=1;</i> <i>inner_accel=3</i>	same as above
Block ILU(0) / GMRes	<i>nsolve=36; ncyc=138500;</i> <i>nrd=1; nru=0</i>	<i>nsys>1</i> and <i>ip</i> vector provided; dimension of Krylov space = 9
ILUT / BiCGstab	<i>nsolve=12; ncyc=129500;</i> <i>nrd=1; nru=0; lfil_smo=12;</i> <i>droptol_smo=0.001</i>	Restart of BiCGstab every 20 iterations (→ <i>nkdim</i> in Section 5.5)
4x Jacobi / CG	<i>nsolve=10; ncyc=110500;</i> <i>nrd=47; nru=0;</i> <i>omega_jacobi=1.0</i> ⁸⁵	Only for sym. problems (<i>isym=1!</i>); restart of CG every 30 steps
6x Gauss-Seidel / GMRes	<i>nsolve=10; ncyc=139500;</i> <i>nrd=231; nru=23131</i>	Dimension of Krylov space = 20
Sym. Gauss-Seidel / CG	<i>nsolve=10; ncyc=110500;</i> <i>nrd=12; nru=-12</i>	Only for sym. problems (<i>isym=1!</i>); restart of CG every 30 steps

7.6.1.2 Using any of the CL solvers as one-level solver (*clsolver_finetest*≠0)

To select any of the CL-solvers **#1-#11** as a one-level method (→ Section 6.3.4.1), simply set the hidden parameters *levelx=1* and select the requested CL-solver by *clsolver_finetest*. This replaces the AMG “cycle” by the selected CL-solver.

- **Iterative solvers #1-#5:** The resulting (iterative) methods can also be selected by different means (→ Sections 7.6.1.1 and 7.7.1).
- **Direct solvers #6, #8:** These methods are suitable only for fairly small problems.
- **Direct solvers #7:** This method is not recommended for problems larger than 2000 variables, say.
- **Direct solvers #10:** This corresponds to a direct *dense* solver which is not practicable in a *serial* environment. However, since the analog of this solver in SAMGp is fully MPI-parallel, it is of *high* interest in parallel MPI-applications.
- **Direct solvers #11:** This corresponds to the direct *sparse pardiso* solver (only available, if Intel’s MKL has been linked). This solver can efficiently be applied even for relatively large problems.

⁸⁴ CG pre-conditioned by 6 ILU(0) steps

⁸⁵ Note that, by default, SAMG assumes that Jacobi’s method is used as a smoother. As such, however, it needs an underrelaxation parameter *omega_jacobi* the default value of which is 0.5 (→ Section 6.3.1). This is no longer meaningful if used as a solver.

examples for one-level solvers	general settings: <i>levelx=1</i>	remarks
pardiso	<i>clsolver_finetest=111</i>	pardiso, independent of <i>nnu</i> .
2x pardiso	<i>clsolver_finetest=11122</i>	pardiso with one additional post iteration step using the currently selected accelerator
dense Gauss	<i>clsolver_finetest=110</i>	Remark: In SAMGp, this solver is MPI-parallel
ILUT / BiCGstab	<i>clsolver_finetest=103030</i>	At most 30 iterations each of which corresponds to one complete ILUT/BiCGstab CL solver process (as defined by <i>lfil_cl</i> , <i>droptol_cl</i> , <i>conv_stop</i> , <i>itmax_conv</i>)
ILUT / BiCGstab	<i>clsolver_finetest=103</i>	Same, except that the max. number of iterations is as defined by <i>ncyc</i> .

7.6.1.3 Examples

Skeleton demonstrating how to use **pardiso** to solve the same system with **multiple right hand sides**:

```

.....
call samg_set_levelx(1)           ! intentional one-level method
call samg_set_clsolver_finetest(111) ! select pardiso
iswtch=41                         ! cleanup & save decomposition
call samg(...,ia,ja,a,f1,u,...)    ! use pardiso with RHS f1
iswtch=11                         ! re-use decomposition
call samg(...,ia,ja,a,f2,u,...)    ! use pardiso with RHS f2
call samg(...,ia,ja,a,f3,u,...)    ! use pardiso with RHS f3
.....

```

In the following - admittedly somewhat artificial - example, we demonstrate how to switch between different solvers, **re-using all their decompositions** (should they have one). That is, whenever we switch methods below, the following happens:

- if a solver is called for the first time, its decomposition will be computed and kept in memory together with that of the previous solver;
- if we call a solver again, or switch back to it from a different one, the respective decomposition will be re-used.

```

.....
call samg_set_levelx(1)           ! intentional one-level method
call samg_set_clsolver_finetest(111) ! select pardiso
iswtch=41                         ! cleanup & save decomposition(s)
call samg(...,ia,ja,a,f1,u,...)    ! use pardiso with RHS f1
iswtch=11                         ! re-use decompositions.....
call samg(...,ia,ja,a,f2,u,...)    ! use pardiso with RHS f2
call samg_set_clsolver_finetest(110) ! switch to full Gauss
call samg(...,ia,ja,a,f3,u,...)    ! use full Gauss with RHS f3
call samg(...,ia,ja,a,f4,u,...)    ! use full Gauss with RHS f4
call samg_set_clsolver_finetest(103) ! switch to ILUT
call samg(...,ia,ja,a,f5,u,...)    ! use ILUT with RHS f5
call samg(...,ia,ja,a,f6,u,...)    ! use ILUT with RHS f6
call samg_set_clsolver_finetest(110) ! switch back to full Gauss
call samg(...,ia,ja,a,f7,u,...)    ! use full Gauss with RHS f7
call samg(...,ia,ja,a,f8,u,...)    ! use full Gauss with RHS f8
call samg_set_clsolver_finetest(110) ! switch back to pardiso
call samg(...,ia,ja,a,f9,u,...)    ! use pardiso with RHS f9
call samg(...,ia,ja,a,f0,u,...)    ! use pardiso with RHS f0
iswtch=41                         ! cleanup
call samg_set_levelx(25)          ! switch back to multilevel
call samg_set_clsolver_finetest(0) ! no special solver treatment
.....

```

Remark on ILU-type methods: Switching between different ILU-type methods (ILU(0) or ILUT) or between ILUT with different parameters (*lfil* and *droptol*) will always cause a **fresh setup** to be done. This is for internal technical reasons.



Dirty trick

Reusing a decomposition by setting *iswit*=1 – either of a direct solver or any of the ILU-type solvers – requires the new matrix to be identical to the previous one. At least, this is according to the definition of the parameter *iswit* (→ Section 5.6.1). However, it is not checked by SAMG whether or not all entries really are the same. Hence, a modification of matrix entries – as long as the matrix *structure* remains unchanged - will not be detected.

This can be taken advantage of by reusing a decomposition in spite of the fact that the matrix entries have changed. Clearly, this will turn a direct solve into an iterative process which makes only sense if used as a preconditioner. However, this may make sense in case matrices change only slowly:

```
.....
call samg_set_levelx(1)           ! intentional one-level method
call samg_set_clsolver_finetest(111) ! select pardiso
iswtch=41                         ! cleanup & save decomposition
call samg(...,ia,ja,a,f,u,...)    ! apply pardiso
iswtch=11                         ! re-use decomposition

! Read a new matrix ia,ja,a (structurally identical!)

call samg_set_clsolver_finetest(111210) ! initiate iterative process
call samg(...,ia,ja,a,f,u,...)          ! apply pardiso to new system
.....
```

7.6.2 Unintentional use of one-level solvers

There are various situations where SAMG will employ a one-level method simply because it turns out during setup that the construction of a hierarchy does not make sense (→ Section 6.2.4). In such cases, we call the use of a one-level method “unintentional”. **Note:** While the parameter *clsolver_finetest* behaves **exactly the same way** as in the intentional case, reasonable settings may be different in these two cases. Typical situations are:

- SAMG may end up with only one level since the given problem is simply *too small* (→ *nptmn* in Section 6.2.4). In this case, for instance, an efficient **direct solver** (such as pardiso) makes sense.
- SAMG may end up with one-level only, because SAMG decides that coarsening is *unacceptably slow*. In this case, AMG seems not suitable and should be replaced by an **iterative solver** which is expected to be robust and reasonably efficient.
- SAMG may end up with one-level only, because SAMG decides that the given matrix is *diagonally dominant* and the use of a hierarchy may be overkill. In this case any **iterative solver** should do.

As mentioned before, if *clsolver_finetest* is not set, SAMG’s solution process will degenerate to the currently selected smoother, used as pre-conditioner if *ncgrad*=1-4. Otherwise, *clsolver_finetest* defines the solution process. Examples are similar as in the previous section except that *clsolver_finetest*=1 is here of particular relevance:

clsolver_finetest	resulting one-level solver
clsolver_finetest=1	CL solver according to current <i>nrc</i> setting. If this is a <i>direct</i> solver and if <i>nnu</i> > <i>nptmax</i> , switch to the solver defined by <i>nrc_emergency</i> .
clsolver_finetest=111	pardiso, independent of <i>nnu</i>
clsolver_finetest=11122	pardiso with one post iteration using the accelerator as defined by <i>ncyc</i> .
clsolver_finetest=103030	At most 30 iterations each of which corresponds to a complete ILUT/ BiCGstab CL solver process ($\rightarrow$ <i>lfil_cl</i> , <i>droptol_cl</i> , <i>conv_stop</i> , <i>itmax_conv</i>).
clsolver_finetest=103	Same, except that the max. number of iterations is as defined by <i>ncyc</i> .

7.7 More one-level methods

There are few very special one-level solvers: by setting *igam*=5-6 ($\rightarrow$ *ncyc* in Section 5.5) all multigrid-specific parameters are ignored and immediately one of the below classical **one-level method** is used. In particular, there is then no need to set *levelx*=1, and *nrd*, *nru*, *inner_accel* are ignored (that is, only one pre-conditioning step is done per iteration). Also *clsolver_finetest* will be ignored.

7.7.1 Diagonally pre-conditioned CG, GMRes or BiCGstab

Set *igam*=5 to use plain diagonal preconditioning for the accelerator currently selected by *ncgrad* and *nkdim* ($\rightarrow$ *ncyc* in Section 5.5).

Remark: For symmetric problems, it is sufficient to pass only the *lower triangular part* of the matrix to SAMG. Indicate this to SAMG by setting *isym*=3 (instead of *isym*=1, $\rightarrow$ *matrix* in Section 5.1).



examples for one-level solvers	general settings: <i>none</i>	remarks
Diag. preconditioned CG	<i>ncyc</i> =510500; <i>matrix</i> =12	Symmetric matrix; full matrix stored; restart of CG every 30 iterations
Diag. preconditioned CG	<i>ncyc</i> =510500; <i>matrix</i> =32	Symmetric matrix; lower triangular part stored; restart of CG every 30 iterations
Diag. preconditioned GMRes	<i>ncyc</i> =539500; <i>matrix</i> =32	Symmetric matrix; lower triangular part stored; dimension of Krylov space = 20
Diag. preconditioned BiCGstab	<i>ncyc</i> =528500; <i>matrix</i> =22	Non-symmetric matrix; full matrix stored; restart of BiCGstab every 9 iterations

7.7.2 ICCG for symmetric matrices

Set *igam*=6 to use ICCG for symmetric matrices ($\rightarrow$ *ncyc* in Section 5.5). In this case, of course, *ncgrad* is also ignored.

Remark: ICCG requires that only the lower triangular part of the matrix is passed to SAMG. Accordingly, *isym*=3 is required on input (instead of *isym*=1, $\rightarrow$ *matrix* in Section 5.1)! Calling SAMG with *igam*=6 and *isym*≠3 will cause a fatal error!



examples for one-level solvers	general settings: <i>none</i>	remarks
ICCG for sym. problems	<i>ncyc</i> =600500; <i>matrix</i> =32	Restart of CG every 30 iterations
ICCG for sym. problems	<i>ncyc</i> =608500; <i>matrix</i> =32	Restart of CG every 9 iterations

7.8 Controlling special output (involving files)

This section describes special file reading/writing features of SAMG. Although these features are essentially reserved for debugging they are also available to an external user. The following hidden parameters are used in the subsequent sections:

Parameter	Explanation	Default
Parameters for file reading/writing features		
ioform	Character*1. Format in writing files:	"b"
	"f" formatted writing	
	"u" unformatted writing	
	"b" binary writing	
filnam	Character. File root name which SAMG tries to find for reading.	"temp"
iodump	Integer. I/O unit to be used if SAMG dumps matrices to disk.	32
filnam_dump	Character. Root name of files to be used for dumping matrices to disk.	"level"
iomovie	Integer. I/O unit to be used if SAMG writes a sequence of iterations ("frames") to a file.	34
ncframes	Integer. Number of 1st frame to be written to file (up to the last one). "-1" means "no writing".	-1
iogrid	Integer. I/O unit to be used if SAMG writes grid info (coarsening pattern) to a file.	33
lastgrid	Integer. Last grid for which coarsening pattern is to be written to a file. "0" means "no writing".	0

7.8.1 Specifying file format

Reading or writing can be *formatted*, *unformatted* or *binary*. In case of **reading**, SAMG knows the format by looking into a special "format" file (suffix ".frm", →Section 7.8.2). In case of **writing** (→Sections 7.8.3-7.8.5), the hidden char*1 variable **ioform** defines the format SAMG is supposed to use. That is, with the char*1 variable *ch* being 'f', 'u' or 'b', issue

```
call samg_set_ioform(ch,1)
```

to specify the requested format, the default being formatted. Note that SAMG writes vector data to a file as follows:

- if formatted:

```
do i=1,nnu; write(io,*) vec(i); enddo
```
- otherwise:

```
write(io) (vec(i),i=1,nnu)
```

7.8.2 Displaying error histories

By default (assuming *iout*=2, → Section 5.8.2), SAMG displays the history of **residuals** from one iteration step to the next. For test purposes, it is sometimes instructive to additionally display the history of **errors**. This requires that the solution vector is known and contained in a file named '**mysolution.sol**', say⁸⁶. In addition, there has to exist a formatted file '**mysolution.frm**' with just one line containing 'f 4', 'u 4' or 'b 4', depending on whether '**mysolution.sol**' is formatted, unformatted or binary, respectively⁸⁷. Note that SAMG will read the solution file as follows

- if formatted:

```
do i=1,nnu; read(io,*) sol(i); enddo
```
- otherwise:

```
read(io) (sol(i),i=1,nnu)
```

⁸⁶ The file's name is arbitrary but the special suffix '.sol' is obligatory.

⁸⁷ The number "4" is used internally as a version control parameter.

To now make SAMG display histories of errors, just make the solution's filename (via the hidden parameter *filnam*) known to SAMG, in this example by issuing the call

```
call samg_set_filnam('mysolution',10),
```

and give the parameter *iswtch* a **negative sign**.

Remarks:

- SAMG will read the above files via the I/O unit *ioscratch*.
- If no solution filename is specified, the default name "temp" is assumed.
- If the solution file does not exist, or if *iout*<2, no history of errors will be displayed.
- Error norms will be computed w.r.t. the norm selected by the parameter *norm_typ*. The special setting *norm_typ*=3 corresponds to the **energy norm**. (If, according to the setting of parameter *isym*, the matrix *A* is not symmetric, the energy norm is automatically replaced by the *L2*-norm.)
- In this special test mode, the setting of *nth_res_scratch* is ignored, that is, the true residuum is computed in every iterations tep.

7.8.3 Dumping matrices to disk

The first digit of *idump* (→ Section 5.8.3) can be used to make SAMG dump matrices to disk⁸⁸, either formatted, unformatted or binary, depending on the setting of *ioform*:

Parameter		Explanation	
Dumping matrices to disk			
idump	[1:1], idmp	1	Standard print output, no matrix dump .
		2-6	Write matrices to disk: level 2 up to level <i>idmp</i> .
		7	Write matrices to disk: level 2 up to the coarsest level.
		8	Write finest-level matrix to disk (incl. right hand side etc.).
		9	Write all matrices to disk.

If *idmp*>1, the matrices as described above are dumped to files using the root name *filnam_dump*, the default name being 'level'. In addition, the level number will be added to the file's root name. Files with the standard SAMG suffixes '.frm', '.amg' and - in case of *coupled* systems - '.iu', '.ip' will be created. In case of the finest level, the right hand side file (suffix '.rhs') is created as well.

Remarks:

- If coordinates are available (→ Section 7.3), a file with suffix '.coo' is created in addition, containing the list of coordinates.
- In writing these files, the I/O unit *iodump* is used.

7.8.4 Writing a sequence of approximations (frames) to disk

In order to make SAMG write a sequence of iterations ('frames') to a file, one after the other, starting with the *k*-th one ($k \geq 0$, where $k = 0$ corresponds to the *first guess*) up to the final one, proceed as follows.

The calling program has to open the file⁸⁹ (formatted, unformatted or binary, depending on the setting of *ioform*), linked to some I/O unit *io*, say. Then the hidden parameters *ncframes* (number of first iteration to be written) and *iomovie* (I/O unit used by SAMG for writing) have to be set:

⁸⁸ Regarding SAMG's file format, see the corresponding separate SAMG document.

⁸⁹ This **cannot** be done in case of shared libraries (suffix .DLL or .so.).

`call samg_set_ncframes(k)` and `call samg_set_iomovie(io)`.

Remarks:

- If *ncframes* is negative (the default), nothing will be written to disk.
- The second call can be skipped, if *io* equals the default value of *iomovie*.

7.8.5 Writing coarsening patterns to disk

For some given $L \geq 1$, SAMG can write the **coarsening pattern** of levels $1, \dots, L$ to a file. More precisely, the coarsening pattern is a double (!) vector, *color*(1:*nnu*) say, defined as follows:

- *color*(*i*)=*k*: if $k \leq L$ is the highest level for which variable *i* is still "alive";
- *color*(*i*)=*L*+1: for all variables which are alive *beyond* level *L*.

In order to make SAMG write the vector *color* to a file, one has to proceed as follows:

The calling program has to open the file⁸⁹ (formatted, unformatted or binary, depending on the setting of *ioform*), linked to some I/O unit *io*, say. Then the hidden parameters *lastgrid* (=L from above) and *iogrid* (I/O unit used by SAMG for writing) have to be set:

`call samg_set_lastgrid(L)` and `call samg_set_iogrid(io)`.

Remarks:

- If *lastgrid* is non-positive (the default), nothing will be written to disk.
- The second call can be skipped, if *io* equals the default value of *iogrid*.

7.9 Multiple instances of SAMG

If not just a single linear system is to be solved but rather a sequence of linear systems, a substantial part of the total solution time can often be saved by re-using setups from previous systems. A typical scenario for such a situation arises in time-stepping methods. Unfortunately, a corresponding re-use of setups is no longer possible, if two or more *different types* of linear systems are to be solved in each time step. However, for such situations *multiple instances* of SAMG can be employed so that each instance is "responsible" for one of these different types of linear systems. This way, for different sequences of linear problems the respective setup data can be remembered and re-used if meaningful.

Another application of multiple SAMG instances arises if each thread of a multicore application wants to use SAMG individually. (Remember that SAMG is not thread-safe, → Section 7.1.1.)

Remark: Provided a *source license* is available, up to 10 additional instances of SAMG can easily be built. For more details on how to do this, we refer to the GettingStarted guide which comes with source distributions.



7.10 Changing parameters at runtime (from file)

For convenience, SAMG allows to read/modify any primary and/or hidden parameter (except for *logio* and *logfile*!) via optional input files **at run time**, before a given linear system is solved. Since input files are read from within SAMG, parameter settings found in such files will overwrite settings which have been done outside SAMG (ie, via *set*-routines → Section 6.1).

Parameter	Explanation	Default
Read parameters from an (optional) input file at run time		
allow_filnam_in	Logical. If <i>.true.</i> , SAMG attempts to read parameters from an input file at the next start. More precisely: SAMG looks for a file with name <i>filnam.in</i> (→ Section 7.8). If no such file exists, SAMG looks for “samg.in” instead. If this does not exist either, SAMG continues without any further attempt to find an input file.	.true.

SAMG expects formatted input files. Except for *logio* and *logfile*, all primary as well as hidden parameters can be changed using some kind of free format as illustrated in the following example:

```
# This is a comment line. Further rules:
# (1) Empty lines will be ignored
# (2) If a '!' is found as the 1st char of a line, reading terminates
# (3) Each line may contain up to a maximum of 100 characters

allow_filnam_in=t      # t=.true., f=.false.
filnam_dump="dumpfile" # strings are enclosed in double quotes
iswtch=5110
ifirst=1; chktol=1.0d-7; iout=21
ewt=0.02d0; nrd=231; nru=-231; nrc=7

# Remark: The order of parameter settings is arbitrary. If a
# parameter is set more than once, only the last setting counts.
! Everything beyond '!' is skipped.
```

If a **series** of linear systems is to be solved, the following should be noted:

Default behavior: By default, SAMG will try to read from an input file **only once**, namely, at the very first start of the solver. After that, *allow_filnam_in* will automatically be set to *.false.*⁹⁰ **Attention:** In this default mode, one has to watch out for parameter settings which may be in conflict with those specified inside the calling program at later steps. For instance, one must not specify *iswtch*=51 in the input file when the user's calling program explicitly sets *iswtch*=11 at later calls to SAMG.

Continue reading input files: In order to make SAMG continue reading input files at subsequent calls, one explicitly has to reset *allow_filnam_in*. This can be achieved by specifying *allow_filnam_in*=*.true.* either as part of the previous input file (cf. the above example), or via the corresponding *set*-routine inside the user's main program before the next SAMG call.

Changing input file names: By successively changing *filnam*, different data files can be selected for each solution process.

Dirty trick: Assume an input file, *samg.in*, is supplied which contains *allow_filnam_in*=*t* as one of its settings. Then this file is read continuously whenever a new call to SAMG takes place. Consequently, by manually changing the input file **while a job is running**, one may even change parameters “on the fly”.

Example: If, in solving a series of many linear systems, a user attempts to minimize numerical solver time by manually controlling the *iswit* parameter to reduce SAMG's setup time (→ *iswtch* in Section 5.6.1), he may wish at some point to compare the corresponding results with those obtained without any reuse of setup data. This can most easily be achieved by simply supplying *samg.in* containing just the line

```
allow_filnam_in=t; iswtch=51
```

⁹⁰ **Attention:** If hidden parameters are reset by calling *samg_reset_hidden* or *samg_leave* (→ Section 2.5), all hidden parameters and, in particular, *allow_filnam_in*, are reset to their initial values. Consequently, SAMG will behave as if the next call was the very first one (in particular, SAMG will again try to find an input file).



7.11 Some auxiliary subroutines

7.11.1 Optimizing SAMG's memory requirement

As already described in Section 5.7, the primary parameters *a_cmplx*, *g_cmplx*, *p_cmplx* and *w_avrge* define the dimensions used to allocate SAMG's **initial memory**. The corresponding absolute amount of initial memory is not of a fixed size, but rather increases linearly with the problem size.

Unfortunately, since SAMG is a completely dynamic program, it is impossible to precisely predict its memory requirements in advance. If the initial memory as defined by the above "complexity values" is not sufficient, SAMG will automatically extend the memory. This is fairly convenient but requires copy and re-allocation processes which may become relatively expensive. Furthermore, parallel performance on multicore machines may degrade considerably (→ Section 7.1.2).

In most applications SAMG is used to solve a whole sequence of linear problems of similar size and type. In such cases, realistic values for *a_cmplx*, *g_cmplx*, *p_cmplx* and *w_avrge* can be obtained by looking at the true memory requirement of a previous run. Corresponding information is provided by the following subroutine if called after the return from SAMG:

```
subroutine samg_get_complexities(istat, valcmplx)
```

where *istat* is an error indicator and *valcmplx* is a double precision vector of length 11 which, upon successful return, will contain the following values:

```
valcmplx( 1) = dstat_a_cmplx  
valcmplx( 2) = dstat_a_avrge  
valcmplx( 3) = dstat_b_cmplx  
valcmplx( 4) = dstat_b_avrge  
valcmplx( 5) = dstat_ib_cmplx  
valcmplx( 6) = dstat_g_cmplx  
valcmplx( 7) = dstat_p_cmplx  
valcmplx( 8) = dstat_w_avrge  
valcmplx( 9) = dstat_t_cmplx  
valcmplx(10) = dstat_c_cmplx  
valcmplx(11) = dstat_e_cmplx
```

Here, only the blue components (1,6,7,8) are relevant: *dstat_a_cmplx*, *dstat_g_cmplx*, *dstat_p_cmplx* and *dstat_w_avrge* which contain just those values of *a_cmplx*, *g_cmplx*, *p_cmplx* and *w_avrge*, respectively, which would have been sufficient in the previous call to SAMG. Hence, before calling SAMG the next time, one may use the following program skeleton:

```
call samg_get_complexities(istat, valcmplx)  
  
if (istat==0) then  
  val=valcmplx(1)*safety_factor; if (val.ge.cmplx_min) a_cmplx=val  
  val=valcmplx(6)*safety_factor; if (val.ge.cmplx_min) g_cmplx=val  
  val=valcmplx(7)*safety_factor; if (val.ge.cmplx_min) p_cmplx=val  
  val=valcmplx(8)*safety_factor; if (val.ge.cmplx_min) w_avrge=val  
else  
  ....  
endif
```

Here, the parameters *safety_factor* and *cmplx_min* should be given reasonable values slightly larger than 1.0, for instance, *safety_factor*=1.1 and *cmplx_min*=1.1.

7.11.2 Checking for disconnected subsets of variables

The following subroutine can be used to check for disconnected subsets of variables. If such subsets are found, they are analysed and new settings for *nsys*, *iu* and *iscale* are proposed. The user can now manually decide what to do. For instance, he may decide to exclude particular variables from the coarsening (based on the features described in Section 7.4). Alternatively, he may tell this subroutine to force some special conditions.

```
subroutine samg_propose_nsys_iscale_iu(
    iscale_new,iu_new,nsys_new,iout_level,io,
    imin,imax,nna,iscale,matrix,iu,nsys,ia,ja,a,
    imodified,ic_set_strategy,ierr)
```

Details on the parameters are given in the following table:

Parameter	Explanation		Default
input parameters			
imin, imax	Integer. Defines the range of variables. An external user will call this routine before SAMG is called and, hence, <i>imin</i> =1 and <i>imax</i> = <i>nnu</i> . ⁹¹		
nna, ia, ja, a iscale, matrix, iu, nsys,	These are the primary parameters describing the current matrix (→ Sections 5.1 and 5.3)		
ic_set_strategy	Integer. Defines what to do if disconnected subsets of variables exist		
	0	Do not do anything, leave actions completely up to the user.	
	1	All independent subsets of variables are forced to stay on the finest level except for the largest one.	
	>1	reserved for further actions	
iout_level	0	No messages are written.	
	1	Messages will be written to the Fortran unit <i>io</i> (see next).	
io	Integer. If <i>iout_level</i> =1, messages will be written to the Fortran unit <i>io</i> . In this case, the user is assumed to have opened that unit for writing and connected to a file. Typically this will be the SAMG logfile as defined by the parameters <i>logio</i> and <i>logfile</i> . Hint: Use the <i>set</i> - and <i>get</i> -routines to define/find out their current settings (→ Section 6.1).		
output parameters			
imodified	0	No disconnected subsets of variables found.	
	1	There exist disconnected components. The below quantities, <i>nsys_new</i> , <i>iu_new</i> and <i>iscale_new</i> return proposed new settings for <i>nsys</i> , <i>iu</i> and <i>iscale</i> , respectively.	
nsys_new	Integer. New number of unknowns proposed, $\geq nsys$.		
iu_new	Integer vector of length <i>imax</i> - <i>imin</i> +1. Modified <i>iu</i> -vector proposed.		
iscale_new	Integer vector of length <i>imax</i> - <i>imin</i> +1 ⁹² . Modified <i>iscale</i> -vector proposed.		

Remark: Currently, the functionality of this routine is rather rudimentary. It will be further enhanced to detect various types of potentially problematic situations and to propose possibilities of how to approach these problems by SAMG.



⁹¹ This more general notion has been introduced because this routine may also be used inside SAMG to check the situation on coarser levels.

⁹² Actually, a length of *nsys_new* would be sufficient. However, since *nsys_new* is not known in advance and – in the worst case – may be as big as *imax*-*imin*+1, *iscale_new* should be declared as above. Should the user for any reason know an upper bound of disconnected subsets, he may use this number to dimension *iscale_new*.

7.12 Smoothed aggregation (under construction)

If applied in the standard way, SAMG does not know anything about geometry. Hence, “accuracy” of interpolation is necessarily limited. Nevertheless, for many applications, it is sufficiently good to ensure a rapid convergence which is essentially independent of the size of the given problem.

In certain applications areas (eg, near-singular applications from **linear elasticity**), however, SAMG cannot ensure interpolation to be sufficiently accurate without knowing anything but the system matrix. Such “critical” applications are characterized by the fact that the first eigenvalues of A are very small and the corresponding eigenmodes are (smooth but) not constant. In linear elasticity, for instance, these modes are (approximations of) the **rigid body modes**.

In the context of SAMG, the main problem with such applications is that, in order to converge rapidly, the smoothest eigenmodes need to be interpolated with a sufficiently high accuracy, which actually has to be the higher the smaller the corresponding eigenvalue is. Consequently, unless the smoothest eigenmodes are essentially constant, additional information is required to ensure a sufficiently good interpolation.

One way to tackle such problems is by making the **coordinates** of grid points known to SAMG as described in Section 7.3. This means that the black-box character of SAMG is lost to some extent. However, this information is usually available anyway, and - as long as no restrictions are put on the *shape* of the domain or the kind of grid to be used - the major advantages of SAMG would still hold.

A more general approach would be to pass one or more “**test vectors**” of length nnu to SAMG which shall be used to fit the interpolation so that these vectors are interpolated exceptionally well. Usually, such vectors should be reasonable approximations to the smallest eigenmodes mentioned above. However, in principle, they can be any other vectors which should be taken care of in improving interpolation. A first version of this approach has been implemented in SAMG as a new feature, open for testing. In the current release its use and generality are still limited, but this will change in the forthcoming releases.

In principle, a fitting process as indicated above can be implemented by employing some kind of least squares fit for the interpolation operator. Unfortunately, this is difficult to realize in a simple, stable and robust way within the concept of the “classical” AMG approach which is the basis of SAMG (→ Section 7.13). The idea to define interpolation by means of additional information in form of some given vectors is easier to realize in the context of **aggregation** based coarsening strategies (**cluster coarsening**, → Section 6.2.3). If combined with a smoothing process for the interpolation, this leads to an alternative AMG approach, called “**smoothed aggregation**” (introduced by Petr Vanek around 1995 and further developed by Marian Brezina).

Remark: Smoothed aggregation within SAMG is still under development and not yet officially released. The interested user may try it but should not expect a technically mature status. The new parameters described below are temporary and may change in the near future. We would very much appreciate any feedback from our users!



7.12.1 Initializing smoothed aggregation

Smoothed aggregation is initialized by passing **$nsmooth > 0$** test vectors of length nnu to SAMG. This is done by first allocating memory by calling the following subroutine:

```
subroutine samg_smooth_vector_init(nnu,nsmooth,ierr)
```

with *nnu* being the total number of variables and *nsmooth*>0 the number of vectors to be used to define interpolation. The allocated memory is initialized to 0.0d0. Once allocated, vector components can be defined by calling

```
subroutine samg_smooth_vector_set(i,j,value,ierr)
```

which assigns the double precision *value* to the *i*-th component of the *j*-th vector. Furthermore,

```
subroutine samg_get_num_smooth_vectors(nsmooth)
```

can be called to return the number of test vectors currently known to SAMG (*nsmooth*=0, if no space has been allocated). Finally, call

```
subroutine samg_smooth_vector_init(nnu,nsmooth,ierr)
```

with *nsmooth*=0 (or *nnu*=0) to deallocate the above memory.

7.12.2 Parameters for smoothed aggregation

In this release of SAMG, smoothed aggregation aims at problems from **linear elasticity**. Once test vectors are defined as described in the previous section, smoothed aggregation is initialized and will be used when calling SAMG provided the following additional settings hold⁹³:

<i>nsmooth</i> ≥ <i>nsys</i> > 1	Only PDE systems (here assumed: linear elasticity)
<i>ncg</i> [2:3]=79 (→ Section 6.2.3.1)	On all levels: cluster coarsening type 2 , piecewise constant
<i>nsolve</i> [1:1] = 5 (→ Section 5.4.1)	Point block approach

Type and size of clusters can be controlled via the parameters *cl_order*, *max_cl_size* and *min_cl_size* (→ Section 6.2.3.3).

Important: At each point, for technical reasons, the **number of variables** times the **clustersize** has to be (at least) larger than *nsmooth*. Hence, the correct setting of *min_cl_size* is important. Since this is crucial for the overall idea of smoothed aggregation, there is an additional check to ensure a minimal cluster size (see below: parameter *maxcorr_cluster*).



The following additional parameters define the smoothed aggregation process used:

Parameter	Explanation	Default
Parameters for smoothed aggregation		
maxcorr_cluster	Integer. Number (≥0) of attempts/corrections to ensure a minimal cluster size (cf. Remark right above)	2
nsmoagg	Integer. Number (≥0) of variable-wise Jacobi-iteration steps to smoothen the interpolation operator	0
omega_smo	Double. Jacobi (under-) relaxation parameter used in smoothing the interpolation operator	0.5
iall_smo	Integer. Couplings to be used in smoothing interpolation	
	0	Use all couplings
	1	Ignore weak couplings

Remark: Currently no **truncation of interpolation** is implemented. Hence, the performance may not yet be satisfactory. This and other optimizations are subject to the forthcoming developmental steps.



⁹³ These will be generalized/removed in future releases

7.12.3 Typical scenario: linear elasticity

As mentioned further above, we mainly have applications from **linear elasticity** in mind.

Assuming *free boundaries conditions*, corresponding 2D and 3D applications have kernels of dimension 3 and 6, respectively, consisting of the **rigid body modes**. Since these modes are known (translations and rotations) it is natural to provide the rigid body modes⁹⁴ as test functions to SAMG, that is, ***nsmooth=3*** or ***nsmooth=6***.

In case of *near-singular* applications, the true eigenfunctions to the smallest eigenvalues are not known. However, the rigid body modes are reasonable approximations and can still be used as test vectors.

⁹⁴ Note that there is actually no real need to pass the (trivial) translations explicitly to SAMG. Currently, however, *all* smooth functions need to be passed as test vectors, including the translations.

7.13 Improvement of classical interpolation (under construction)

In principle, an improvement of interpolation by exploiting suitable test vectors as for smoothed aggregation (cf. previous section) can also be achieved within the classical AMG environment, for instance, by applying some kind of least squares fit. Unfortunately, this is rather difficult to realize. Although there have been several attempts to generalize the classical environment, these attempts have – up to now – not led to an alternative approach which is simple, stable and robust enough to be used easily by non-experts.

Although the corresponding parameters are still accessible, they are included here mainly for internal testing and development, not for use by an unexperienced user. In the following we do not assume the existence of test vectors but rather that **coordinates** have been provided to SAMG (→ Section 7.3).

The features outlined in this section are under development and not officially released. The unexperienced user should deactivate all related features by setting the main control parameter to zero, $np_opt=0$ (which is the default).



7.13.1 The main control parameter

The parameter np_opt selects the level of optimization and passes some basic information regarding the origin of the problem at hand.

Parameter	Explanation		Default
Optimization of interpolation			
np_opt	Integer. Unless $np_opt=0$, optimization of interpolation is activated.		0
	[1:1], nopt	Level of optimization: 1-9.	
	[2:2], ndim_space	Spatial dimension of the current problem (2 or 3). If $ndim_space = 0$, optimizations which <i>require</i> the knowledge of the dimension, will be skipped.	
	[3:], npclass	Provides information on the current type of application	
		0	
	1	<i>Linear elasticity</i> (displacements only)	

7.13.2 Details of optimization process

Currently, all of the below parameters are **reserved for internal use** in further developing SAMG. In particular, np_mod1 and np_mod2 are active only if $np_opt \neq 0$, in which case they define details on the optimization process (ie, they replace the default process selected by the sub-parameter $nopt$ of the previous section).

Parameter	Explanation			Default	
For internal use only					
np_mod1	Precise specification of optimization processes. If np_mod1=0 , the default is selected, controlled by the parameter <i>nopt</i> .				
	[1:2], nopt_strong	Definition of strong couplings (not yet installed).			0
		0	no action		
		>0	nothing implemented yet		
	[3:4], nopt_cforce	A priori forcing particular U-variables to become C-variables.			0
		0	no action		
		11	Variables which have only one strong coupling, are forced to become C-variables		
	[5:6], nopt_split	Modify (increase or decrease) the <i>measure-of-importance</i> of particular U-variables. Also: Force particular U-variables to become C-variables.			0
		0	no action		
		11	SAMG attempts to detect the boundary and increases the <i>measure-of-importance</i> for boundary variables. That is, the set of boundary variables will be split first.		
	[7:8], nopt_fit	Modify (or change totally) SAMG's interpolation weights.			0
		0	no action		
		11- -19	Various versions of LSQ fit, based on the knowledge of coordinates.		
np_mod2	Not yet used.			0	
eps_lsq	Accuracy required in LSQ-fit for interpolation.			0.5d-2	

Remark: If the optimization of interpolation fails for certain (F-) variables, SAMG needs to know whether it is allowed to (a posteriori) put such variables into C. By default, this is allowed. Note, however, that there may be quite many of such variables, causing the coarsening process to become inefficient. In such cases, one may wish to ignore critical situations (ie, do **not** create any new C-variables) and just perform the optimization "as good as possible". This is achieved by giving np_mod1 a **negative sign**.



7.14 Improving convergence by (re-)scaling

Usually, re-scaling a given discretized PDE or PDE system (by rows and/or columns) may seriously influence the properties of the underlying matrices and, through this, often **degrade** convergence properties. However, there are also other situations, where focused re-scaling may improve things. Since related questions are subject to research, SAMG contains some features to investigate the influence of (re-) scaling by rows or columns. However, these features are experimental and not meant to be used in any kind of production environment.

In the following, scaling vectors s are computed by applying a sequence of Gauss-Seidel relaxations to the homogeneous linear system - depending on the type of scaling, either A or A^T - starting with the first guess being 1.⁹⁵

Parameter	Explanation		Default
Special parameters for matrix rescaling			
modify_mat	Integer. Defines some type of matrix modification.		0
	0	No modification	
	1	Slightly increase the diagonal entries	
	2	Rescale matrix columns (by row relaxation)	
	3	Rescale matrix rows (by column relaxation)	
iter_matrix_scale	Integer. Max. number of relaxations to obtain a scaling vector		100
matrix_scale_renorm	Integer. Defines whether scaling vectors are to be normalized		0
	0	Never normalize scaling vectors. In this case, relaxation is terminated if either the max. number is reached, or if the criterion given by <i>factor_matrix_scale</i> is met.	
	1	Normalize scaling vectors after each relaxation sweep. In this case, run <i>iter_matrix_scale</i> relaxation steps.	
factor_matrix_scale	Double. Termination criterion with value between 1.0 and 1.1: Currently, the scaling iteration will terminate if $\max_i \left\{ \frac{s_{new}(i)}{s_{old}(i)} : s_{old}(i) \neq 0 \right\} < factor_matrix_scale$		1.1d0

Important: Scaling by the scaling vector s (obtained after Gauss-Seidel relaxation) is justified only if positive vectors remain positive in relaxation (for instance, in case of off-diagonally non-positive matrices). The final scaling factors are $s(i)$ if $s(i) > 0.0$, otherwise 1.0.



⁹⁵ The parameters in the table can also be alternatively defined by certain settings of the *matrix*-parameter, more precisely, by setting the digits *matrix*(3:).

8 Special Interfaces

8.1 Alternative interfaces to call SAMG

8.1.1 Simplified interfaces

In previous releases, a simplified SAMG interface had been provided which allowed reading all parameters, primary as well as hidden, from a file at run time. Since this is now also possible in connection with the standard SAMG interface (→ Section 7.10), the simplified interface is **no longer supported**.

8.1.2 An AMG1R5-like interface

The very first “classical” AMG version for scalar applications, known as AMG1R5, is still being used frequently. In previous SAMG releases, we had provided an interface to SAMG which was identical to that of AMG1R5. However, since compatibility was only obtained at the expense of a significantly reduced generality, this interface (still exists but) should **no longer be used**.

8.2 Defect correction interface

While it is often easier to efficiently solve linear systems by multigrid which are derived from *lower order* discretizations, it is usually *higher order* discretizations which are of practical importance. The so-called **defect correction (DC)** is frequently used - for example in CFD – because it allows to apply multigrid to the lower order matrix but still gives the accuracy of the higher order discretization.

Assume, we can efficiently solve a lower order problem, $Au=f$, by SAMG but what we actually want to solve is a higher order problem, $A_{dc}u=f$. Defect correction is nothing else than an iterative process which uses A as a preconditioner for the DC matrix A_{dc} :

$$u^{(n)} = u^{(n-1)} + A^{-1}(f - A_{dc}u^{(n-1)}) \quad \text{for } n = 1, 2, \dots, ncycle_{dc}$$

starting with some first approximation, $u^{(0)}$. To define this DC process we need to provide the DC matrix A_{dc} , the maximum number of DC iterations, $ncycle_{dc}$, as well as a stopping criterion given by some eps_{dc} . That is, the DC process terminates if either the maximum number of iterations is reached or if the residual⁹⁶

$$res^{(n)} = ||f - A_{dc}u^{(n)}||$$

drops below a given value, whatever is reached first. Furthermore, to accelerate the DC process, some “outer” FGMRes is usually employed with a Krylov space of dimension $nkdim_{dc}$, say.

There is a new interface for SAMG, `samg_dc`, which facilitates **defect correction (DC)**:

```
subroutine samg_dc(nnu,nna,nsys,                                &
                  ia,ja,a,f,u,iu,ndiu,ip,ndip,matrix,iscale, &
                  res_in,res_out,ncyc_done,ierr,              &
                  nsolve,ifirst,eps,ncyc,iswtch,              &
                  a_cmplx,g_cmplx,p_cmplx,w_avrge,           &
                  chktol,idump,iout,                          &
                  nna_dc,ia_dc,ja_dc,a_dc,                    &
                  nkdim_dc,eps_dc,ncycle_dc)
```

⁹⁶ As usually, the norm used is the one selected by the parameter *norm_typ* (→ *iswtch* in Section 5.6.4)

While the last seven parameters just define the DC process as indicated above, everything else refers to the (approximate) solution of

$$Av = f - A_{dc}u^{(n-1)}$$

in each iteration of the DC process except for the following self-evident modifications:

1. **ifirst**: This parameter does not select the first guess of the above lower order system (this is always $v^{(0)}=0$!) but rather that of the defect correction process, $u^{(0)}$.
2. **idump**: As expected, this initiates the output regarding SAMG's hierarchy as set up for the lower order matrix A. However, starting from the second DC iteration, further dumping will automatically be suppressed.
3. **iout**: As expected, this parameter initiates the display of a convergence history. However, this history now refers to the DC process, not to the lower order system. In addition, for each iteration step of the DC process, the corresponding residual reduction of the "inner" iterations as well as the corresponding number of cycles is also displayed. Example:

```
AMG solver information
=====

Basic Method: Defect correction approach
               using AMG as inner solver
Acceleration: Flexible GMRES (Krylov dim= 10)
Stop. criter: eps= 0.1D-05, ncycle_dc= 30

iter  0:  L2-res=0.185D+02      cfac      cfac(inner)  cycl
-----
iter  1:  L2-res=0.813D+00      0.440D-01    (0.419D-01)    2
iter  2:  L2-res=0.259D+00      0.319D+00    (0.745D-01)    3
iter  3:  L2-res=0.760D-01      0.293D+00    (0.585D-01)    2
iter  4:  L2-res=0.265D-01      0.349D+00    (0.762D-01)    3
iter  5:  L2-res=0.144D-01      0.543D+00    (0.953D-01)    2
....
iter 20:  L2-res=0.323D-04      0.763D+00    (0.752D-01)    2
iter 21:  L2-res=0.258D-04      0.801D+00    (0.826D-01)    2
iter 22:  L2-res=0.215D-04      0.832D+00    (0.790D-01)    2
iter 23:  L2-res=0.169D-04      0.785D+00    (0.706D-01)    2
-----
average reduction      : 0.546D+00
```

The new parameters as outlined further above are summarized in the following table:

Parameter	Explanation
Defect correction (DC) matrix	
nna_dc	Integer. Number of non-zero matrix entries of the DC matrix
ia_dc, ja_dc, a_dc	Compressed row storage representation of DC matrix (→ Section 4.1): Integer vectors $ia_dc(1:nna_dc+1), ja_dc(1:nna_dc)$; double vector $a_dc(1:nna_dc)$.
Defect correction process	
nkdim_dc	Integer. If >0, this is the dimension of the Krylov space to be used for the "outer" FGMRes acceleration. Otherwise, no outer acceleration is performed.
ncycle_dc	Integer. This is the maximum number of defect correction iterations.
eps_dc	Double. Stopping criterion for the DC iteration: Iteration stops if the residual of the current approximation, res , is smaller than a certain threshold, (see also the above parameter $ncycle_dc$). In detail:
	$\geq 0.0d0$ Iteration stops if $res \leq eps_dc * res0$ ($res0$ = initial residual).
	$< 0.0d0$ Iteration stops if $res \leq eps_dc $.

Remark: Although defect correction usually refers to matrices corresponding to different orders of discretization, this is not really necessary. A can actually be *any* matrix which is suitable as a preconditioner for A_{dc} . The better A approximates A_{dc} , the faster the outer convergence will be. Generally, however, the convergence of this kind of “global” defect correction is rather low. The performance can be improved by applying the defect correction process not “globally” but only on the level of smoothing. Corresponding developments are subject to future releases.



8.3 Special user interfaces (in case of source licenses)

As part of an SAMG source licence, user interfaces for special purposes are provided via certain subroutines which, except for the coarsest-level solver interface (→ Section 8.3.1), are contained in the source file `amguser.f90` (found in directory `samg-main/samg`). Some particularly useful subroutines are described in the following. For more details, we refer directly to `amguser.f90`.

8.3.1 Interface to the coarsest-level solver

A source file `amguser_solve.f90` is provided which is supposed to contain a user-defined solver for solving the coarsest-level problems. This alternative solver is selected by setting the hidden parameter `nrc=9` (→ Section 6.3.4). By default, `amguser_solve.f90` is just a dummy routine with a detailed description of how a user can incorporate his own solver. The corresponding description is given further below.

Intel's PARDISO solver. In previous releases, Intel's PARDISO solver was provided as exemplary `amguser_solve.f90`. This has been changed: While PARDISO can now be selected via `nrc=11` (→ Section 6.3.4), `amguser_solve.f90` is freely available as described above.



Below we will describe the above-mentioned user interface. In this context, note the following: In solving a problem by SAMG, the coarsest-level solver will be called (at least) once per iteration step. Since, from step to step, only the right hand side changes, the necessary setup work (eg, decomposition in case of a direct solver) needs to be done only during the very first call and should be saved for later calls. Clearly, if SAMG itself performs a new decomposition, the setup part of the coarsest-level solver has to be re-done as well. In the following subroutine interface, the integer parameter `deco_required` indicates whether or not a new “setup” is required.

```

subroutine samg_user_solve(ilo,ihl,a,ia,ja,u,f, &
                        ndu_low,nda_low,ndu_hig,nda_hig,deco_required,ierr)

!   user-defineable solver for the coarsest-level equations

!   *****
!   * note: Set NRC=9 to select this solver! *
!   *****

!   remark:
!
!       in addition to this routine, the routine "samg_user_cleanup"
!       (see below) has to be provided which is used by SAMG to clean up
!       all memory allocated by this routine, if necessary.

!   input:
!
!       ilo,ihl - the coarsest-level range of variables: ilo<=i<=ihl
!
!       the arrays a,ia,ja,u and f are dimensioned by ndu_low, nda_low, ndu_hig
!       and nda_hig as seen below. the relevant parts of these arrays are only a
!       small fraction of the total arrays. the relevant parts are described below.

```

```

!      the matrix is stored in compressed row form (Harwell-Boeing). the matrix
!      rows are contained in the input array a and can be accessed via the pointer
!      arrays ia and ja in the standard way:

!      a - array containing the rows of the matrix, one after the other, each row
!           starting with its diagonal element. the first row starts at position
!           ia(ilo) and the last row ends at position ia(ihi+1)-1 (see next).

!      ia - pointer array indicating the beginning of each row in a. that is,
!           the i-th row (ilo<=i<=ihi) is stored in a(ia(i):ia(i+1)-1).

!      ja - pointer array pointing to the column indices. that is, for each matrix
!           element a(j) with ia(ilo)<=j<=ia(ihi+1)-1, ja(j) contains the column
!           index of that element. since, within each row, the diagonal element is
!           stored first (see above), we always have ja(ia(i))=i.

!      u - vector which has to contain the solution in u(ilo:ihi) upon return
!           from this routine. on input, u(ilo:ihi) is initialised to zero (the
!           natural first approximation in case of an iterative solution).

!      f - vector containing the current right hand side in f(ilo:ihi)

!      deco_required - integer variable indicating whether a new decomposition
!                      is required (ie, whether the input matrix a has changed compared to
!                      a previous run; 0=no, 1=yes). AMG keeps track of this variable. Only
!                      when this variable =1 upon entering this routine, a new decomposition
!                      should be computed and saved for re-use in subsequent solution steps.

!      output:

!      u - u(ilo:ihi) has to contain the solution. the solution should be correct
!           to (at least) 1-2 digits. Generally, a much higher accuracy is not
!           needed since it would not improve AMG's convergence!

!      deco_required - whenever a new decomposition has been computed by this
!                      routine, deco_required should be set to 0 before returning.

!      ierr - error indicator. if an error occurs, this program should print a
!              message via samg_info and ierr should be set to a positive number in
!              case a fatal error occurred and to a negative number in case the
!              program is supposed to continue. otherwise, ierr=0 upon return.

implicit none
integer deco_required
integer ilo,ihi,ndu_low,nda_low,ndu_hig,nda_hig,ierr

double precision a(nda_low+1:nda_hig), &
                 u(ndu_low+1:ndu_hig),f(ndu_low+1:ndu_hig)
integer          ia(ndu_low+1:ndu_hig+1),ja(nda_low+1:nda_hig)

call samg_info(1,1,1,'No solver currently installed by USER (user_solve)!')
ierr=1
return

ierr=0
if (deco_required.eq.1) then

!      perform here any kind of necessary preparations which need to be done
!      only once and can be re-used if the same system is to be solved with
!      just a different right hand side. Example: Decomposition in case of
!      direct solvers.

      deco_required=0

endif

!      compute the solution vector, u, up to (at least) 1-2 digits accuracy
!      (eg, back-substitution in case of direct solver)

```

```

!   if a fatal error occurred, issue a message & set ierr to a positive value:
!       call samg_info(1,1,1,'Too bad, a fatal error occurred (user_solve)!')
!       ierr=1

!   if an non-fatal error occurred (warning), issue a message & set ierr to
!   a negative value:
!       call samg_info(-1,1,1,'...Insert your warning text here... (user_solve)!')
!       ierr=-1

!   if you just want to print a message, issue
!       call samg_info(0,1,1,'...Insert your message here... (user_solve)!')

return
end

!   =====

subroutine samg_user_cleanup

!   routine to clean up memory which has been allocated by the above
!   user-defined coarsest-level solver "samg_user_solve"

!   use ...

implicit none

!   if (allocated(...)) then
!       deallocate(...)
!   endif

return
end subroutine samg_user_cleanup

```

8.3.2 Interface to check the input matrix

The below skeleton of a subroutine provides an interface to integrate user-defined checks of the input matrix into SAMG. For demonstration, an exemplary test is implemented below. This routine will be called at the beginning of SAMG if **chk_{tol} ≥ 0.0**.

```

subroutine samg_user_check(tol,nnu,nna,iscal,nsys,npnts,nstyp,
                          a,ia,ja,f,iu,ndiu,ip,ndip)

!   user-defined check(s) of the input matrix

!   input: all parameters are input parameters described earlier in this manual.
!           only new parameters:

!           nstyp: =1: scalar approach has been selected
!                  =2: unknown-based approach has been selected
!                  =3: any of the point-based approaches has been selected

!           npnts: In case nstyp=3: number of points. otherwise: npnts=0.
!           note: this routine will be called at the beginning of SAMG if the SAMG
!           parameter chktol ≥ 0.0. in this case, tol=chktol on input.

implicit none
integer nnu,nna,nsys,npnts,nstyp,ndiu,ndip
double precision tol
character(len=200) outstring

integer iscale(nsys),ia(nnu+1),ja(nna),iu(ndiu),ip(ndip)
double precision a(nna),f(nnu)

integer n_select,i,j,i1,jlo,jhi,mode_mess

! => modify this if necessary !!!
call samg_getinteger('mode_mess',mode_mess)

```

```

    if (mode_mess.lt.0) return

    write(unit=outstring,fmt=9000)
    call samg_message(1,0,outstring)

!> Insert user defined checks here

n_select=0    ! adjust this value to whatever check is requested

select case (n_select)
  case(0); write(unit=outstring,fmt='(a)') 'None!'
    call samg_message(0,0,outstring)
  case(1); write(unit=outstring,fmt='(a)') 'User check #1:'
    call samg_message(0,0,outstring)
    do i=1,nnu
      if (iu(i).ne.4) cycle
      write(unit=outstring,fmt=9010) i,iu(i),a(ia(i))
      call samg_message(0,0,outstring)
      jlo=ia(i)+1; jhi=ia(i+1)-1
      do j=jlo,jhi
        il=ja(j)
        write(unit=outstring,fmt=9015) i1,iu(il),a(j)
        call samg_message(0,0,outstring)
      enddo
    enddo
    write(unit=outstring,fmt='(a)') 'Done!'
    call samg_message(0,0,outstring)
  case default
    write(unit=outstring,fmt='(a)') 'None!'
    call samg_message(0,0,outstring)
end select

!> finished

write(unit=outstring,fmt=9001)
call samg_message(0,0,outstring)
return

9000 format('>>> User-defined checks (begin)')
9001 format('>>> User-defined checks (end)')
9010 format('var=',i8,' unknown=',i2,' entry=',d8.2)
9015 format(25x,'var=',i8,' unknown=',i2,' entry=',d8.2)

end subroutine samg_user_check

```

8.3.3 Interface to a user's license checker

A proper modification of the below dummy routine provides a means to interface with a user's license checker. This routine is called at the very beginning of SAMG.

```

subroutine samg_check_license(licensed)

!  routine to interface with a user's license checker.
!  licensed is an integer output parameter:
!    return with licensed=0 to indicate that there exists no license,
!    otherwise return with licensed=1.

implicit none
integer licensed
licensed=1    ! currently no check done (always satisfied)
return
end subroutine samg_check_license

```

8.3.4 Interface for getting current cycle number and residual

The following interface allows a user to access the current *cycle number* and the corresponding *residual*. This routine is regularly called during the solution phase. Depending on the current values of *ncycle* and *residual*, the user may decide to perform some action. Currently, no action is coded:

```

subroutine samg_current_residual(ncycle,residual)

!   provides user-access to current cycle number and residual

implicit none
integer ncycle
double precision residual
character(len=200) outstring

!   no action at the moment! writing can be done as follows:
!       write(unit=outstring,fmt='(a,i4,a,e11.3)') 'Iter=',ncycle,', Res=',residual
!       call samg_message(0,0,trim(outstring))

return
end subroutine samg_current_residual

```

8.3.5 Message routing

All output of SAMG is routed through the following routine, `samg_message`. This routine can be adjusted by the user to his needs. For instance, all output of SAMG may be transferred to a GUI.

```

subroutine samg_message(nbefore,nafter,outstring)

!   sends string "outstring" to unit "logio_valid" (connected to the screen,
!   a file fort.nn or a file "logfile", see also parameter "logfile").
!   in addition, nbefore/nafter empty lines are printed before/after.

!   this routine is not to be used outside SAMG!

implicit none
logical exist
integer nbefore,nafter,i,iounit
character(len=*) outstring

call samg_get_logio_valid(iounit)
do i=1,nbefore; write(unit=iounit,fmt=*) ; enddo
write(unit=iounit,fmt='(1x,a)') trim(outstring)
do i=1,nafter; write(unit=iounit,fmt=*) ; enddo

return
end subroutine samg_message

```

9 Code Numbers of Errors and Warnings

In case a **fatal error** occurred during the execution of SAMG, the program will terminate with a message and the error indicator *ierr* will return one of the values listed in the following table. In case a **non-fatal error** occurred, a warning message will be issued but SAMG continues. Upon return from SAMG, *ierr* will be assigned a *negative* value (its absolute value corresponds to one of the below error codes).

IERR	Description
General errors	
1	General error (unclassified)
5	Error in license checking
10	Insufficient dimensioning (happens only if memory extension is turned off)
20	Illegal input parameter
30	Undefined or missing input
40	Error in input arrays
50	Incorrect or inconsistent input
51	SAMG re-start terminated: initial call with iswit=4 required
52	SAMG re-start terminated: no AMG setup data available
53	SAMG re-start terminated: illegal change in parameters
54	SAMG re-start terminated: illegal change in matrix
60	Memory management failed (including file I/O to scratch files)
70	Allocation and de-allocation errors
71	Unexpected allocation status of allocatable array
72	Unexpected association status of pointer array
73	Error in restoring internal data
80	Requested AMG component not installed in current release
90	Logfile exists but could not be opened
91	Logfile already connected to different unit
92	Logfile does not exist and could not be opened
93	Specified unit does not exist
94	Illegal use of unit 5: Specify a logfile or another unit number
99	Unexpected internal SAMG error (only XSAMG)
AMG's setup phase	
100	General error (unclassified)
101	Setup failed in CNTRL-routine (automatic setup mechanism)
110	Error in defining strong connectivity
120	Error in the splitting process
130	Error in the coarsening process
140	Error in defining interpolation
150	Error in computing the coarse-level Galerkin operators
151	Galerkin truncation failed: non-positive diagonals produced
152	Galerkin truncation failed: symmetric counterpart missing
153	Negative diagonals on coarse levels could not be avoided, try to increase <i>neg_diag</i>
154	No correction of negative diagonals on current level was possible
160	Error in performing the setup optimization
170	Error in parallel Uzawa's communication (setup phase)
180	Error in parallel Uzawa's matrix operations (setup phase)

IERR	Description (continued)
AMG's solution phase	
200	General error (unclassified)
210	Divergence of the method
211	Residual is not a number (NaN)
220	Error in relaxation (smoothing)
230	Error in ILU (smoothing)
240	Error in ILUT (smoothing)
250	Error in inter-grid transfers
260	Error in alternating "Schwarz process"
270	Error in Uzawa (eg, negative diagonals)
290	Error in solver control: convergence seems ok, <i>eps</i> -criterion not reached
291	Error in solver control: divergence occurred for both solvers
Code numbers specific for SAMGp	
300	General error in communication
301	Error in XSAMG data distribution
302	Error in MPI routine (MPI has returned error code)
303	An error occurred in a different process (detected in safe mode)
310	Illegal use of routine in sequential context (ie, parallel only)
320	Illegal use of routine in parallel context (ie, serial only)
398	A feature has been selected which is not yet supported by SAMGp
399	A parameter has been accessed which is not yet supported by SAMGp
Auxiliary components	
800	General error (unclassified)
810	Error in ILU (one-level)
820	Error in ILUT (one-level)
830	Error in conjugate gradient (CG or flexible CG)
831	Quasi residual check has been reached (CG or flexible CG)
832	Residual stagnation check has been reached (CG or flexible CG)
833	Limit of accelerator re-starts has been reached (CG or flexible CG)
840	Error in BiCGstab
841	Quasi residual check has been reached (BiCGstab)
842	Residual stagnation check has been reached (BiCGstab)
843	Limit of accelerator re-starts has been reached (BiCGstab)
850	Error in GMRES
851	Quasi residual check has been reached (GMRES)
852	Residual stagnation check has been reached (GMRES)
853	Limit of accelerator re-starts has been reached (GMRES)
Solution on coarsest level	
900	General error (unclassified)
910	Error in method #1 (iterative application of current smoother)
920	Error in method #2 or #4 (preconditioned CG)
930	Error in method #3 or #5 (preconditioned BiCGstab)
960	Error in method #6 (Full Gauss elimination)
970	Error in method #7 (Sparse Gauss elimination)
980	Error in method #8 (Least squares solver)
990	Error in method #9 (user defined solver)

10 What is New?

Various bugs have been fixed which are not listed in detail. More important, many new features have been released, more will be released soon. The most important ones are listed below.

Modified default values

parameter	new default	old default	Section
<i>nptmn</i>	100	500	6.2.4
<i>nptmax</i>	1500	3000	6.3.4.1
<i>nkdim_default</i>	30	10	6.4.2
<i>factor_quasi_res</i>	1.5	10.0	7.2.2
<i>irestriction_openmp</i>	2	-1	7.1.1
<i>ioform</i>	"b"	"f"	7.8

Formal changes

- Those special **set-** and **get-routines** which are usable only from within Fortran are still available but will not be supported any more (→ Section 6.1).
- The parameter **fullystored** is no longer available. Instead, use *isym*=matrix[1:1] to indicate that only the lower triangular part of a symmetric matrix is passed to SAMG (→ Section 5.1).
- The parameter **struct_ident** now has to be set immediately before calling SAMG with *iswit*=4 (→ Section 6.2.8).
- In the **automatic solver control** all parameters **_one* have been renamed to **_2nd* (→ Section 7.5).
- The parameter **nrc_default** has been removed. **nrc=0** now means *nrc*=11 if Intel's MKL has been linked and *nrc*=7 otherwise (→ Section 6.3.4.1).
- The variable-to-point vector, **ip**, can now be ordered arbitrarily (→ Section 4.2).

New features/options

- As a new feature, **defect correction** has been implemented (→ Section 8.2).
- The use of **one-level solvers** has been implemented systematically (→ Sections 7.6 and 0).
- All coarsest-level solvers (including pardiso) are now directly usable also as one-level solvers (→ Section 7.6.1.2). A new hidden parameter, **csolver_finetest**, has been introduced for this purpose.
- Diagonally preconditioned CG, BiCGstab, GMRes are available as one-level solvers (→ Section 7.7.1).
- For symmetric matrices, **ICCG** is now available as a one-level solver (→ Section 7.7.2).
- SAMG's default behavior in case of *unintentional* one-level situations (ie, number of levels =1 after setup) can now be changed in a flexible and robust way (→ Section 7.6.2).
- The setting of *norm_typ* is now respected also if **GMRes** is used as an accelerator (before, residuals were always measured in the *L2*-norm, independent of the setting of *norm_typ*). Exception: Independent of the setting of *norm_typ*, the **quasi-residual check** is always performed w.r.t. the *L2*-norm if GMRes is selected (→ Section 7.2.2).
- A beta version of **smoothed aggregation** is now included (→ Section 7.12).
- All accelerators now allow for user-defined **re-start points** (→ *nkdim* in Section 5.5).
- A subroutine is available which returns the currently available number of levels (→ Section 5.10.4.3). This can be used to detect unintentional one-level cases to adjust the solver.

- A dummy **user interface** is available again allowing the user to implement his own coarsest-level solver (→ Section 8.3.1).⁹⁷
- Safety check for **integer overflow** introduced (→ Section 5.6.2).

Performance

- The **automatic solver control** has been further improved and generalized (→ Section 7.5). There is more flexibility in the selection of the second solver (in particular, it may now also be an AMG solver with parameters different from the first one).
- In addition to previous **coarsest-level solvers**, Intel's **pardiso** is now fully integrated in SAMG if Intel's MKL is available (optionally with OpenMP support). This improves performance and allows terminating the coarsening process on relatively fine levels, adding robustness in case of critical applications. **pardiso** is selected via *nrc*=11 (→ Section 6.3.4.1).
- In addition to previous **coarsest-level solvers**, now also an efficient **dense direct solver** is available. Although also useful for SAMG, this solver is particularly efficient for parallel SAMGp applications on a large number of processes. The dense solver can be selected via *nrc*=10 (→ Section 6.3.4.1).
- **pardiso** has been included also as a solver in the alternating Schwarz method (→ Section 7.4.3).
- **OpenMP parallelization** has been improved for SAMG's coarsening process: Now also SAMG's coloring process is parallel which has been a bottleneck in case of larger number of cores. In this context, a new parameter, *icolor_omp*, has been introduced (→ Section 7.1.1).
- **OpenMP parallelization** has been improved for Gauss-Seidel relaxation and conjugate gradient.
- New **aggregative strategies** have been implemented ("cluster coarsening, type 2") facilitating very effective coarsening processes (→ *cl_order* in Section 6.2.3.3, in particular, settings 3-5).
- For most **aggregative strategies** now not only a *maximum* but also a *minimum* cluster size can be specified (→ parameter *min_cl_size* in Section 6.2.3.3).

Forthcoming releases

For the next releases, **performance issues** will continue to be in the very focus of our development, in particular, w.r.t. to new hardware. Also the further development of the **automatic solver control** module will be a major issue. Moreover, the following topics are under development.

In the short term

- Removing OpenMP-induced race conditions due to **OpenMP reductions**.
- Multi-color relaxation
- Dynamic cycling strategies
- Implementation of **defect correction** also in SAMGp.
- Alternative treatment of "**special**" variables by SAMG (instead of a direct solver).
- Extension of **OpenMP parallelization** and continuous optimization of its performance.
- Optimization of **aggregative coarsening** strategies, in particular, of **smoothed aggregation**.
- Implementing **smoothed aggregation** also for SAMGp.
- Improved treatment of **linear elasticity** applications with focus on near-singular cases.
- Further performance improvement by optimizing **hybrid coarsening** strategies.

In the medium term

- Investigation of **mixed order interpolation** as well as mixed *-cycles for coupled systems of PDEs.
- Improvements for **non-definite** applications (negative diagonal entries on coarser levels).
- Context-based support to make SAMG **re-entrant** (based on C or new Fortran standards).

⁹⁷ This feature requires a source code license.

11 References

The following gives a list of literature on AMG which is relevant in the context of SAMG. [1-5] give some early references. In particular, [5] contains an extensive description regarding both theory and numerical investigations on AMG. [6-9] are some more recent references. In particular, [8] contains a review on the development of AMG including an extensive list of references. [9] refers to an early parallel version of AMG. Finally, [10-29] refer to various applications.

1. **Brandt, A.; McCormick, S.; Ruge, J.:** *Algebraic Multigrid (AMG) for Sparse Matrix Equations*, in "Sparsity and its Applications" (D.J. Evans, ed.), Cambridge University Press, pp. 257-284, Cambridge, 1984.
2. **Brandt, A.:** *Algebraic Multigrid Theory: The Symmetric Case*, Appl. Math. Comp. 19, pp. 23-56, 1986.
3. **Ruge, J.W.; Stüben, K.:** *Efficient solution of finite difference and finite element equations by algebraic multigrid (AMG)*, in Multigrid Methods for Integral and Differential Equations (Paddon, D.J.; Holstein H.; eds.), The Institute of Mathematics and its Applications Conference Series, New Series Number 3, pp. 169-212, Clarendon Press, Oxford, 1985.
4. **Stüben, K.:** *Algebraic multigrid (AMG): Experiences and comparisons*, Appl. Math. Comp. 13, pp. 419-452, 1983.
5. **Ruge, J.W.; Stüben, K.:** *Algebraic Multigrid (AMG)*, In "Multigrid Methods" (S. McCormick, ed.), Frontiers in Applied Mathematics, Vol 5, SIAM, Philadelphia 1986.
6. **Krechel, A.; Stüben, K.:** *Operator dependent interpolation in algebraic multigrid*, Proceedings of the Fifth European Multigrid Conference, Stuttgart, Germany, Oct 1-4, 1996. Lecture Notes in Computational Science and Engineering 3, Springer Verlag, 1998. Also available as Arbeitspapier der GMD 1042, January 1997.
7. **Stüben, K.:** *An Introduction to Algebraic Multigrid*. Appendix in the book "Multigrid" by U. Trottenberg; C.W. Oosterlee; A. Schüller. Academic Press, pp. 413-532, 2001. Also available as GMD Report 70, November 1999.
8. **Stüben, K.:** *A Review of Algebraic Multigrid*, Journal of Computational and Applied Mathematics (JCAM) 128, pp. 281-309, 2001 (Volume 7 of JCAM Millenium Issue). Also available as GMD Report 69, November 1999.
9. **Krechel, A.; Stüben, K.:** *Parallel Algebraic Multigrid Based on Subdomain Blocking*, Parallel Computing 27, pp. 1009-1031, 2001. Also available as GMD Report 71, December 1999.
10. **Füllenbach, T.; Stüben, K.; Mijalkovic, S.:** *Application of an algebraic multigrid solver to process simulation problems*, Proceedings of the Intern. Conference on Simulation of Semiconductor Processes and Devices, Seattle (WA), USA, Sep 6-8, 2000. IEEE, Piscataway (NJ), USA, pp. 225-228, 2000.
11. **Füllenbach, T.; Stüben, K.:** *Algebraic multigrid for selected PDE systems*, Proceedings of the Fourth European Conference on Elliptic and Parabolic Problems, Rolduc (The Netherlands) and Gaeta (Italy), 2001. World Scientific, New Jersey, London, pp. 399-410, 2002.

12. **Clees, T.; Stüben, K.:** *Algebraic multigrid for industrial semiconductor device simulation*, Proceedings of the First Intern. Conference on Challenges in Scientific Computing, Berlin, Germany, Oct 2-5, 2002. Lecture Notes in Computational Science and Engineering 35, Springer, Heidelberg, Berlin, 2003.
13. **Stüben, K.; Delaney, P.; Chmakov, S.:** *Algebraic Multigrid (AMG) for Ground Water Flow and Oil Reservoir Simulation*, Proceedings of the Conference "MODFLOW and More 2003: Understanding through Modeling", International Ground Water Modeling Center (IGWMC), Colorado School of Mines. Golden, Colorado, Sept 17-19, 2003.
14. **Clees, T.:** *AMG Strategies for PDE Systems with Applications in Industrial Semiconductor Simulation*. Ph.D. Thesis, University of Cologne, Shaker Verlag, Aachen, Germany, October 2005.
15. **Zitzmann, M.; Clees, T.; Weigel, R.:** *Iterative methods for reluctance based PEEC models*. In Proceedings of 17th International Zurich Symposium on Electromagnetic Compatibility (EMC-Zurich), Singapore 2006, Feb 27 - Mar 3, IEEE, pp. 81–84, 2006.
16. **Zitzmann, M.; Grillmair, R.; Clees, T.; Weigel, R.:** *Hybrid solver strategies in automotive emc simulation*. In Proceedings of 17th International Zurich Symposium on Electromagnetic Compatibility (EMC-Zurich), Singapore 2006, Feb 27 - Mar 3, IEEE, pp. 340–343, 2006.
17. **Klie, H.; Wheeler, M.F.; Clees, T.; Stüben, K.:** *Deflation AMG Solvers for Highly Ill-Conditioned Reservoir Simulation Problems*, paper SPE 105820 presented at the 2007 SPE Reservoir Simulation Symposium, Houston, TX, Feb. 26-28, 2007.
18. **Stüben, K.; Clees, T.; Klie, H.; Lou, B.; Wheeler, M.F.:** *Algebraic Multigrid Methods (AMG) for the Efficient Solution of Fully Implicit Formulations in Reservoir Simulation*, paper SPE 105832 presented at the 2007 SPE Reservoir Simulation Symposium, Houston, TX, Feb. 26-28, 2007.
19. **Stüben, K.:** *Solving Reservoir Simulation Equations*, 9th International Forum on Reservoir Simulation, December 9-13, 2007, Abu Dhabi, United Arab Emirates.
20. **Larson, G.; Snyder, D.; Vanden Abeele, D.; Clees, T.:** *Application of single-level, pointwise algebraic, and smoothed aggregation multigrid methods to direct numerical simulations of incompressible turbulent flows*, Computing and Visualization in Science, Springer Berlin/Heidelberg, Vol. 11, Number 1, 2008.
21. **Batycky, R.P.; Förster, M.; Thiele, M.R.; Stüben, K.:** *Parallelization of a Commercial Streamline Simulator and Performance on Practical Models*, paper SPE 118684 presented at the 2009 SPE Reservoir Simulation Symposium, Houston, TX, Feb. 2-4, 2009.
22. **Clees, T.; Ganzer, L.:** *An Efficient Algebraic Multigrid Solver Strategy for Adaptive Implicit Methods in Oil-Reservoir Simulation*, paper SPE 105789, SPE Journal, Vol 15, No 3, 2010
23. **Zaretskiy, Y.; Geiger, S.; Sorbie, K.; Förster, M.:** *Efficient flow and transport simulations in reconstructed 3D pore geometries*, Advances in Water Resources (2010). doi:10.1016/j.advwatres.2010.08.008.
24. **Zaretskiy, Y.; Geiger, S.; Sorbie, K.; Förster, M.:** *Pore-scale modelling of chemically induced effects on two-phase flow*, in: 12th European Conference on the Mathematics of Oil Recovery, 2010.

25. **Förster, M.; Kraus, J.:** *Scalable parallel AMG on cc-NUMA machines with OpenMP*. Accepted for publication in Proc. International Supercomputing Conf (ISC), Hamburg, Germany, 2011.
26. **Thum, P.; Hesch, W.; Stüben, K.:** *LMG2: Accelerating the SAMG Multigrid-Solver in MODFLOW*. Proceedings Modflow & More, U.S.G.S., USA, Golden, July 2011.
27. **Kraus, J.; Förster, M.:** *Efficient AMG on Heterogeneous Systems*. Proceedings of the Multicore Challenge II, 28th- 30th Sept 2011, Karlsruhe, Germany, 2011.
28. **Kraus, J.; Förster, M.; Brandes, T.; Soddemann, T.:** *Using LAMA for Efficient AMG on Hybrid Clusters*. Submitted to: International Supercomputing Conf (ISC), Hamburg, Germany, 2012.
29. **Zecchin, A.; Thum, P.; Simpson, A.; Tischendorf, C.:** *Steady-state behaviour of large water distribution systems: The algebraic multigrid method for the fast solution of the linear step*. Accepted for publication in: ASCE's Journal of Water Resources Planning and Management, 2012.

12 Index

*

*-cycles 35, 36, 66

A

a 24
 $A_{(i,j)}$ 19
 $A_{[n,m]}$ 18
a_cmplx 43
a_cmplx_agg_default 71
a_cmplx_default 71
a_dc 120
 accelerator 35
 accelerators, inner iterations 66
 aggregative coarsening 13, 55
 aggressive coarsening 13, 55
 A-interpolation 31
 algebraic multigrid 8
allow_elim 60
allow_filnam_in 110
 alternating Schwarz method 84, 87
 alternating Schwarz method with overlap 88
 alternative solver 92
 AMG 8
 AMG convergence on coarser levels 57
 AMG1R5 8, 119
 approximate Schur complement 91
 approximation criterion 79
 automatic solver control 12, 92

B

b_cmplx 60
b_cmplx_agg_default 60
b_cmplx_default 60
backup 97
 BiCGstab 12, 35
 BLAS 69
blk_fillexp 88
blk_stab 88
 block coarsening 29
 block coupling 19
 block ILU 28
 block ILU(0) / GMRes 102, 103
 block relaxation 28
 block relaxation, pivoting 65
 block size in writing data 40

C

C/F-relaxation 37, 64
 C1-interpolation 31
 C2-interpolation 31
 calling sequence 24
 cellular automaton 56
 changing number of threads 78
chktol 44
 CL solver 67, 69
cl_cpl 56
cl_order 56
 classical clustering 56
 classical coarsening 54
cldense_ctrl 68
clsolver_finet 101
 cluster coarsening 13, 54
 cluster coarsening, type 1 55, 56
 cluster coarsening, type 2 55, 56
 cluster size 114
 cluster size, maximum 56
 cluster size, minimum 56
cmplx_min 111
cntrl_enforce 98
cntrl_logfile 97
cntrl_method 92, 95
cntrl_release 97
cntrl_res_diff 98
cntrl_retry_iter 98
cntrl_steady 98
 coarse-grid correction 8
 coarsening 13
 coarsening pattern 109
 coarsening strategy 53
 coarsening, aggregative 13
 coarsening, aggressive 13, 55
 coarsening, cluster 13
 coarsening, history 45
 coarsening, slow 57
 coarsening, standard 13, 55
 coarsening, very slow 57
 coarsest-level solver 10, 14, 38, 67
 coarsest-level solver, choice of 69
 coarsest-level solver, user-supplied 67
 complex-valued problems 23
 compressed row format 21
 compressed row storage 21
 conjugate gradient, CG 12, 35
 conjugate gradient, new 72
 conjugate gradient, old 72
 console output 44

conv_stop 68
convergence criteria 79
coordinates 12, 83
copying data to disk 40
coupled system 17
couplings, negative 53
couplings, positive 53
couplings, strong 10, 53
couplings, weak 10
C-relaxation 64
cset_lastpnt 86
cset_lessvars 86
cset_longrow 86
cset_read 91
cset_unknown 86
cset_unknown1 86
cset_unknown2 86
cset_zerodiag 86
CSR 21
C-variables 10
cycling 14

D

data structure 21
DC 119
deco_required 121
default switch (n_default) 41, 72
defect correction 119
delta_milu 65
dense direct solver 67
densx 57
diagonal dominance, strong 57
diagonal dominance, strong (local) 53
diagonal dominance, strong violation 53
diagonal preconditioning 35, 106
diagonally preconditioned BiCGstab 67, 106
diagonally preconditioned CG 67, 106
diagonally preconditioned GMRes 106
diameter minimization 56
direct solvers, multiple RHS 104
dirty trick 110
divergence 97
droptol_2nd 95
droptol_cl 68
droptol_smo 65
dummy unknown 19, 30
dump_correctw 61
dumping matrices to disk 108

E

ecg 53
ecg_default 53
ecg1 53

ecg2 53
elliptic 17
energy norm 108
eps 25, 79
eps_dc 120
eps_dd 57
eps_diag 59
eps_lsq 117
eps_optrunc 63
error codes 126
error flag (ierr) 46
errors, fatal 126
errors, non-fatal (warnings) 126
etr 63
etr_default 63
etr1 63
etr2 63
ewt 53
ewt_default 53
ewt1 53
ewt2 53
exceptional F-variables 55
exceptional variables 12

F

f 24
F*-cycle 14
factor_app_var 80
factor_matrix_scale 118
factor_quasi_res 81
factor_res_var 81
fallback solver 101
F-cycle 11, 35
FE discretization, anisotropic 52
file access 107
filnam 107
filnam_dump 107
first touch policy 76
flexible accelerators 36
flexible CG 12, 35
flexible GMRes 12, 35
force_accel 66
force_accel_2nd 95
forcing into coarse levels 85
forcing into finest level 85
format file, .frm 107
Fortran convention 15
frames 108
free current license 16
F-relaxation 64
front-side bus (FSB) 78
FSB 78
full_pivoting 68
F-variables 10

F-variables, exceptional 55

G

g_cmplx 43
g_cmplx_agg_default 71
g_cmplx_default 71
 Galerkin principle 8, 10
 Galerkin, block-diagonal 18, 27
 Galerkin, negative diagonal entries 61
 Galerkin, treatment of zero entries 61
 Gauss-Seidel 28
 Gauss-Seidel / GMRes 103
 Gauss-Seidel relaxation, full sweep 64
 Gauss-Seidel relaxation, partial sweeps 64
 general control switch, iswtch 39
 geometric coarsening 29
 geometric information 83
 geometric multigrid 8
 get-routines 50
glk_mult_zeros 61
 GMRes 12, 35
 graph 17
 grid complexity 43
 GUI 70, 125

H

hardware threads 77
 hidden parameters 14, 50
 hidden parameters, priority 73
 hierarchical algorithms 8
 history of errors 42, 107
 history of residuals 107
huge 41
 hyperthreading 74, 77

I

ia 24
ia_dc 120
iadp 45
iall 59
iall_smo 114
iauto_stop 81
iauto_stop_level 81
ib_cmplx 60
ib_cmplx_agg_default 60
ib_cmplx_default 60
ibgs_pivot 65
ic_set_strategy 112
 ICCG for sym. problems 35, 95, 106
icdp 45
icheck_quasi_res 81
icolor_omp 75

icolor_omp_level 75
icolor_omp_mode 75
icrits 80
idmp 45, 108
idump 45, 108
ierr 46
iextent 40
ifirst 25
iforce0 24
igam 35
igdp 45
ignore_warn 96
ihuge 41
 ILU(0) 28
 ILU(0) / CG 67, 95, 102, 103
 ILU(0) / GMRes 95
ilu_speed 65
ilu4allschwarz 90
 ILUT 28
 ILUT / BiCGstab 67, 95, 102, 103
 ILUTP 28
 ILU-type one-level solvers 97
 ILU-type smoothers 28, 65
imax 112
imin 112
imodified 112
impldo_read_write_len 40
 initial memory allocation 42
inner_accel 66
 input at run time 109
 input matrix checking 44
 integer overflow 41
internal 29
 interpolation 10, 13
 interpolation, blockwise 27
 interpolation, cluster 13
 interpolation, direct 13, 59
 interpolation, extended standard 13, 59
 interpolation, F-relaxation 13
 interpolation, Jacobi 13, 59
 interpolation, multi-pass 13, 59
 interpolation, piecewise constant 13, 55
 interpolation, rescaling 59
 interpolation, row sum one 59
 interpolation, separate 27
 interpolation, standard 13, 59
 interpolation, the same 27
intlev 59
io 112
iodump 107
ioform 107
iogrid 107
iomovie 107
ioscratch 42
ioscratch_default 71

iout 44
iout_level 112
ip 22, 26
ipass_max_set 59
ipriority 86, 89
irestriction_openmp 75
irow0 24
iscale 26
iscale_new 112
iswit 39
iswit, 1-5 93
iswit, 6-9 94
iswtch 39, 40, 41, 42
isym 24
iter_check 68
iter_matrix_scale 118
iter_pre 72
itint 59
itmax_conv 68
itypu 25
iu 22, 26
iu_new 112
iwdp 45

J

ja 24
ja_dc 120
 Jacobi 28
 Jacobi / CG 103
 Jacobi F-relaxation (for interpolation) 58
 Jacobi relaxation 64
 Jacobi smoothing for coupled systems 65

K

Krylov method 35
 Krylov space 35, 71

L

LAPACK 69
lastgrid 107
 least squares solver 67, 69
 levels used 49
levelx 57
levelx_2nd 95
lfil_2nd 95
lfil_cl 68
lfil_smo 65
 license checker 16
 limiting re-starts check 81
 linear elasticity 113, 116
 logfile 47, 70
 logically equivalent, point numbering 22

logio 70
 lower triangular matrix 25
 lower triangular part of matrix 106
 LSQ solver 67
 lu_reuse 94, 97

M

matrix 24
 matrix density, maximum 57
 matrix, differences upon return 25
 matrix, lower triangular part 25
matrix_scale_renorm 118
max_cl_size 56
max_cyc 96
max_level 57
max_nb_list_size 56
maxcorr_cluster 114
 maximum cluster size 56
maxop_restart 61
 memory bandwidth 76
 memory extension switch 40
 memory management 11, 16
 memory requirement 42
 message routing 70
 messages 70
milu 28, 65
 MILU(0) 28
 MILUTP 28
min_cl_size 56
min_multi_size 98
 minimum cluster size 56
 mixed-sign rows 54
 MKL library 67
 M-matrices 17
mode_cntrl 96
mode_debug 72
mode_mess 44, 70
 modified cycles 35, 36, 66
modify_mat 118
 multigrid cycle 10, 11, 35
 multigrid principle 8
multipass_allcoup 60
 multiple instances 12, 109

N

n_default 41, 72
napproach 27
nblk_debug 88
nblk_max 87
nblk_overlap 90
nblk_resid 88
nblk_solve 87
nblk_solver 87

ncframes 107
ncg 54, 55
ncgrad 35
ncgrad_default 71
ncgtyp 54
ncyc 35
ncyc_2nd 95
ncyc_default 71
ncyc_done 46
ncyc_min 72
ncyc_start 72
ncycle 35, 79
ncycle_dc 120
ndim_space 116
ndip 26
ndiu 26
ndyn_smo 64
neg_diag 61
neg_diag_brute 61
negative couplings 53
negative diagonal entries on coarser levels 61
negative off-diagonal entries 52
negative sign, *iswtch* 42
negative sign, *levelx* 57
negative sign, *nblk_overlap* 90
negative sign, *ncyc* 35
negative sign, *np_mod1* 117
negative sign, *nrd* or *nru* 65
negative sign, *nsolve* 27
new_topology 96
nfailed_accsteps 81
nint_pat 31
nint_rowsum1 59
nint_weights 31
nkdim 35
nkdim_dc 120
nkdim_default 71
nmin_matrix 75
nmin_matrix_resc 75
nmin_vector 75
nna 24
nna_dc 120
nru 17, 24
non-linear iterative processes 36
non-uniform memory architecture 76
nopt 116
nopt_cforce 117
nopt_fit 117
nopt_split 117
nopt_strong 117
norm, maximum 60
norm, row sum 60
norm, Schur 60
norm_typ 42
normalization 46
note books 78
np_mod1 117
np_mod2 117
np_opt 116
npclass 116
npcol 55
npnts 17
npr_is_dummy 30
nprim 30
nptmax 57, 67
nptmn 57
nrc 67
nrc_emergency 67
nrd 64
nrd_2nd 95
nrdtyp 64
nr dx 64
nred 55
nredlev 55
nru 64
nru_2nd 95
nrutyp 64
nrux 64
nsmoagg 114
nsmooth 113
nsolve 27, 28, 29, 30, 31
nsolve_2nd 95
nsolve_default 71
nstar_typ 66
nstrict_ncyc 96
nstrict_quasi 96
nstrict_stagr 96
nstrict_stagu 96
nsys 17, 26
nsys_new 112
ntake_res_in 46, 72
nth_res_scratch 81
ntr 63
ntrunc 63
ntyp_accel 35, 72
ntyp_galerkin 75
num_old_apps 81
num_old_res 81
NUMA 76
number of levels used 49
numtry_max_set 59
nwt 59
nwtint 59
nx_f_clean 55
nxtyp 28
nxtyp_coarse 65

omega_smo 114
OMP_GET_WTIME() 78
OMP_NUM_THREADS 74
one-level solvers 12, 36, 101
one-level solvers, CL solver 103
one-level solvers, default behavior 48
one-level solvers, diag. preconditioning 106
one-level solvers, ICCG for symmetric problems 106
one-level solvers, intentional use 101
one-level solvers, smoothers 102
one-level solvers, unintentional use 48, 101, 105
OpenMP 12, 74
 coarsening process 75
 Galerkin operators 75
 residual restriction 75
OpenMP, front-side bus (FSB) 78
OpenMP, hyperthreading 77
OpenMP, note books 78
OpenMP, race condition 77
OpenMP, reduction 77
OpenMP, sparse matrices 77
OpenMP, time measurements 78
operator complexity 43
optimization, level of 116
order of priority assignments 87, 91
ordering by points 19
ordering by unknowns 18
ordering, consecutive 23
output unit 70

P

P, primary matrix 19
p_cmplx 43
p_cmplx_agg_default 71
p_cmplx_default 71
pairwise aggregation 56
pairwise clustering 56
parallel dense solver 103
parameters, constants 24
parameters, primary 24
pardiso 67, 103, 104
pardiso, Schwarz method 87
pattern, SW 31
P-interpolation 31
point 17
point complexity 43
point-based approaches 18, 27
point-coarsening 18
positive couplings 53
positive couplings, critical 41
positive couplings, large 53
positive couplings, non-critical 41

positive diagonal entries 23
positive off-diagonal entries 52
preconditioner 11, 12, 35
pre-cycling 38, 72
pre-iteration 38, 72
prim_norm 60
prim_print 30, 60
priml.amg 60
primary matrix 19, 29
primary matrix, external 29
primary matrix, internal 29
primary matrix, non-zero pattern 30
primary matrix, sparsity pattern 30
primary matrix, writing to disk 60
primary parameters 14, 24
primary solver control 92, 94
primary unknown 19, 30
print output, setup phase 45
print output, solution phase 44
priority values 84

Q

quasi residual 80
quasi residual check 80, 81

R

race condition 77
race condition, Gauss-Seidel 77
race condition, introduced by algorithm 77
race condition, introduced by OpenMP 77
RAMG 8
rcondx 68
real residual 80
re-directing output 70
reference matrix 52
refresh, forcing a new seup 94
relaxation 28
release setup data 40
res_in 46
res_out 46
rescaling of interpolation 59
residual criterion 79
residual criterion, absolute 79
residual criterion, relative 79
residual criterion, time stepping 79
residual stagnation check 81
residual, quasi 80
residual, real 80
re-start 81
restriction 10
re-using (I)LU decompsitions 102
re-using ILU decompsitions 94
re-using setup data 12

re-using setup data, automatically 92, 94
 re-using setup data, manually 39, 93
rho_min 96
rho_ok 96
 rigid body modes 113
 row sum one interpolation 59

S

s.p.d. 17
 saddle point problems 18, 28
 safety checks 80
safety_factor 111
 SAMG 8
 SAMG with no setup 93, 94
 SAMG with partial setup 93, 94
samg_check_license 124
samg_coo_vec_alloc 83
samg_coo_vec_dealloc 83
samg_coo_vec_set 83
samg_current_residual 125
samg_cvec_alloc 84
samg_cvec_dealloc 84
samg_cvec_set 85
samg_dc 119
samg_get_ 50
samg_get_complexities 111
samg_get_levels_used 49
samg_get_num_smooth_vectors 114
samg_iget_ 51
samg_iset_ 51
samg_leave 16, 51
samg_message 70, 125
samg_omega_jacobi_alloc 66
samg_omega_jacobi_dealloc 66
samg_omega_jacobi_set 66
samg_propose_nsys_iscale_iu 112
samg_refresh 16, 40, 94
samg_reset_hidden 51
samg_reset_secondary 51
samg_set_ 50
samg_smooth_vector_init 113, 114
samg_smooth_vector_set 114
samg_user_check 123
samg_user_solve 121
 scalar approach 17
 scalar system 17
 scaling 26, 46
 scaling by rows or columns 118
 Schwarz 84, 87
 Schwarz variables 87
 Schwarz with overlap 88
 scratch file 40, 42

scratch files, unit and name 71
scratch.amg 40, 42
scratchfile 71
 second solver 95
 secondary solver control 92, 93
 set-routines 50
 setup phase 10
 setup, full reuse 39
 setup, partial reuse 39
show_un_res 72
 skipping error messages 70
 skipping warning messages 70
slow_coarsening 57
 smoothed aggregation 55, 113
 smoothing 8, 10, 13
 smoothing on coarser levels 28, 65
 smoothing strategy 28
 smoothing, post- 64
 smoothing, pre- 64
 solution file, .sol 107
 solution phase 10
solution stagnation check 81
 solution strategy 27
 solver control, automatic 92
 solver control, primary 92
 solver control, secondary 92, 95
 solver switching 93, 95
 solving very small problems 48
 sparse direct solver 67
stability 68
 stand-alone 11, 12, 35
 stand-alone cycling 35
 standard coarsening 13, 55
 standard cycles 35
 stopping criteria 12, 79
 strong couplings 52, 53
struct_ident 62
 structural identity 62
 structural symmetry 61
 structurally identical 62
 sub-parameters 15
 suffix, .crw 61
 suppress output 70
 sym. Gauss-Seidel / CG 103

T

term_coarsening 57
 termination criteria, coarsening process 57
 test vectors 113
 threads 74
 thread-safe 74, 109
 trailing zeroes 15
 transfer operators 10
 treatment of positive entries 54

truncation of Galerkin 13, 63
 truncation of interpolation 13, 63
 two-level cycle 11

U

u 24, 46
 $u_{(i)}$ 19
 $u_{[n]}$ 18
 underrelaxation, Jacobi 64
 unknown 17
 unknown-based approach 18, 27
 user interface, coarsest-level solver 121
 user interface, cycle number and residual 125
 user interface, input matrix check 123
 user interface, license checker 124
 user interface, message routing 125
 user interfaces 121
 Uzawa 28

V

V*-cycle 14
 variable 17
 variable-based approach 17, 27

variable-to-point pointer, ip 22, 26
 variable-to-unknown pointer, iu 22, 26
 V-cycle 11, 35
 VV-cycle 35

W

W*-cycle 14
 w_{avrge} 43
 $w_{avrge_agg_default}$ 71
 $w_{avrge_default}$ 71
 wall-clock times 78
 warnings 46
 W-cycle 11, 35
 workspace management 16

X

XF-variables 55

Z

zero entries on coarser levels 61
 zero row sum matrix 24